

# CSCI 598H

## Applied Computer Vision for Engineering Systems

### Lecture 3-2: Convolutional Neural Networks and BackPropagation

**Dr. Minghan Li**

**Assistant Professor**

**Computer Science Department**

**Colorado School of Mines**

**Aug/27/2026**



# Administration

- Assignment 1: released, each of you has your own GitHub repo
- Code section: Sept/11 4.30-5.30pm @ Marquez 204
- Office hours: 8am-10am this Friday
- Career Days : Sept/15 next Tuesday, no classes

# CSCI 598H Course Overview



**Weeks 2-4**

**Weeks 5-11**

**Weeks 12-16**

## Module 1

### Deep Learning Basics

- Linear Classifier and Optimization
- **CNNs and BackPropagation**
- RNNs, Attention, Transformers

## Module 2

### Modern Visual Intelligence

- Perceiving and understanding the visual world
- Generative models and foundation models
- Geometry, 3D vision and spatial understanding

## Module 3

### Engineering Systems

- Robotics
- Earth science
- Medical imaging
- Energy

# Recap

**W3: Neural Networks and BackPropagation**

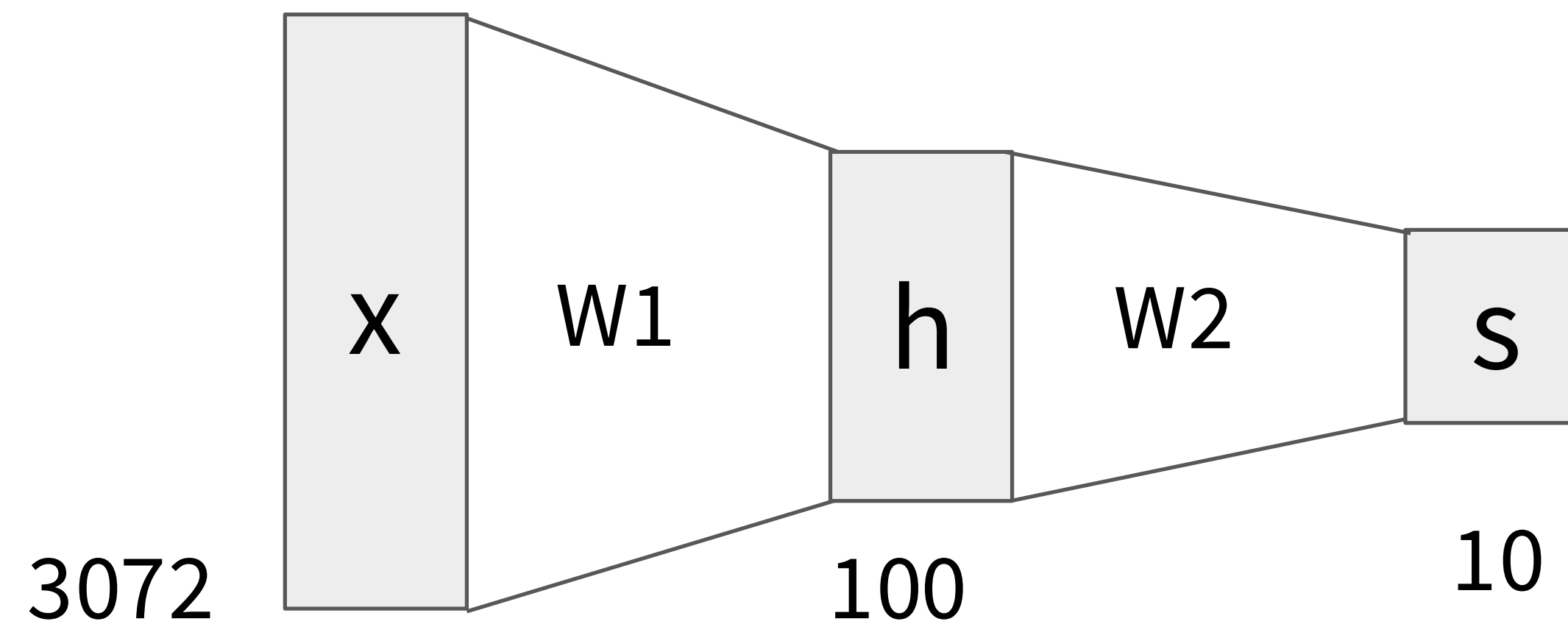
# Last time: Neural Networks

Linear score function:

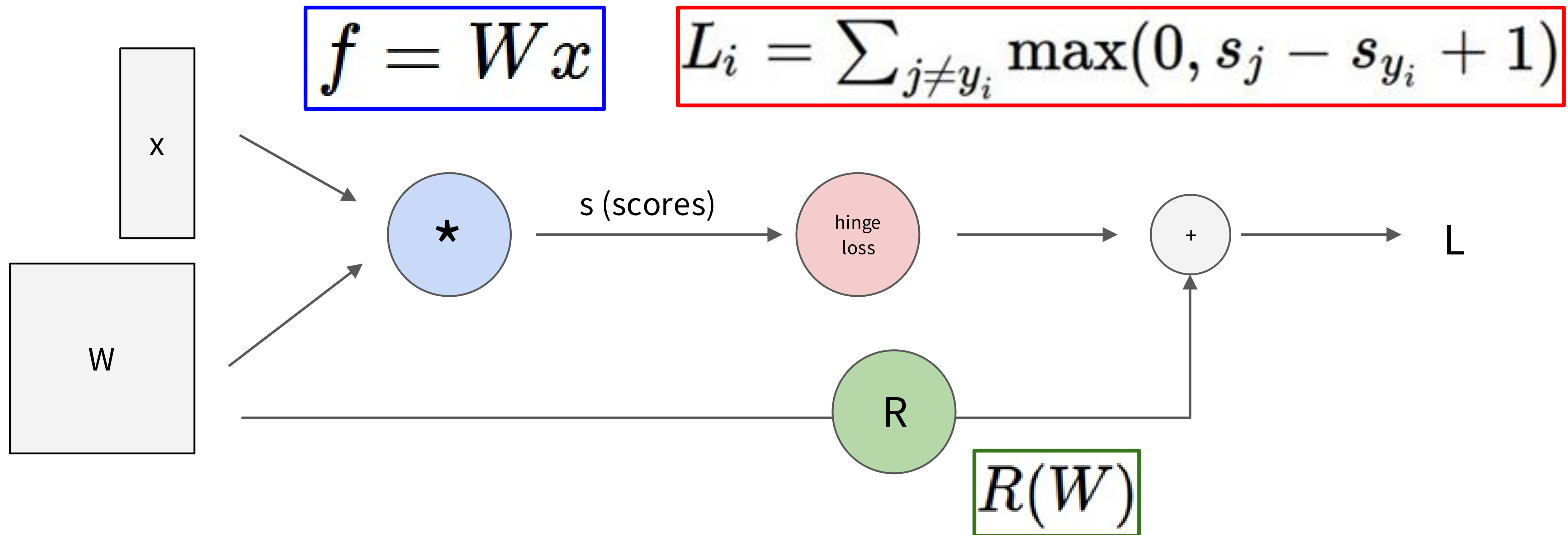
$$f = Wx$$

2-layer Neural Network

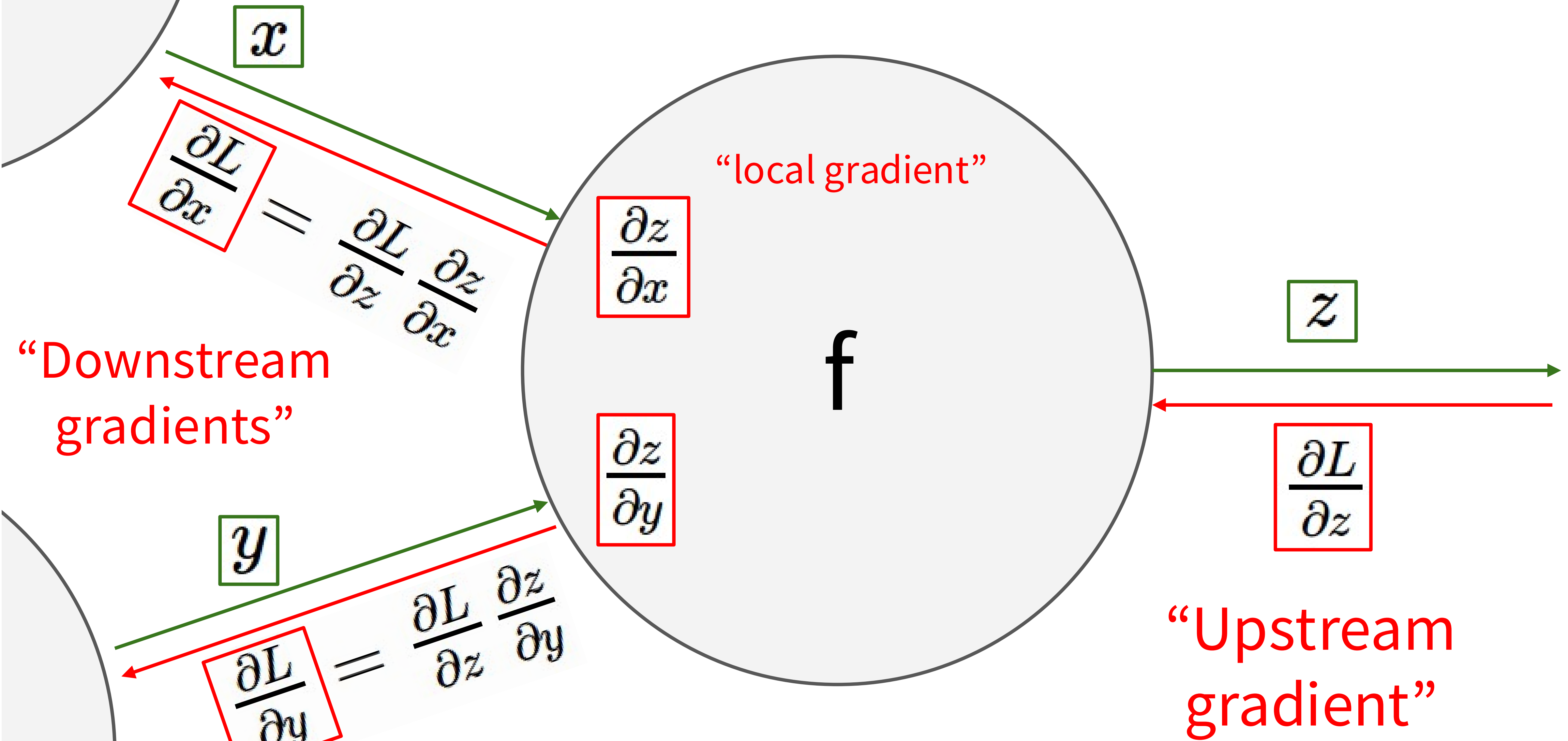
$$f = W_2 \max(0, W_1 x)$$



# Last time: Computation Graph



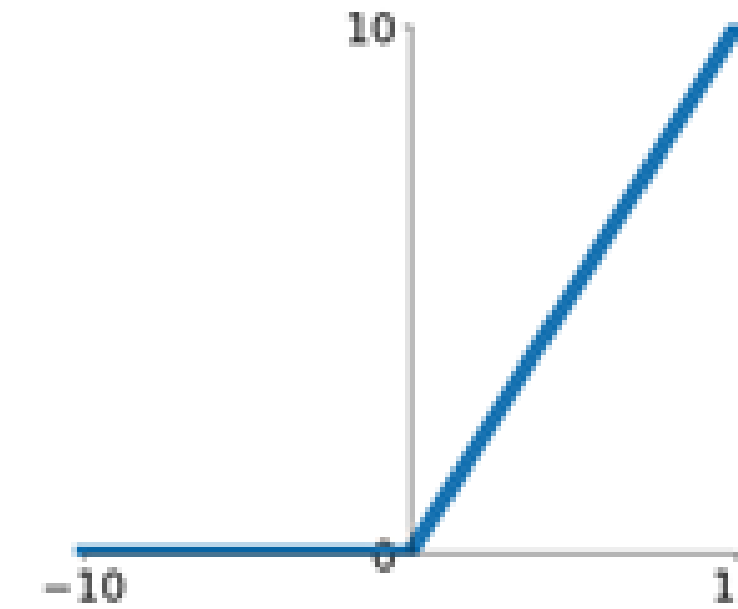
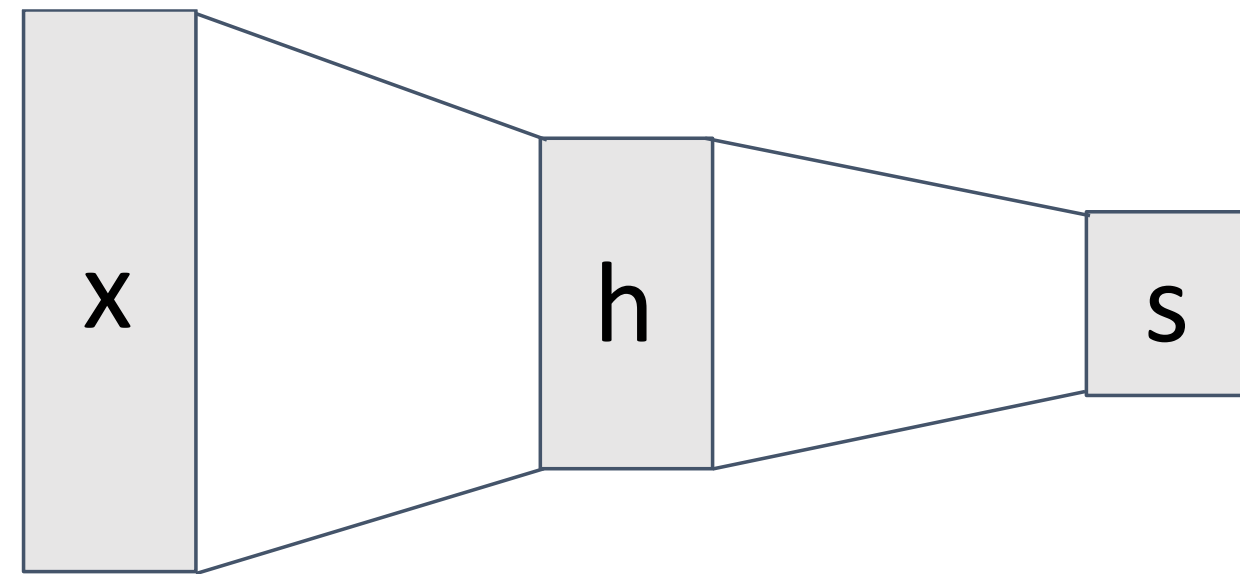
# Last time: Backpropagation



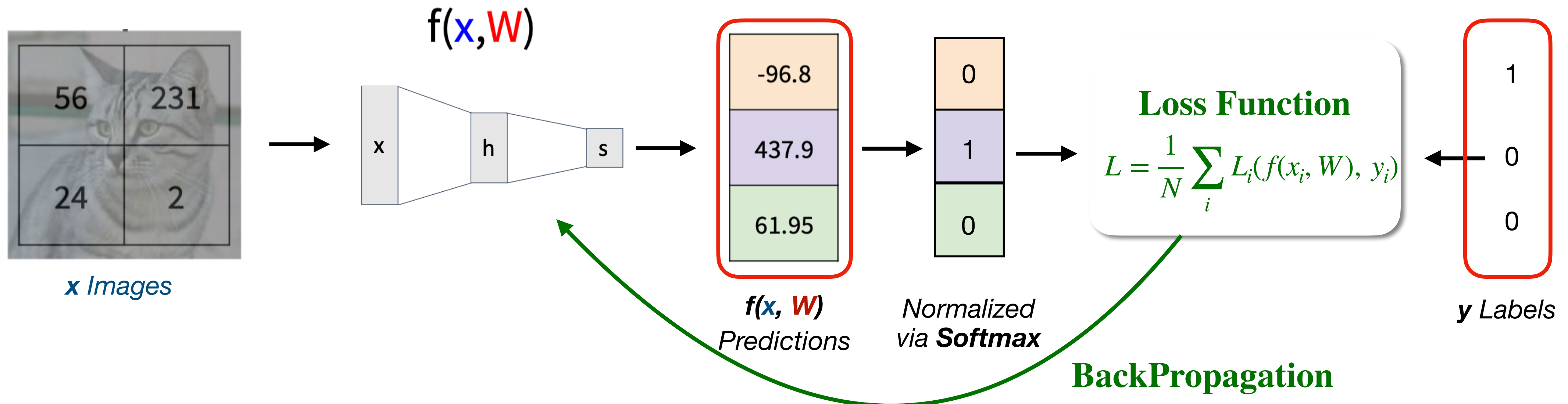
## Fully-Connected Layer

## Activation Function

We have  
already  
seen these



## Model Training with fully-connected layers



# Last Time: Neural Networks

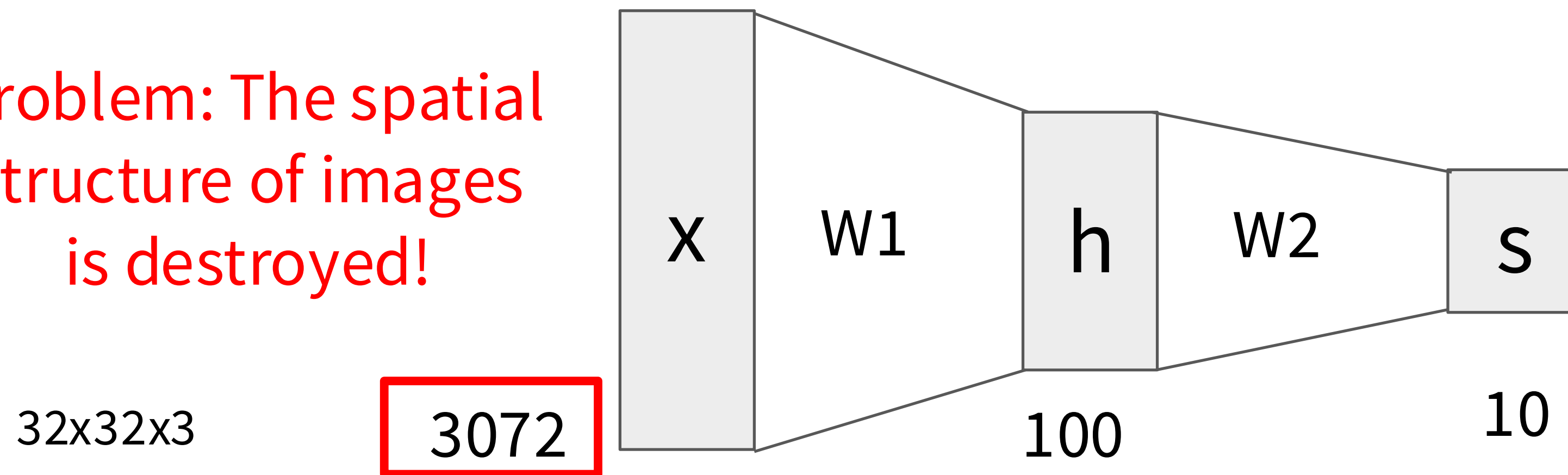
Linear score function:

$$f = Wx$$

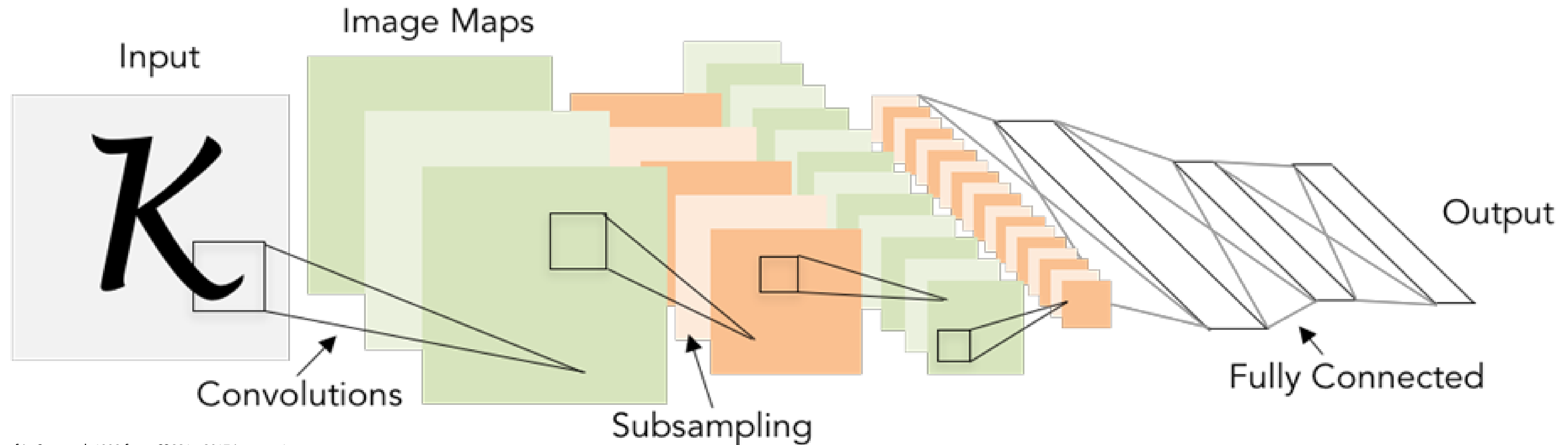
2-layer Neural Network

$$f = W_2 \max(0, W_1 x)$$

Problem: The spatial structure of images is destroyed!



# *Today: Convolutional Neural Networks*



[https://github.com/d2l-ai/d2l-en/tree/master/chapter\\_convolutional-neural-networks](https://github.com/d2l-ai/d2l-en/tree/master/chapter_convolutional-neural-networks)

**Additional materials and code**

# Convolutional Neural Networks

How to build CNNs?

**Layers in CNNs**

Activation Functions

CNN Architecture

How to train CNNs?

Weight Initialization

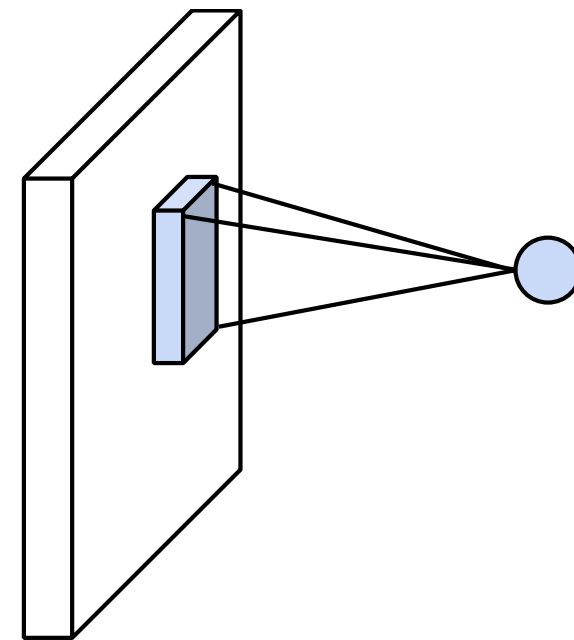
Data Preprocessing & Augmentation

Hyperparameter Selection

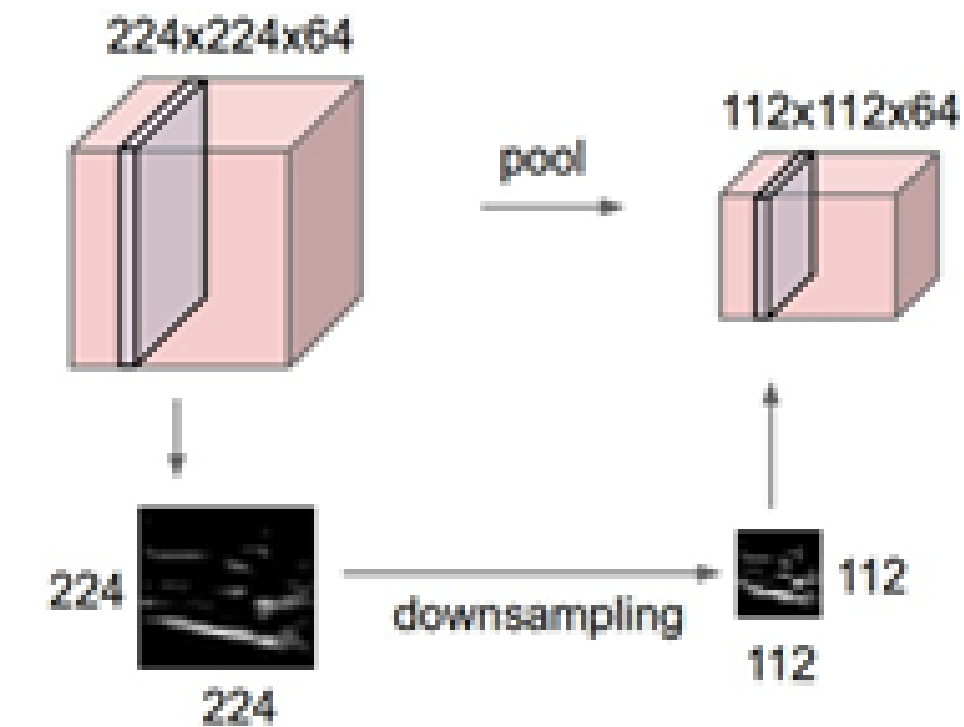
# Today Convolutional Networks

Today: Image-specific operators

## Convolution Layer



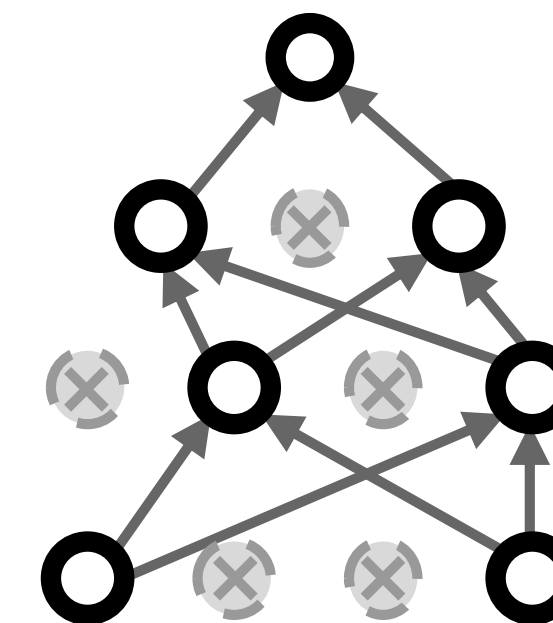
## Pooling Layer



## Normalization Layers

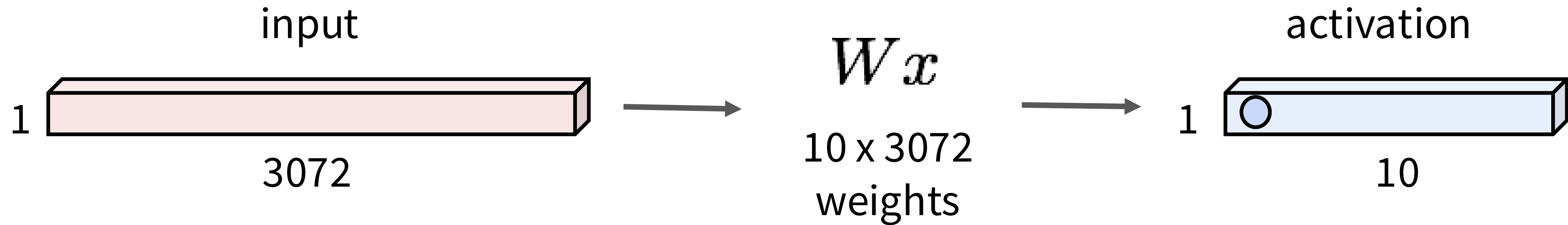
$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

## Dropout (sometimes)



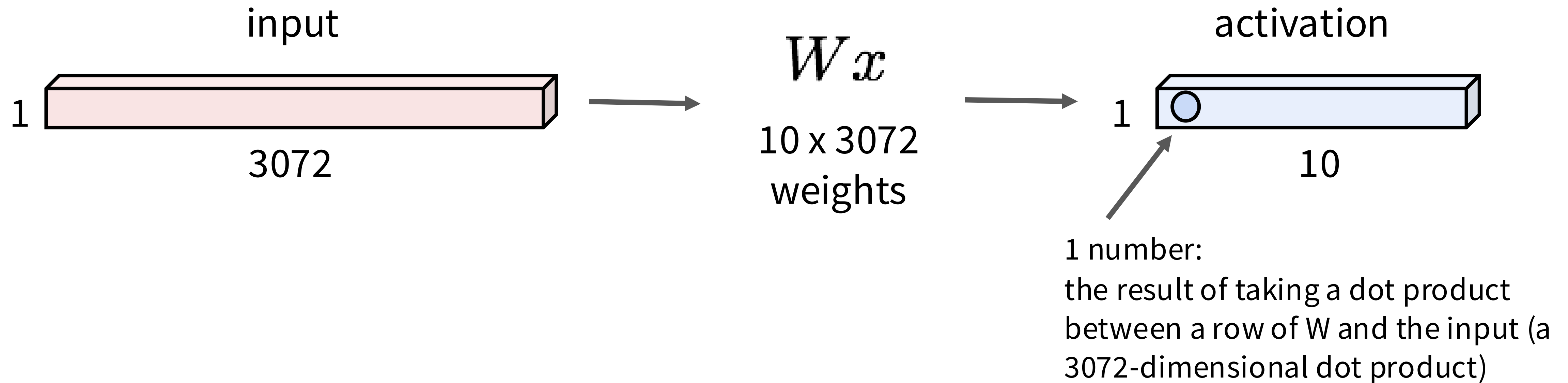
# Recap: Fully Connected Layer

32x32x3 image -> stretch to 3072 x 1

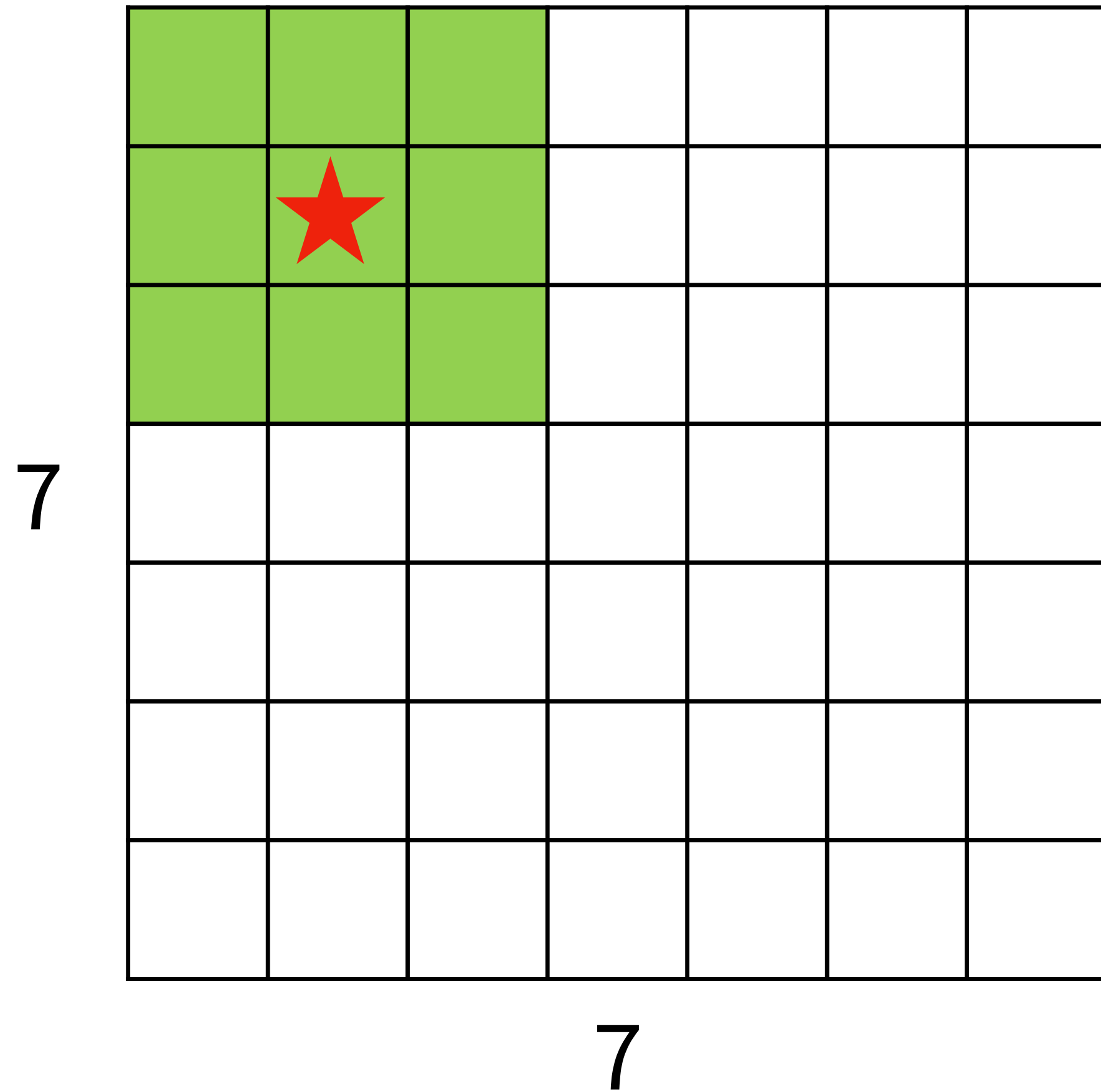


# Fully Connected Layer

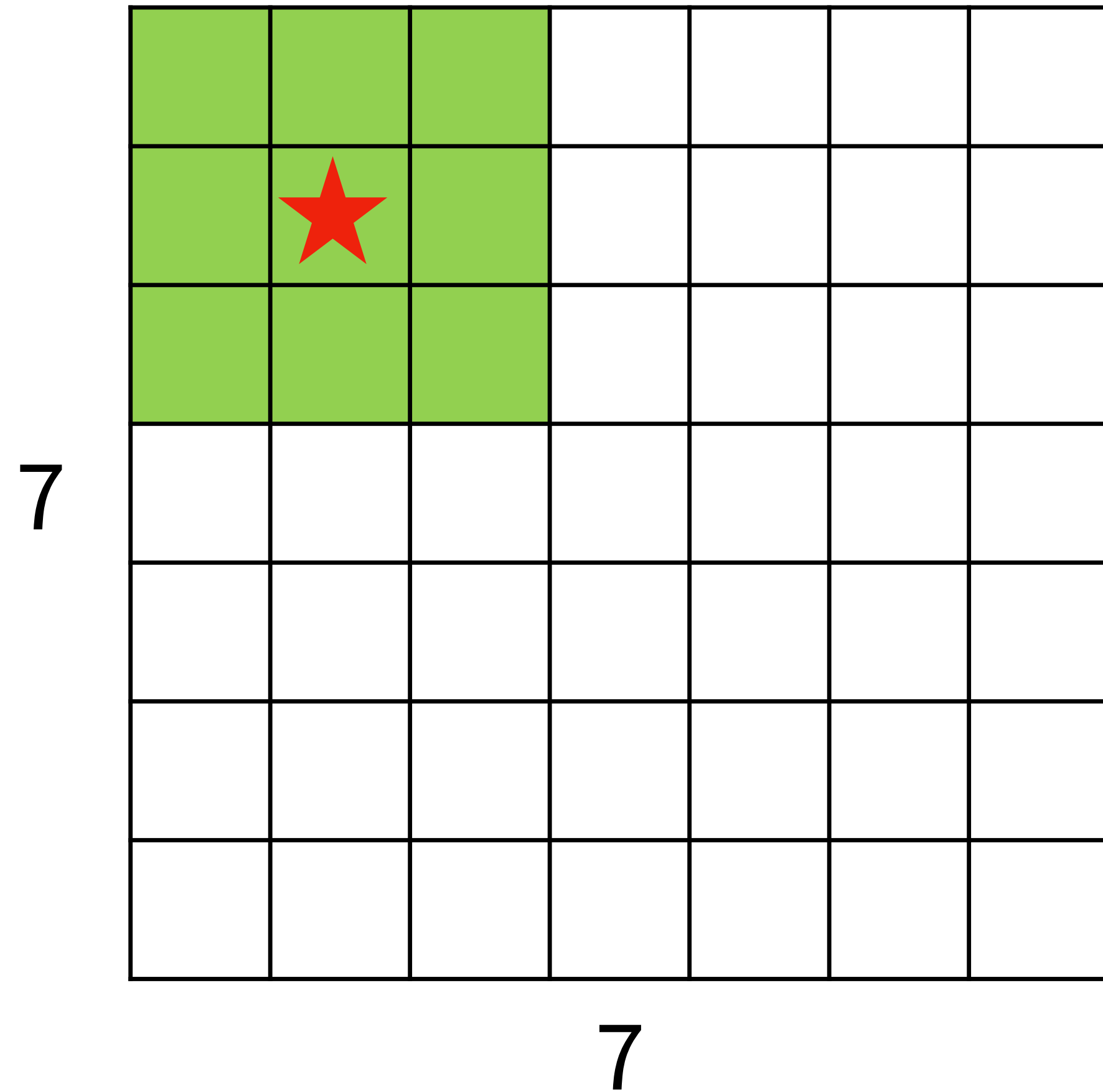
32x32x3 image -> stretch to 3072 x 1



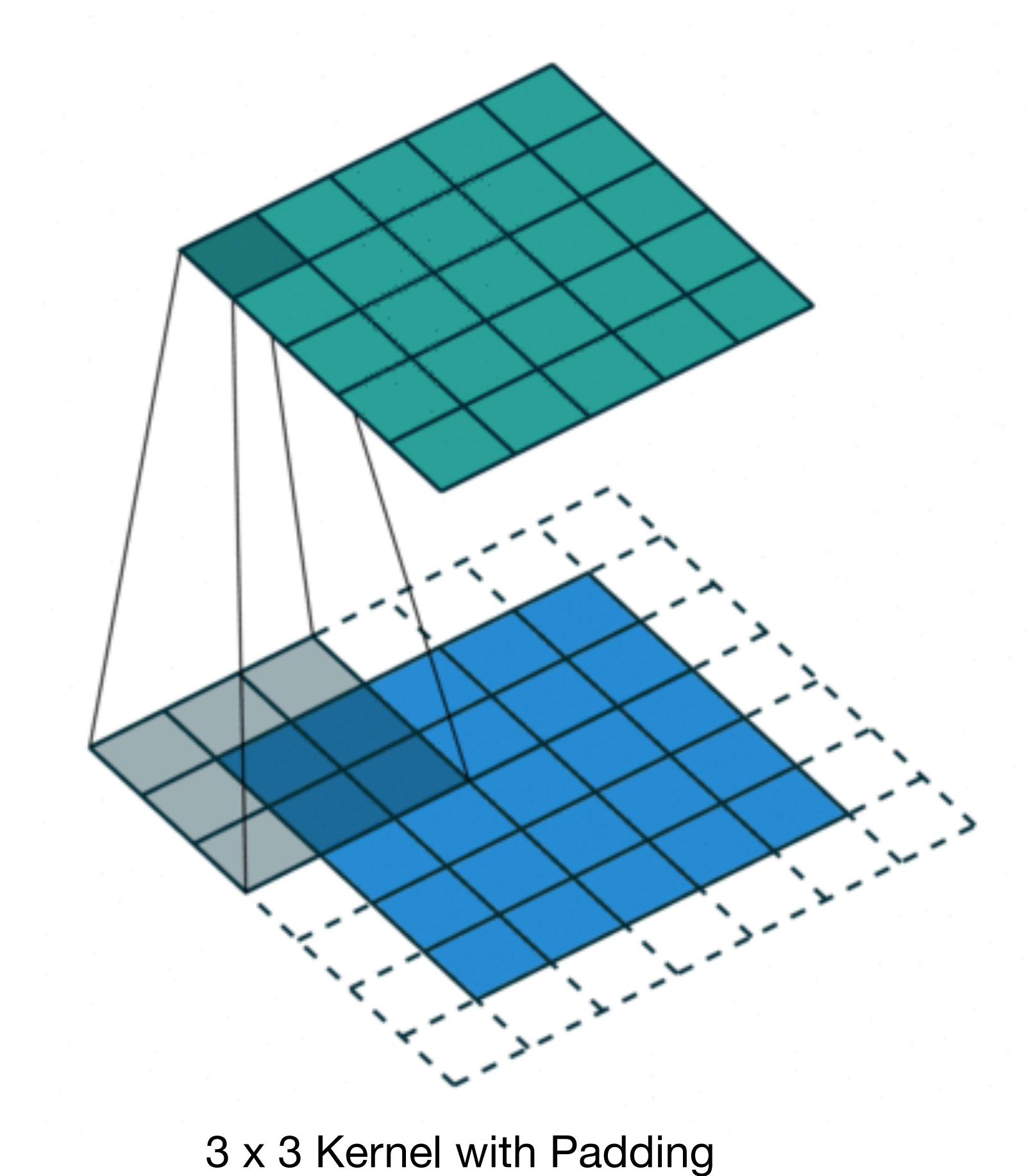
# Convolution: spatial relationship



# Convolution: spatial relationship



# Animations

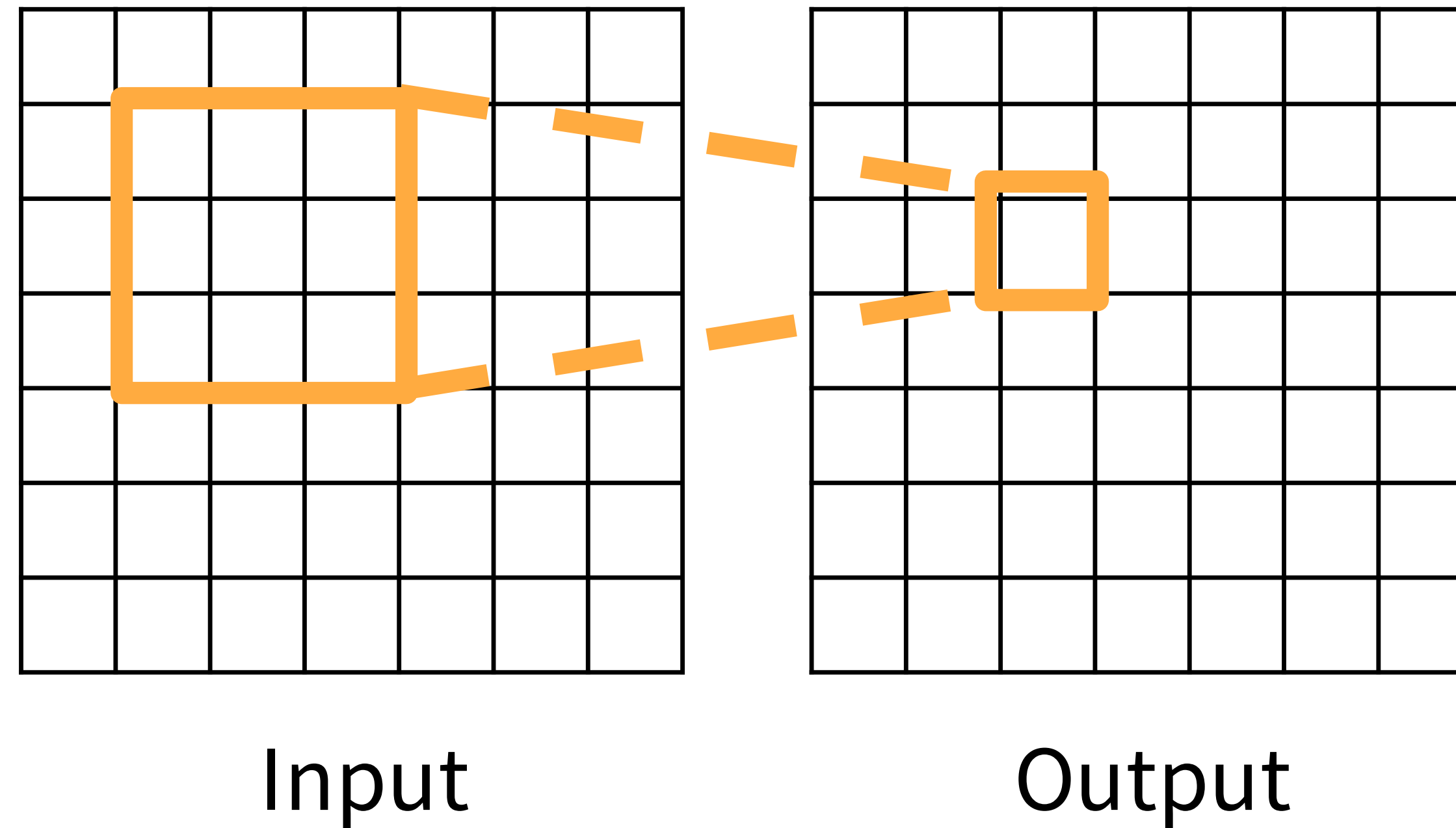


3 x 3 Kernel with Padding

[https://github.com/vdumoulin/conv\\_arithmetic](https://github.com/vdumoulin/conv_arithmetic)

# Receptive Fields

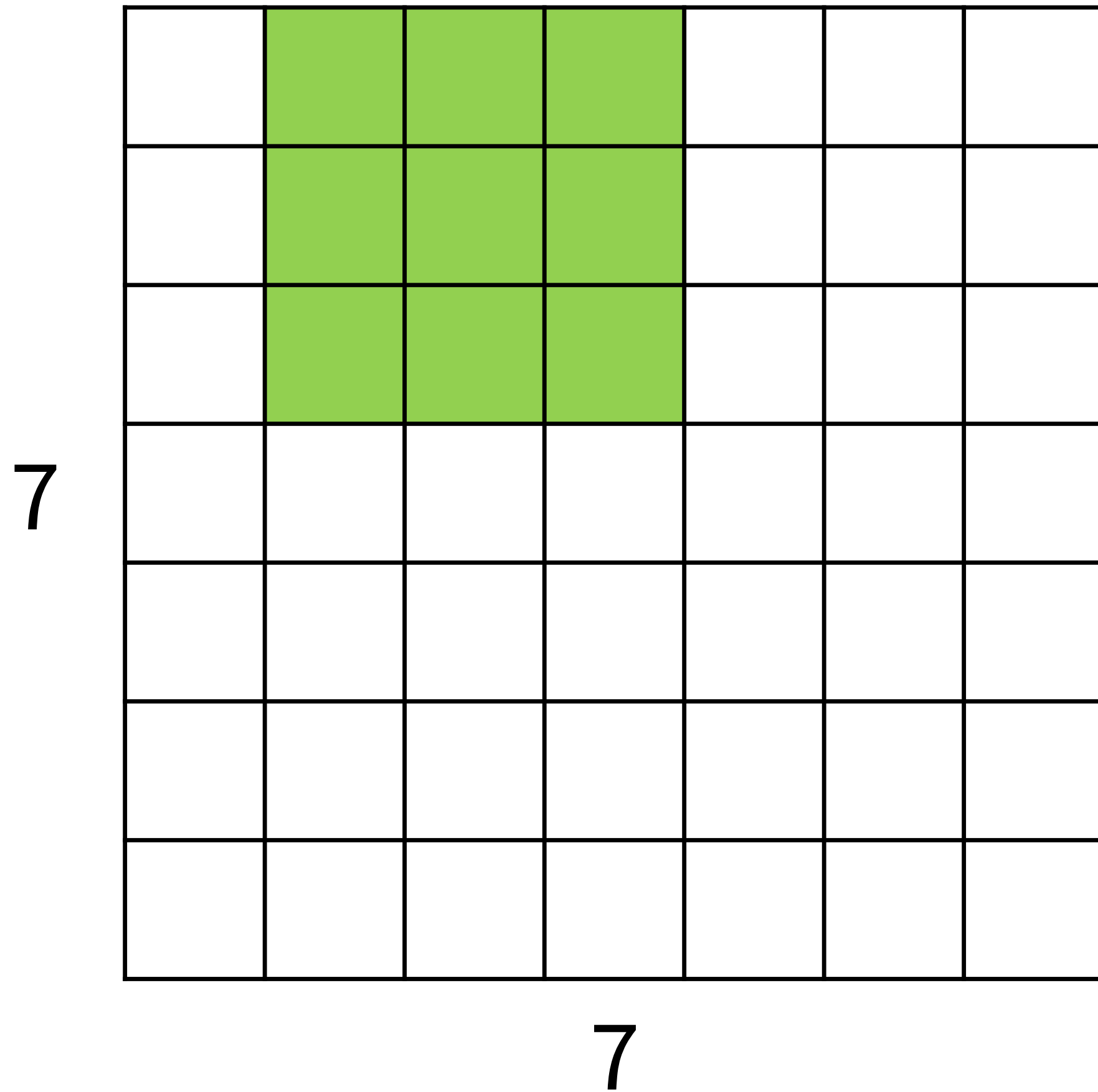
For convolution with **kernel size K**, each element in the output depends on a  $K \times K$  receptive field in the input



# Convolution: Spatial Dimensions

# Input: 7x7

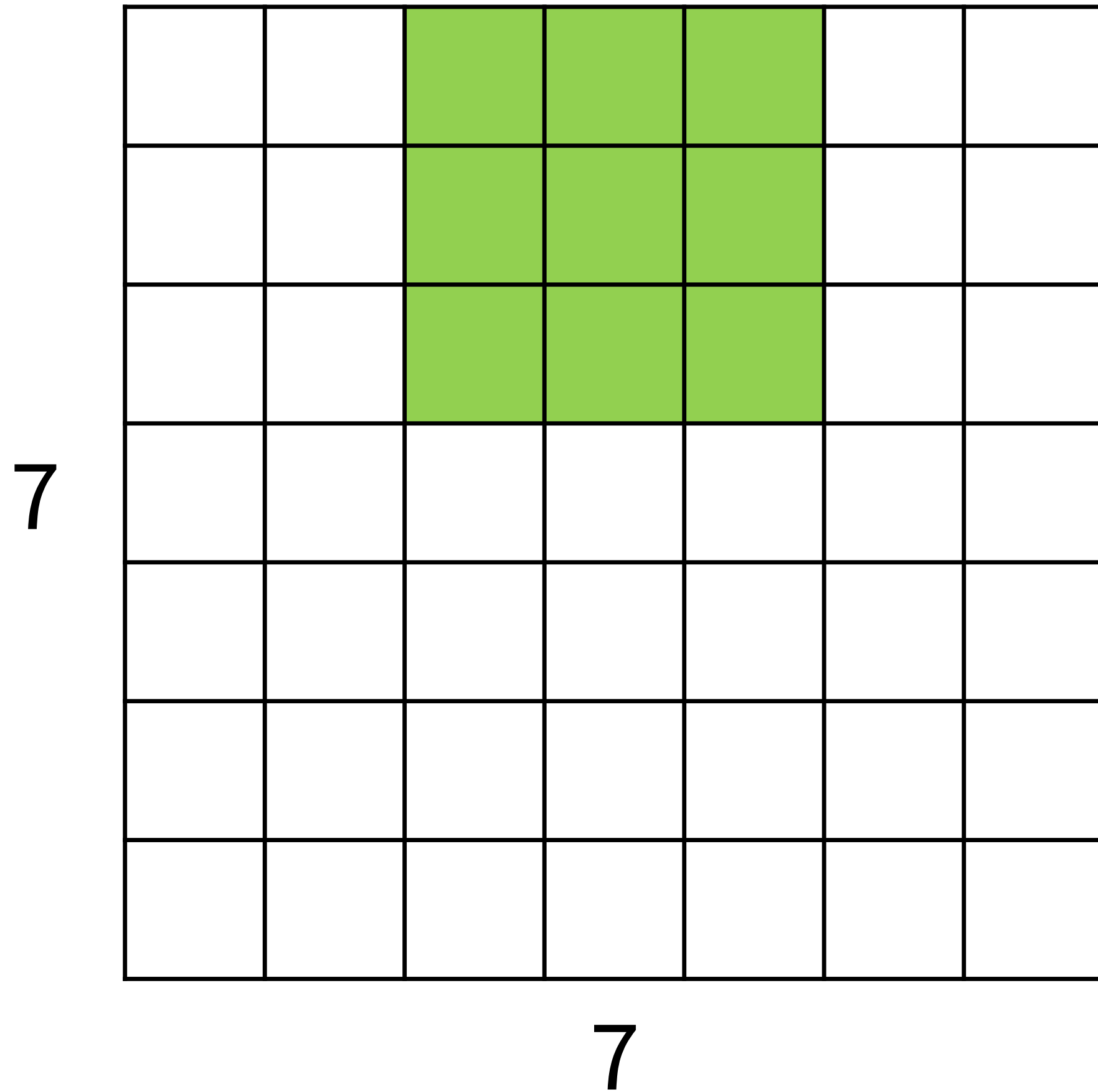
# Filter: 3x3



# Convolution: Spatial Dimensions

# Input: 7x7

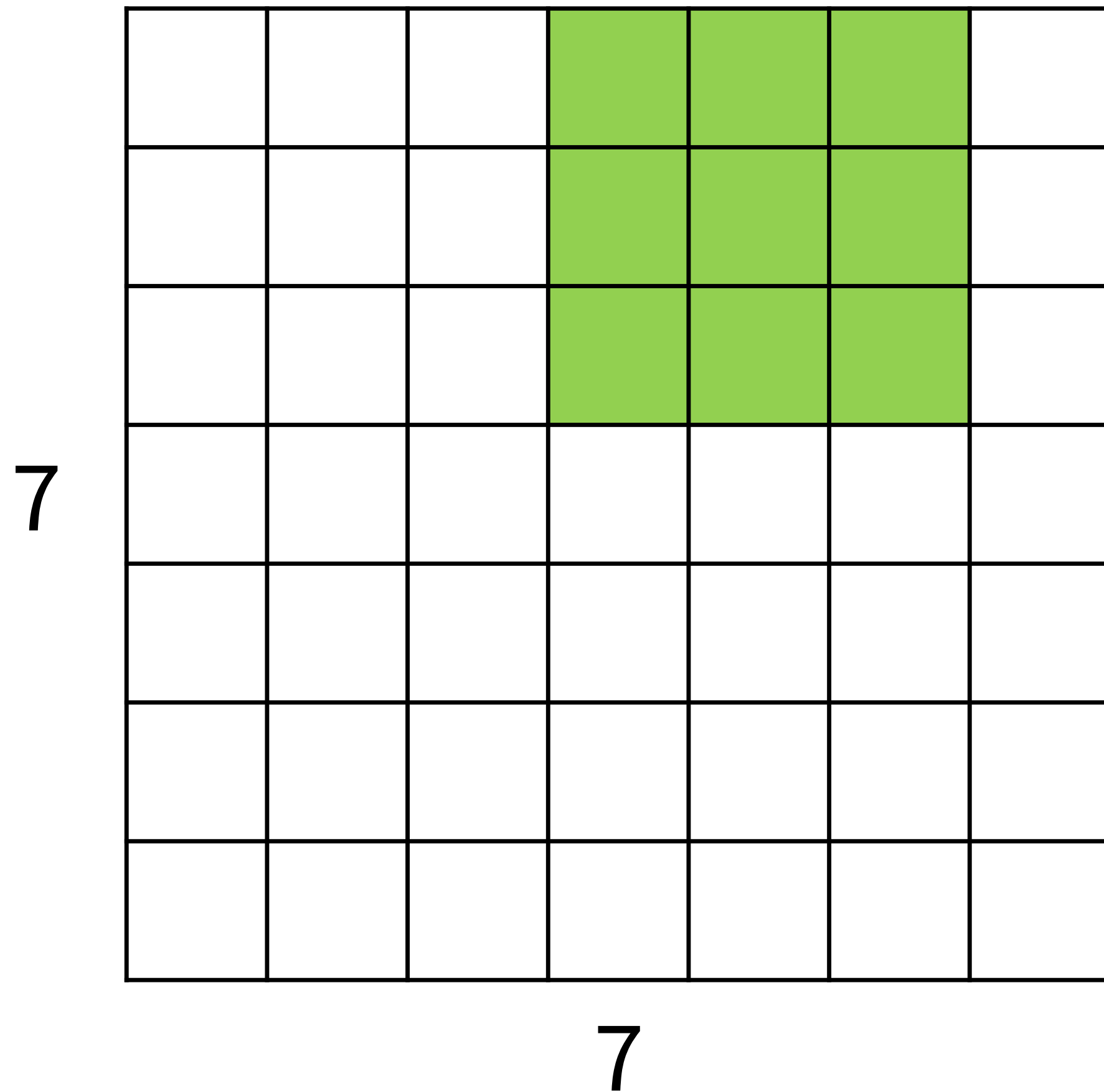
# Filter: 3x3



# Convolution: Spatial Dimensions

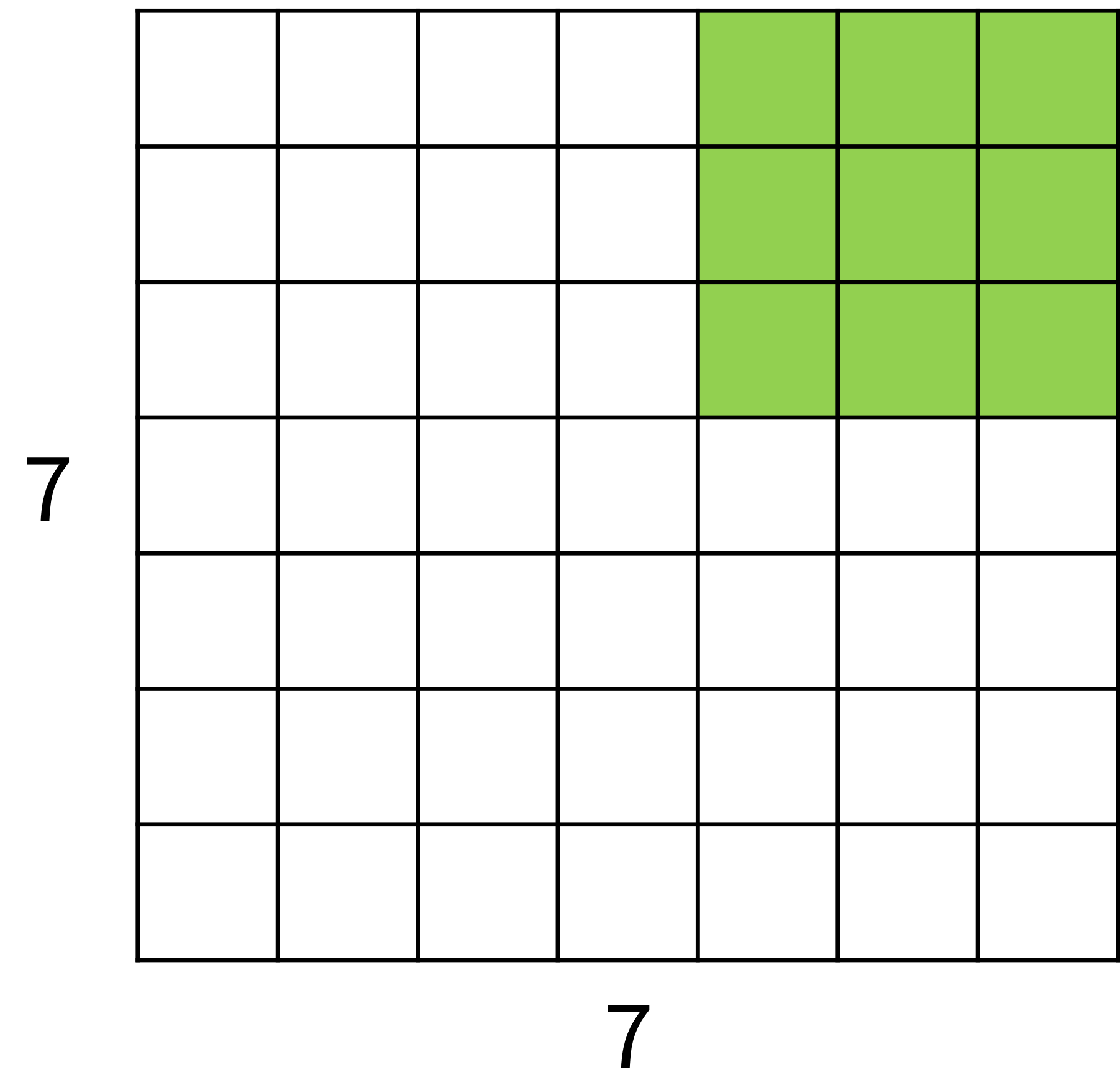
Input: 7x7

Filter: 3x3

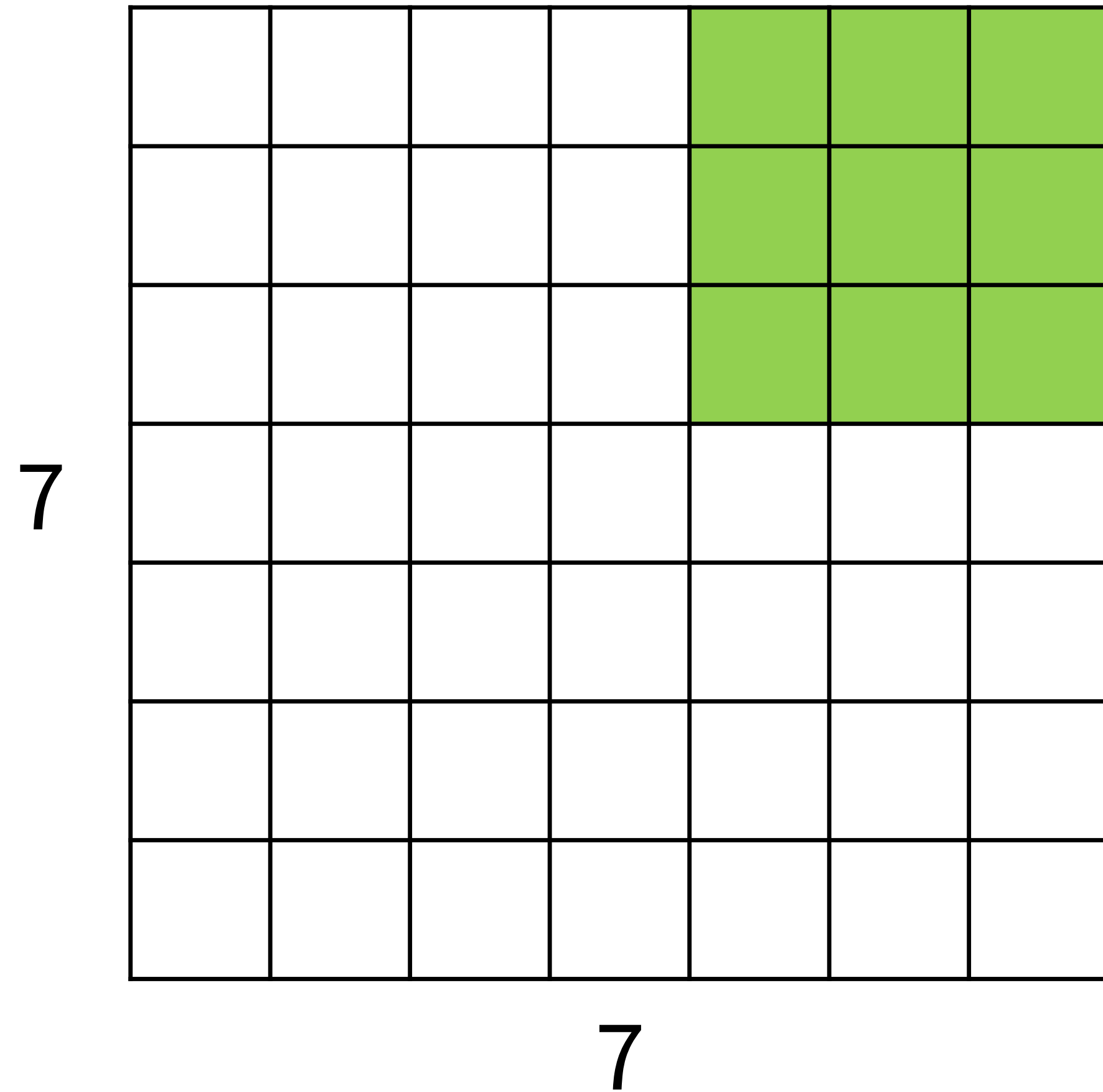


# Convolution: Spatial Dimensions

Input: 7x7  
Filter: 3x3  
Output: 5x5



# Convolution: Spatial Dimensions



Input: 7x7

Filter: 3x3

Output: 5x5

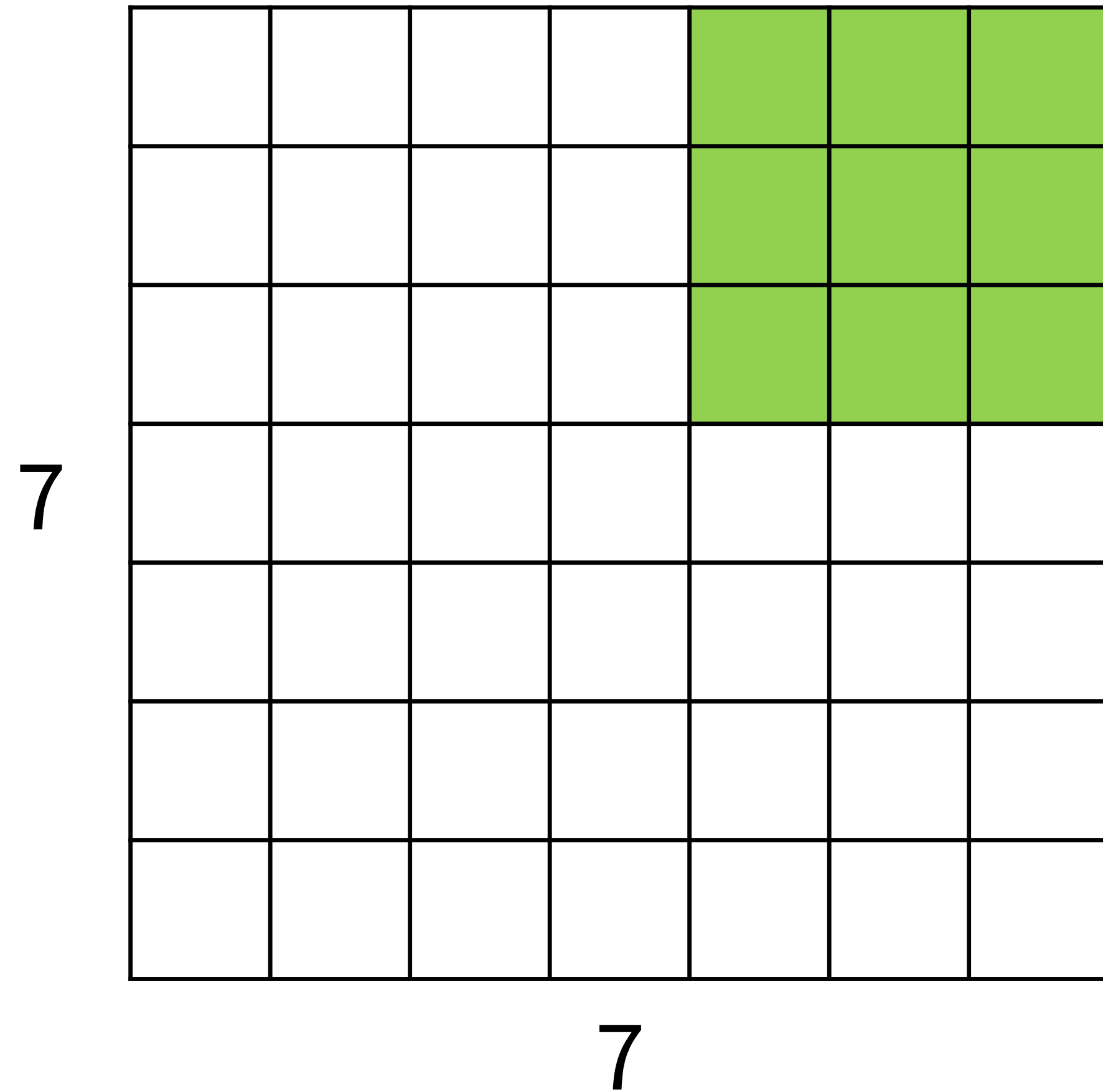
In general

Input:  $W$

Filter:  $K$

Output:  $W - K + 1$

# Convolution: Spatial Dimensions



Input: 7x7  
Filter: 3x3  
Output: 5x5

Problem: Feature maps shrink with each layer!

**Q: What's the solution to avoid shrink?**

In general  
Input:  $W$   
Filter:  $K$   
Output:  $W - K + 1$

**Padding**

# Convolution: Spatial Dimensions

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 |   |   |   |   |   |   |   | 0 |
| 0 |   |   |   |   |   |   |   | 0 |
| 0 |   |   |   |   |   |   |   | 0 |
| 0 |   |   |   |   |   |   |   | 0 |
| 0 |   |   |   |   |   |   |   | 0 |
| 0 |   |   |   |   |   |   |   | 0 |
| 0 |   |   |   |   |   |   |   | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Input: 7x7  
Filter: 3x3  
Output: 5x5

In general  
Input:  $W$   
Filter:  $K$   
Padding:  $P$   
Output:  $W - K + 1 + 2P$

Problem: Feature maps shrink with each layer!

Solution: Add **padding** around the input before sliding the filter

# Convolution: Spatial Dimensions

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 |   |   |   |   |   |   |   | 0 |
| 0 |   |   |   |   |   |   |   | 0 |
| 0 |   |   |   |   |   |   |   | 0 |
| 0 |   |   |   |   |   |   |   | 0 |
| 0 |   |   |   |   |   |   |   | 0 |
| 0 |   |   |   |   |   |   |   | 0 |
| 0 |   |   |   |   |   |   |   | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Input: 7x7

Filter: 3x3

Output: 5x5

## Padding

Common setting:

$$P = (K - 1) / 2$$

Means output has  
same size as input

In general

Input: W

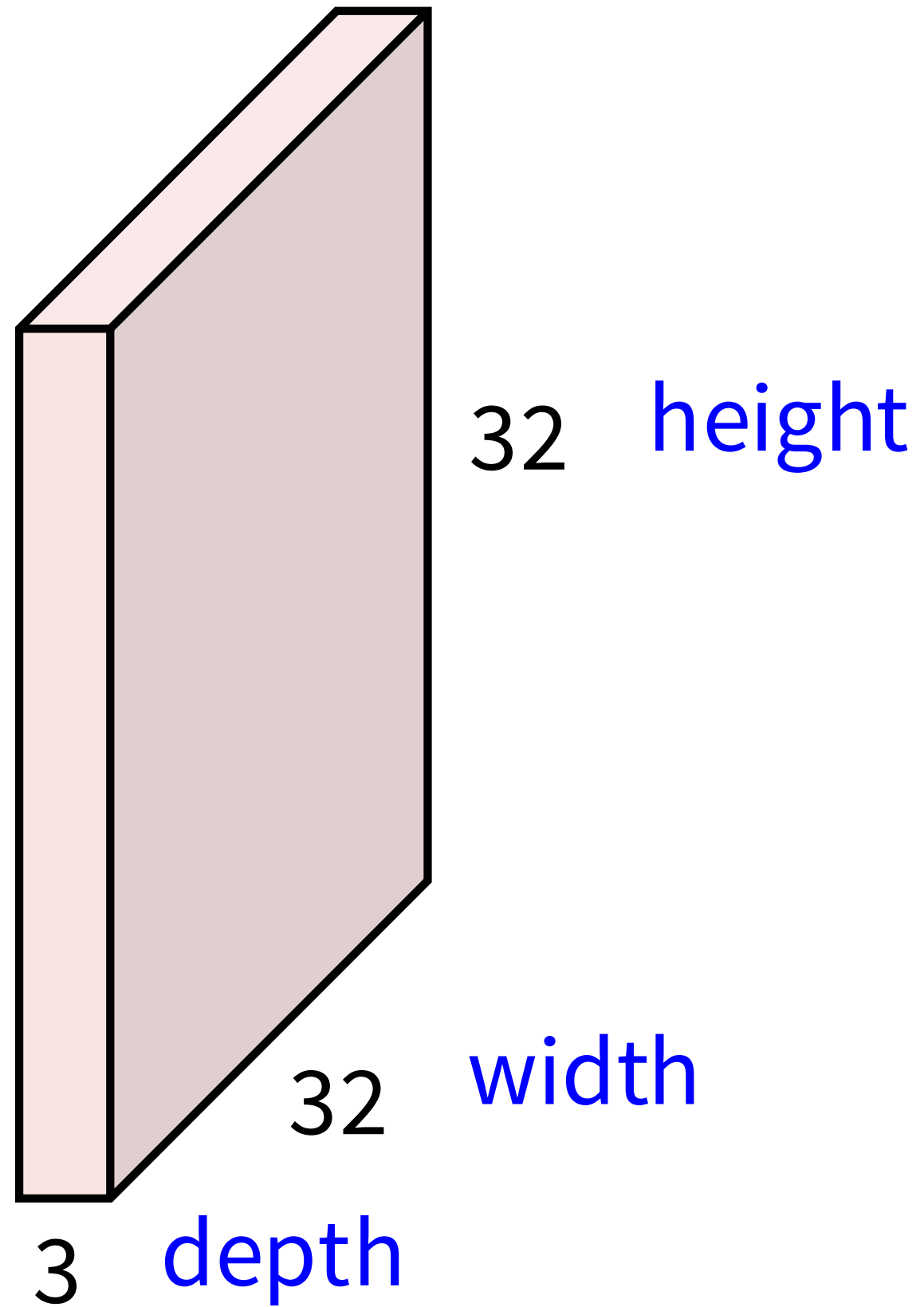
Filter: K

Padding: P

Output:  $W - K + 1 + 2P$

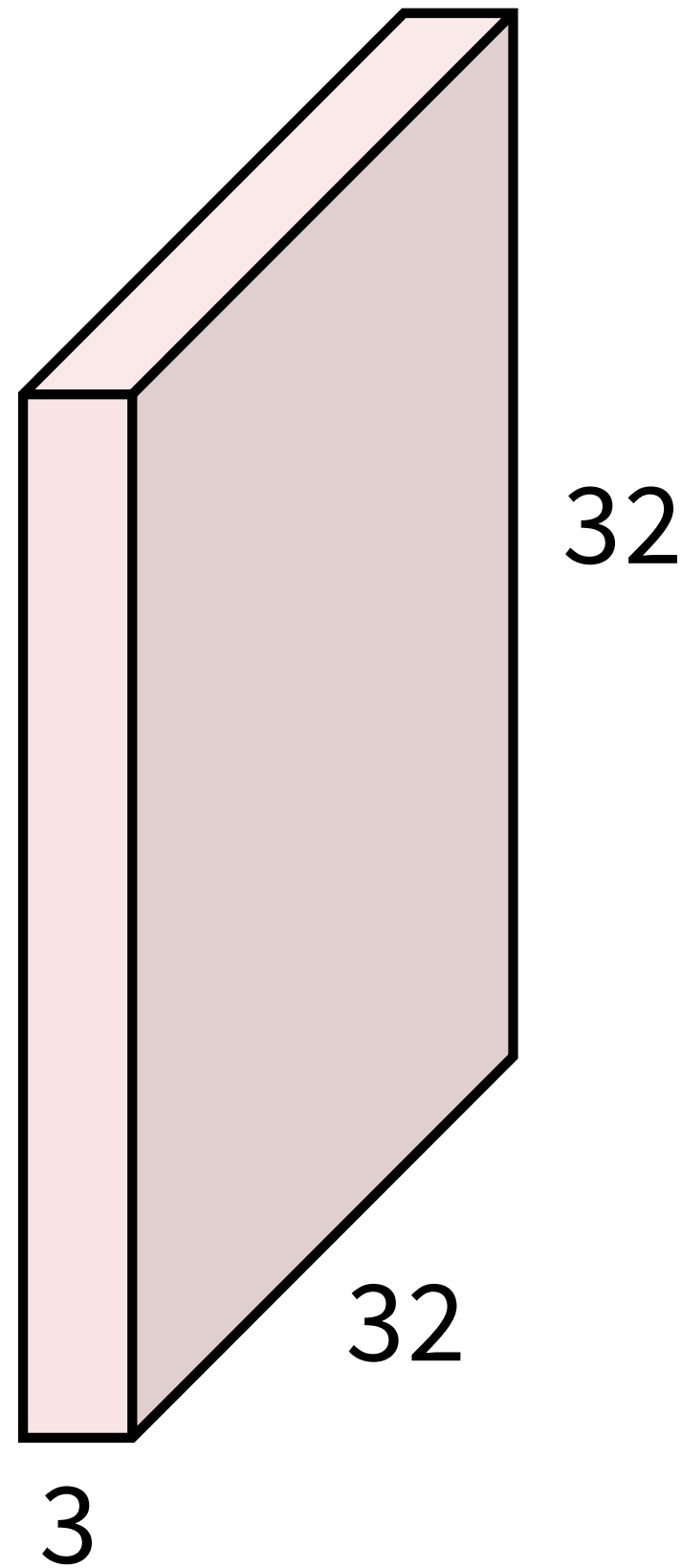
# Convolution Layer

32x32x3 image -> preserve spatial structure

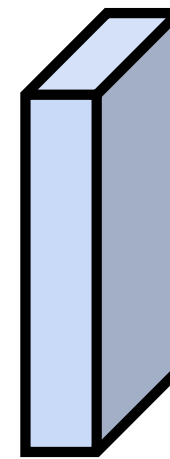


# Convolution Layer

32x32x3 image



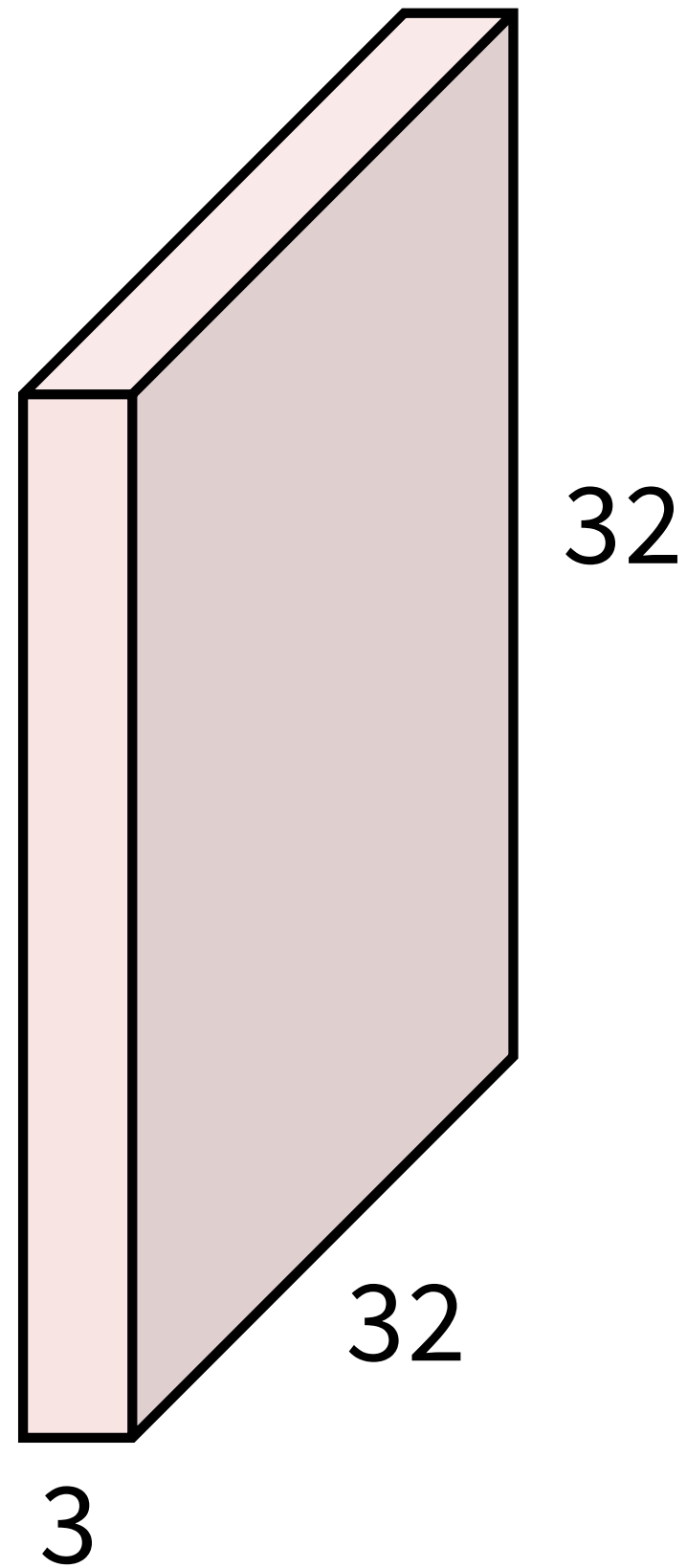
5x5x3 filter



Convolve the filter with the image  
i.e. “slide over the image spatially,  
computing dot products”

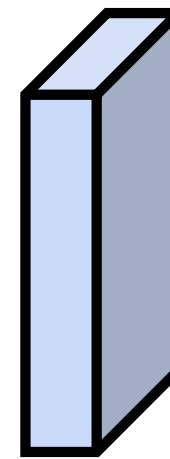
# Convolution Layer

32x32x3 image



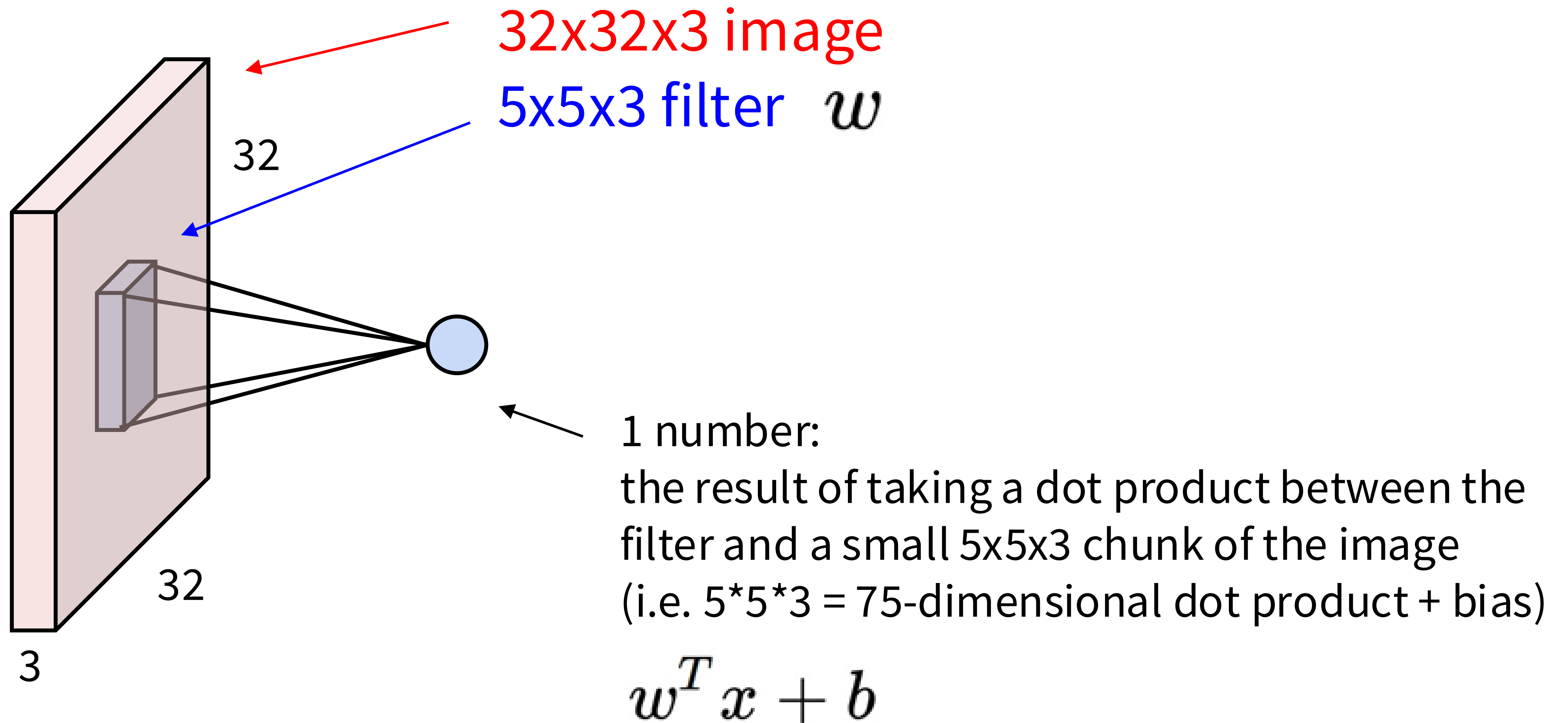
Filters always extend the full depth of the input volume

5x5x3 filter

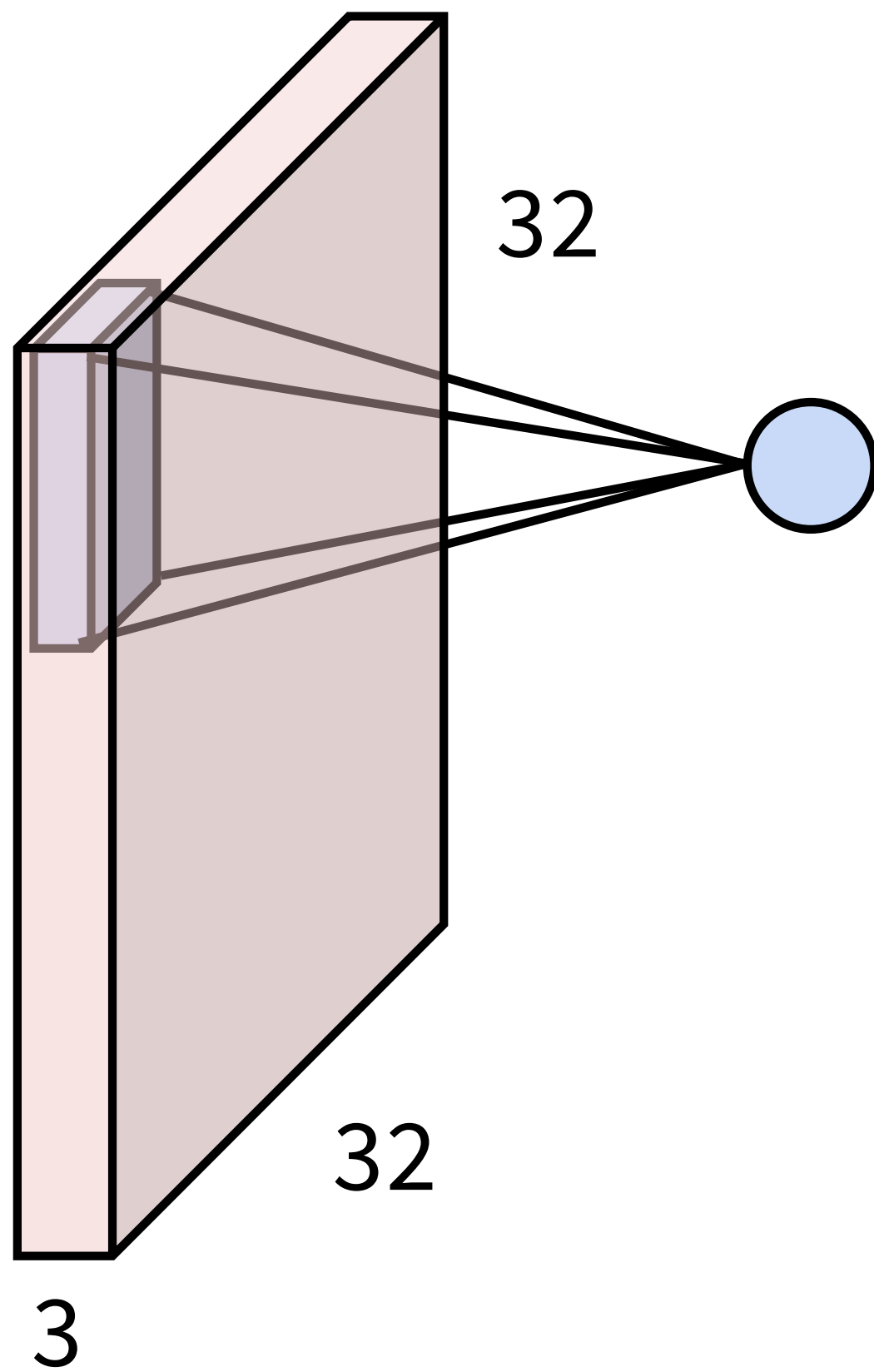


Convolve the filter with the image  
i.e. “slide over the image spatially,  
computing dot products”

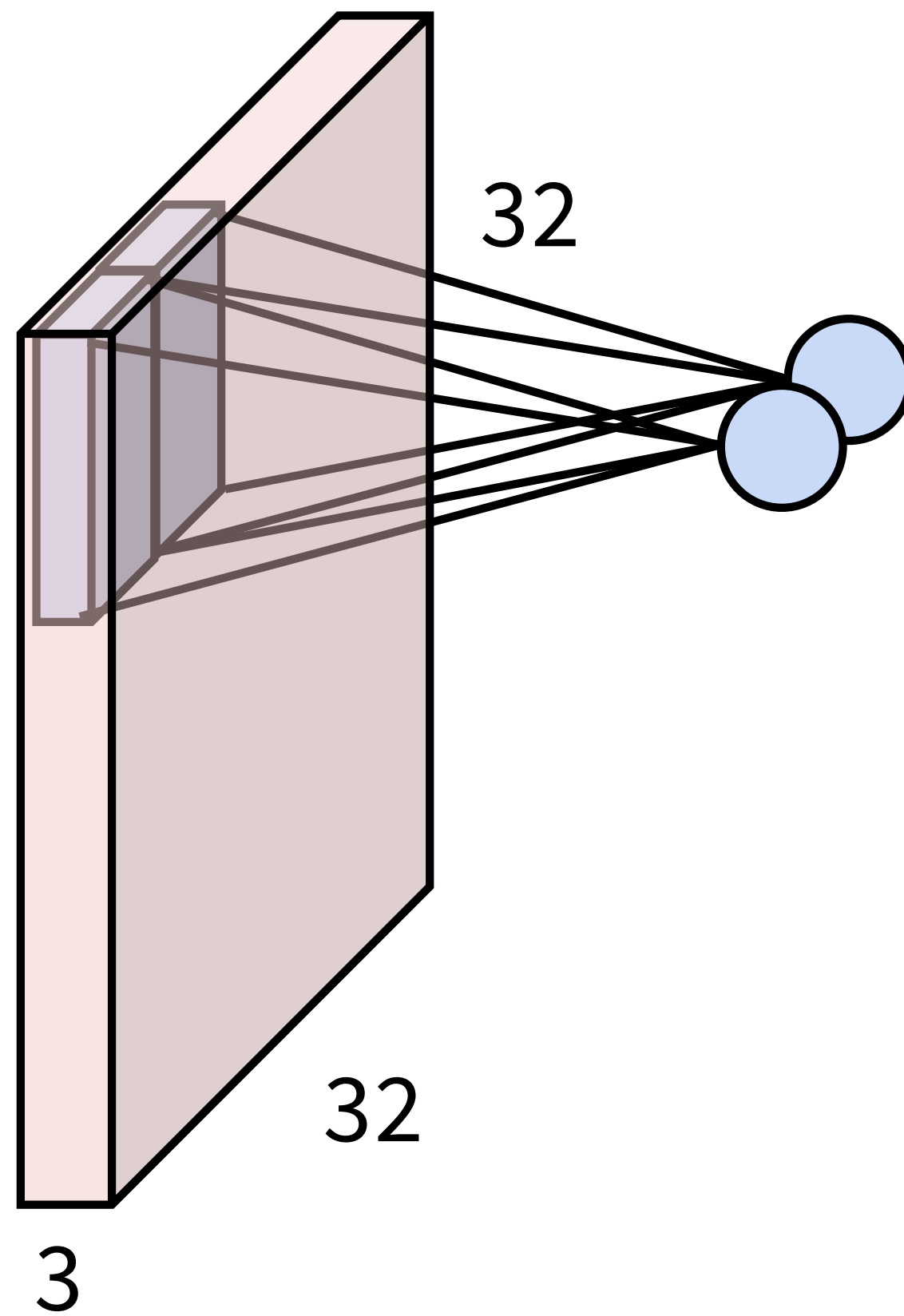
# Convolution Layer



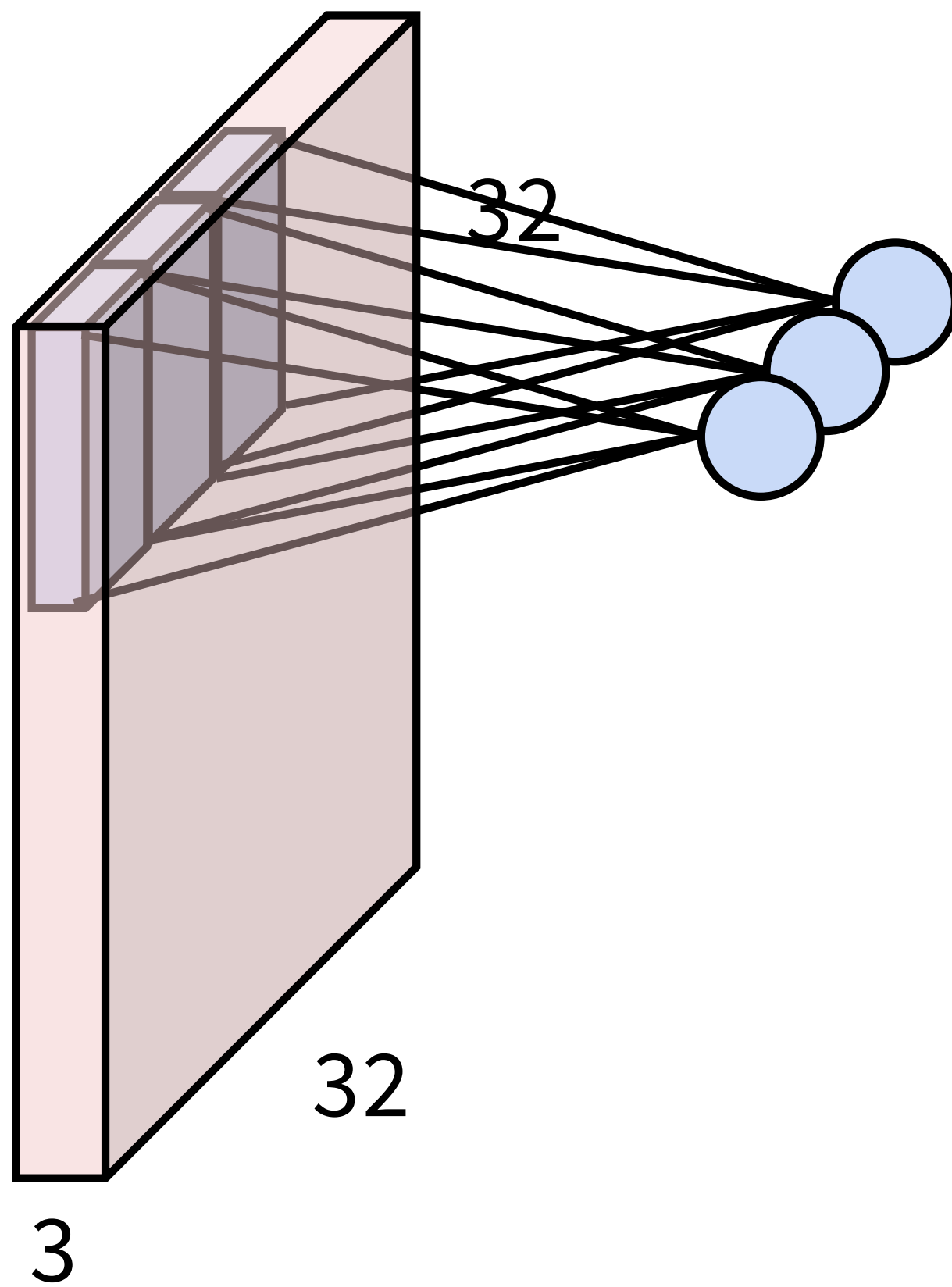
# Convolution Layer



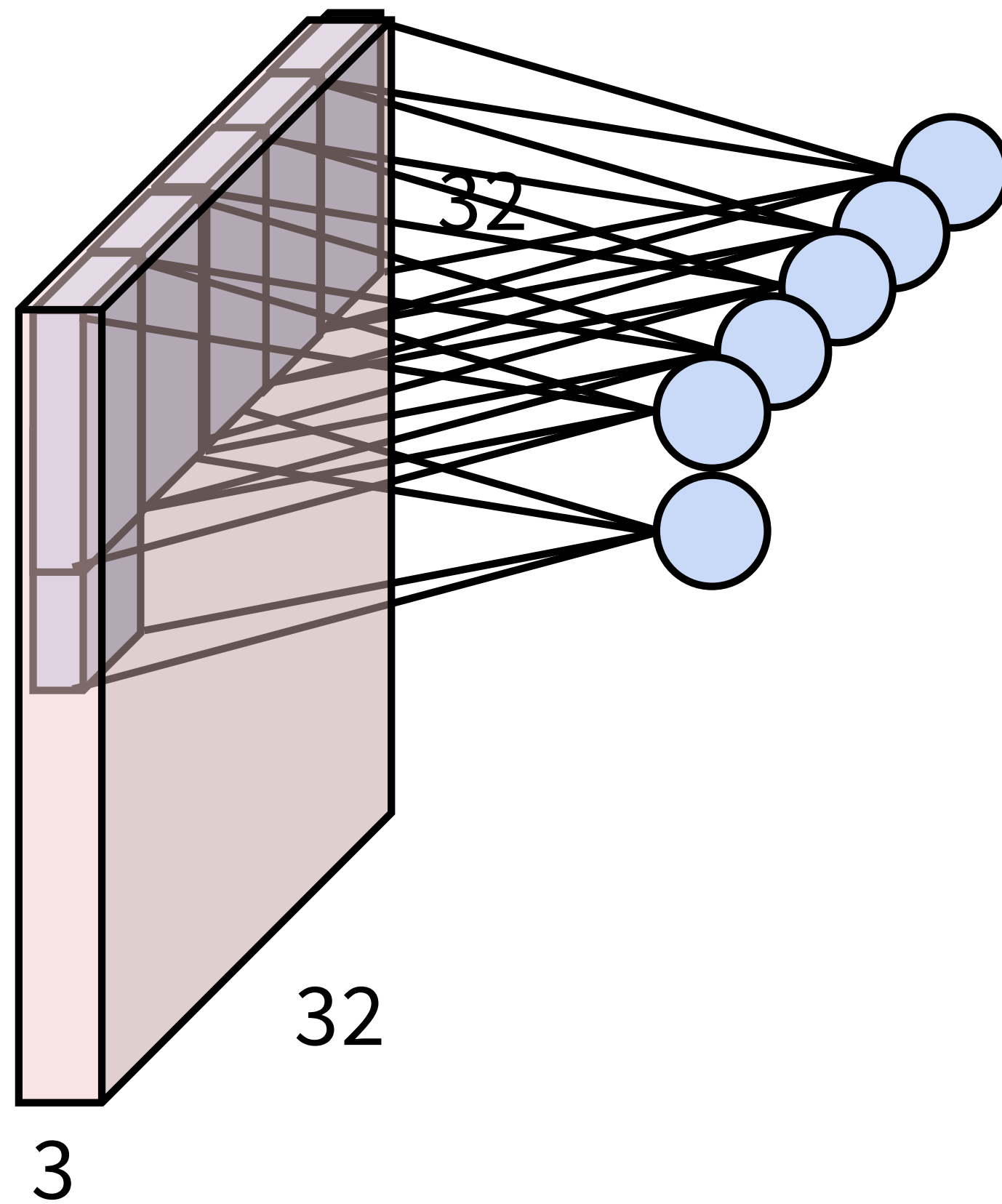
# Convolution Layer



# Convolution Layer



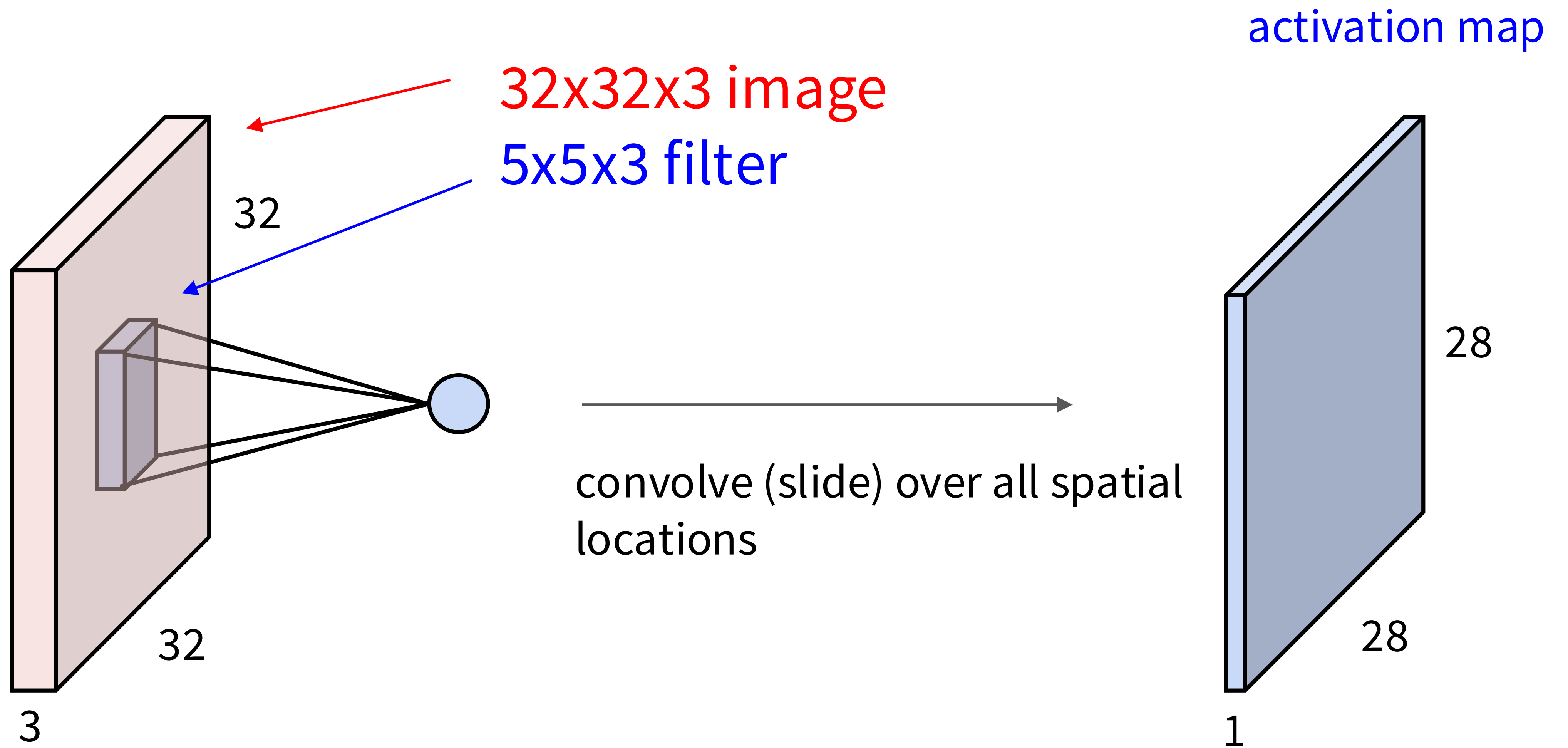
# Convolution Layer



# Convolution Layer

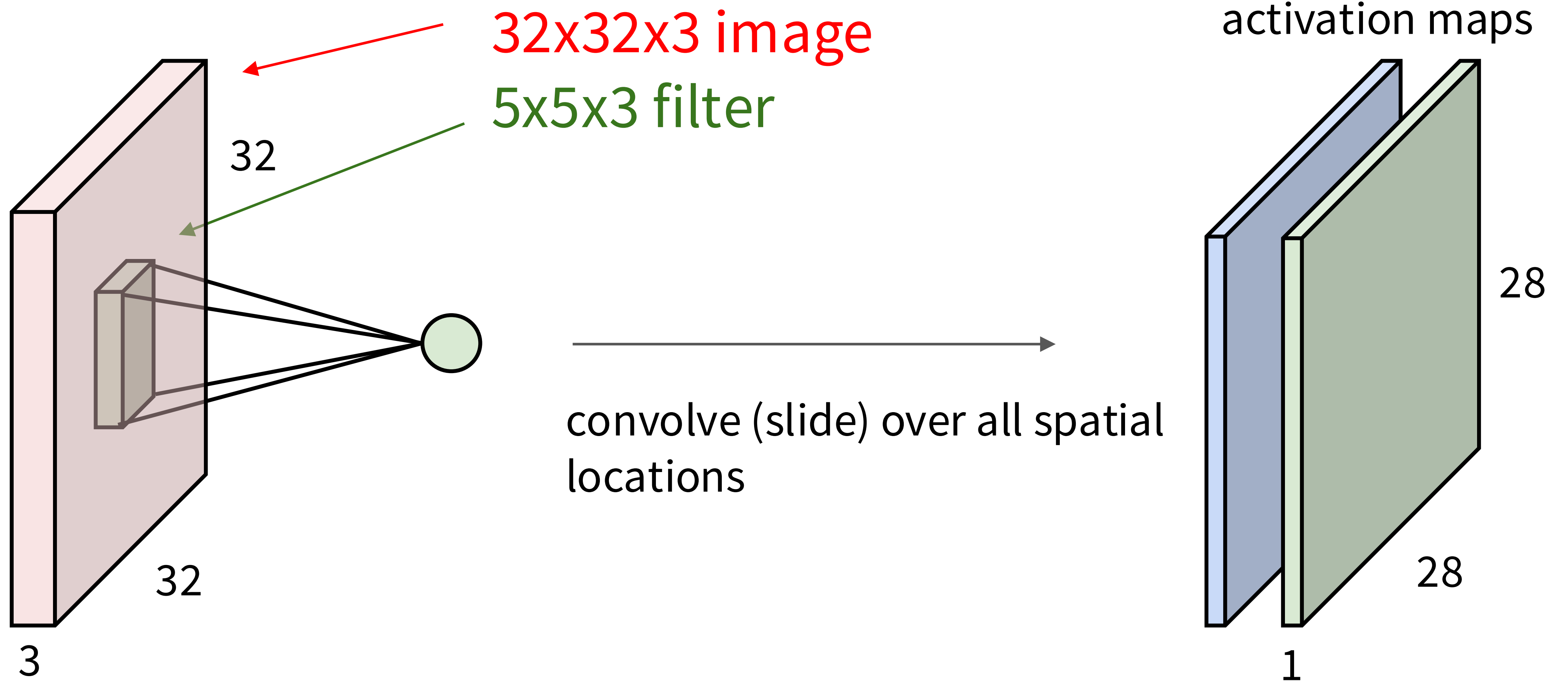


# Convolution Layer



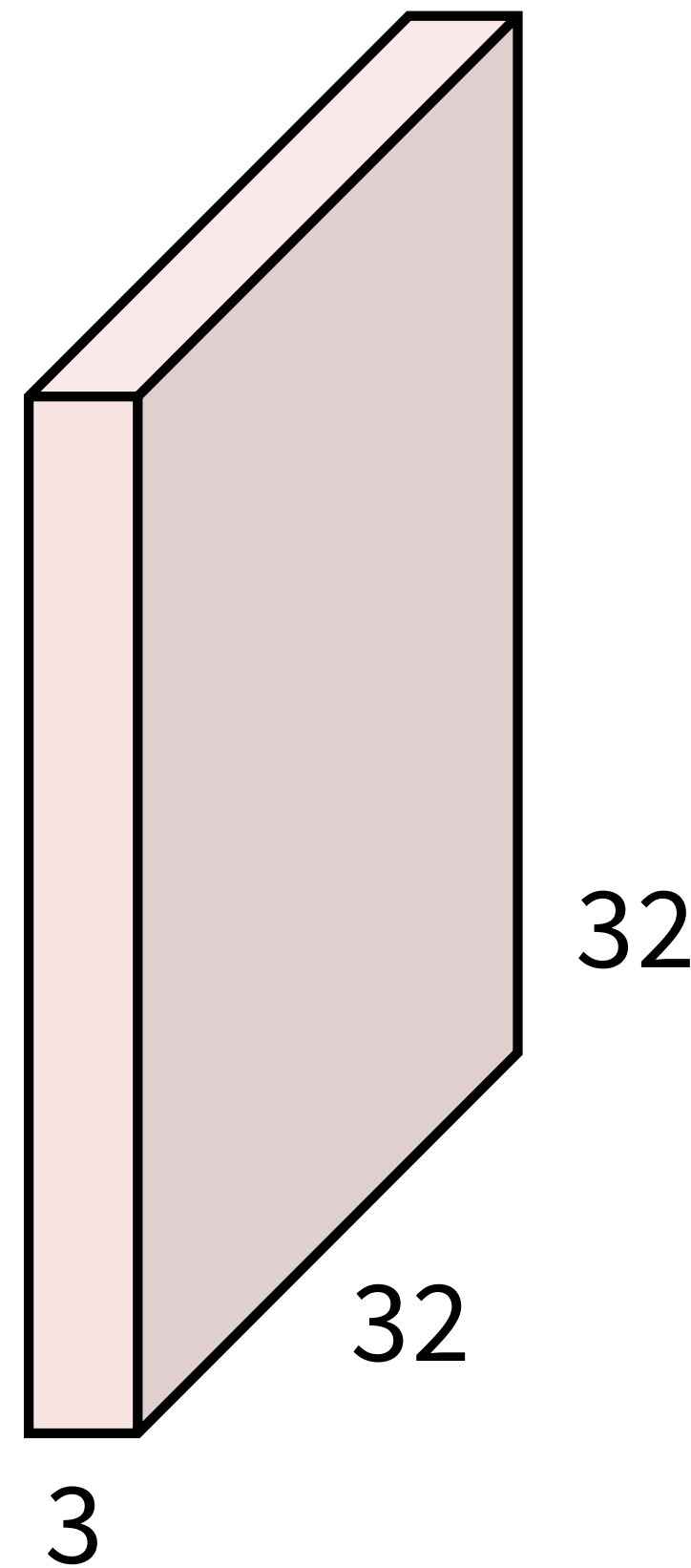
# Convolution Layer

consider a second, **green** filter



# Convolution Layer

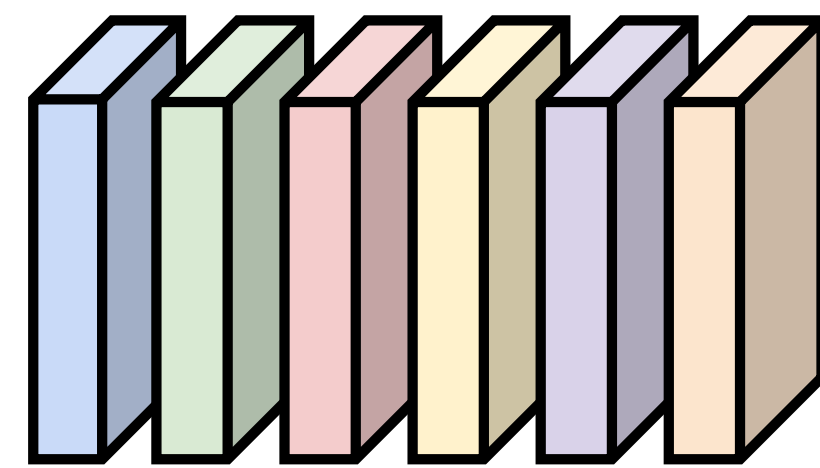
3x32x32 image



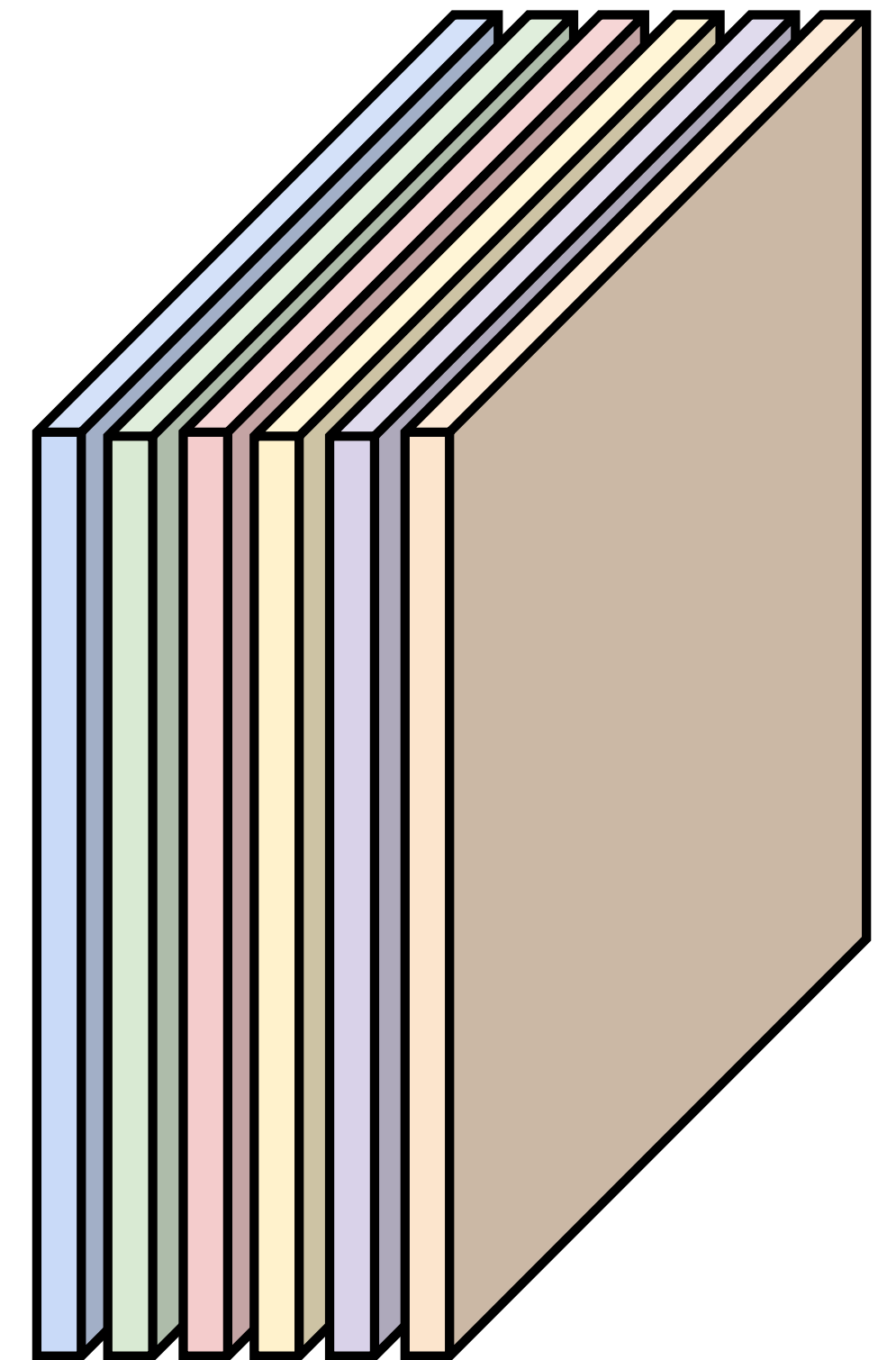
Consider 6 filters,  
each 3x5x5

Convolution  
Layer

6x3x5x5  
filters



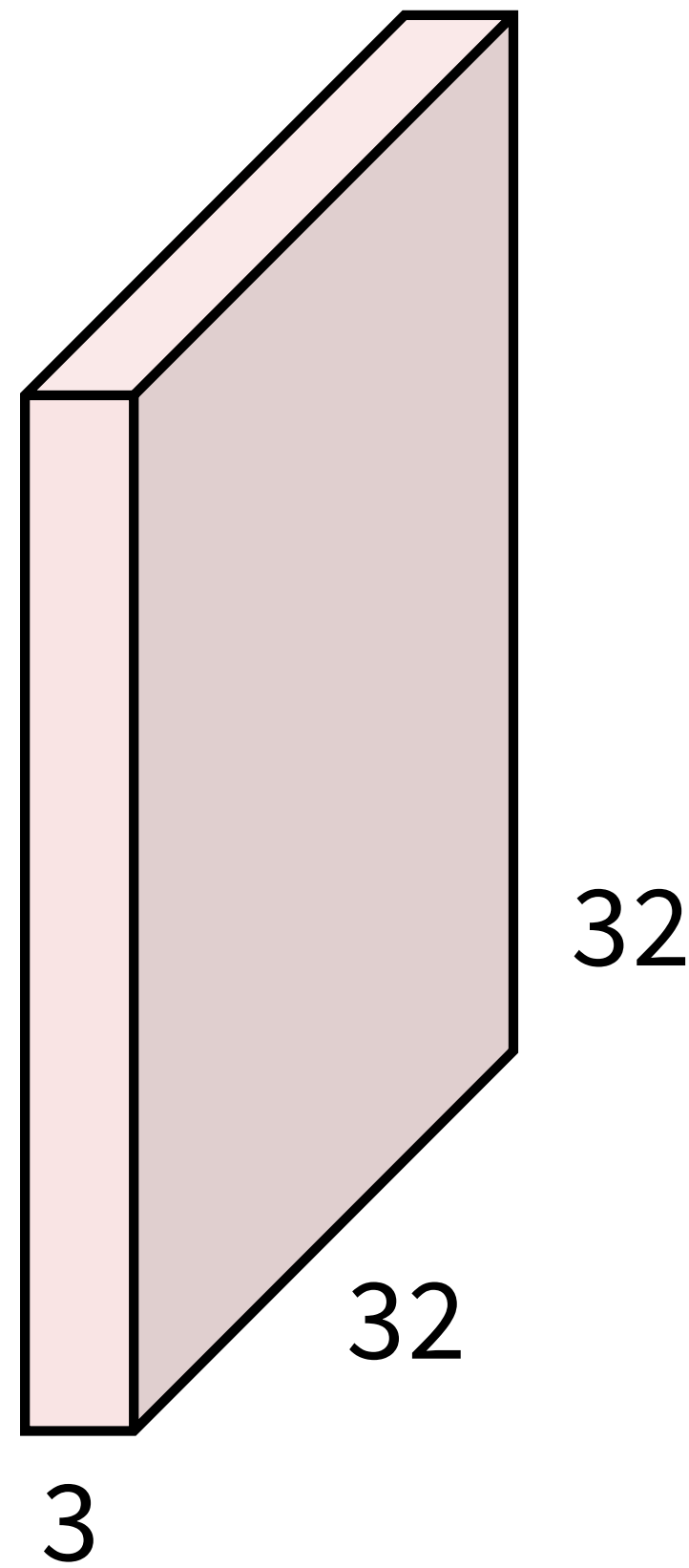
6 activation maps,  
each 1x28x28



Stack activations to get a  
6x28x28 output image!

# Convolution Layer

3x32x32 image

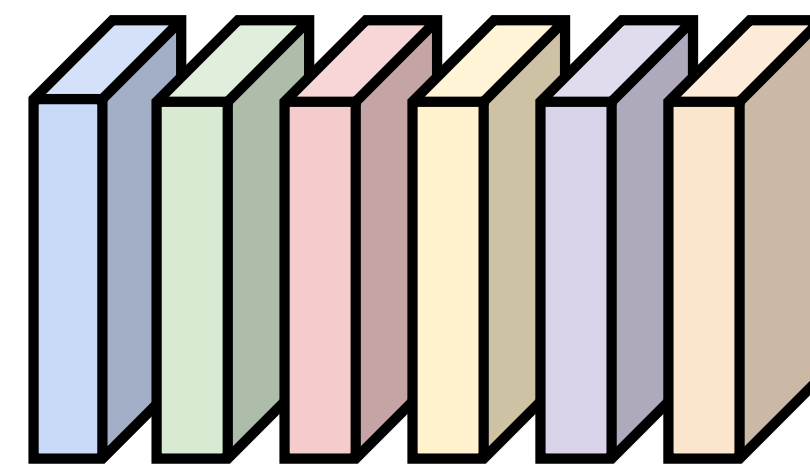


Also 6-dim bias vector:

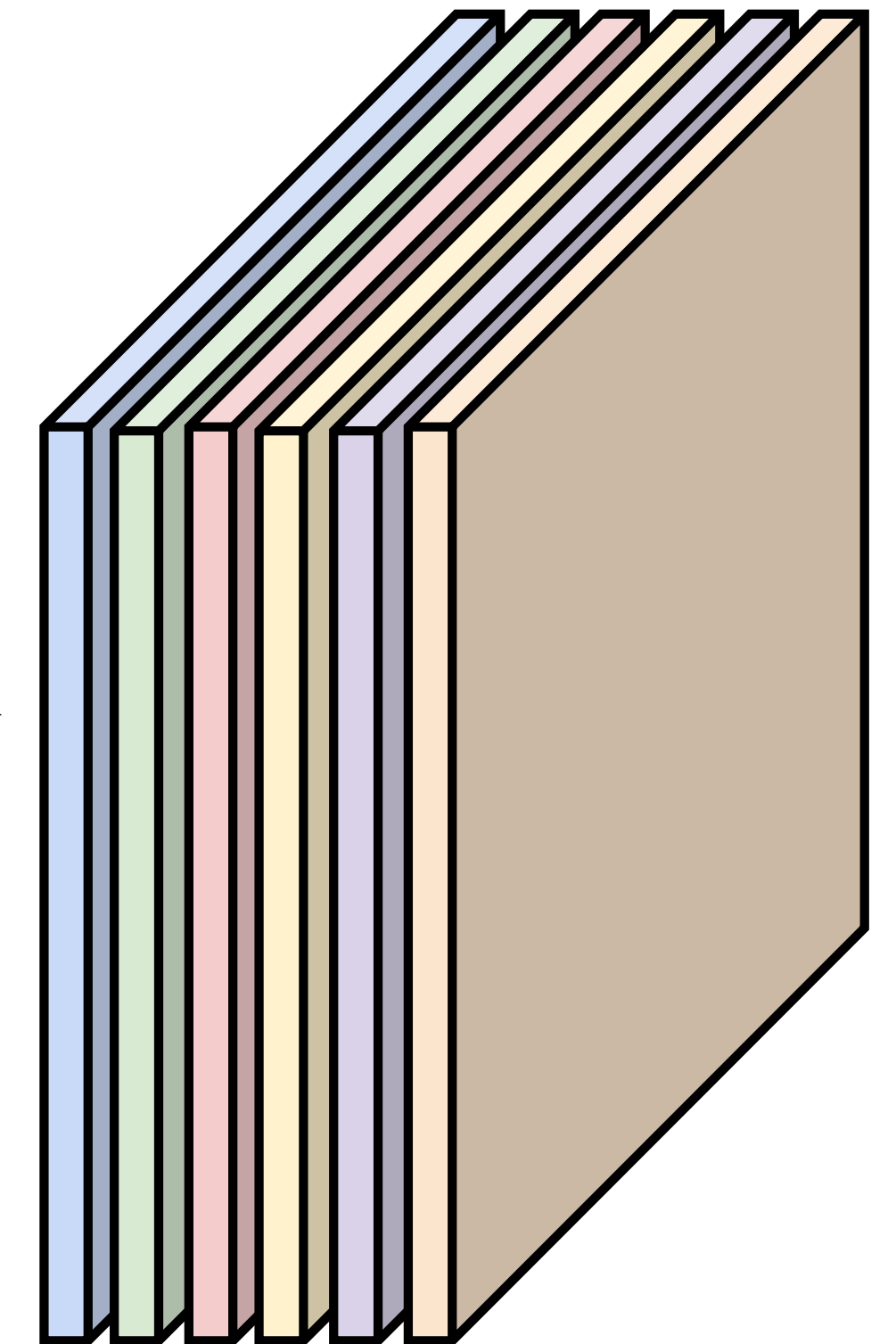


Convolution  
Layer

6x3x5x5  
filters



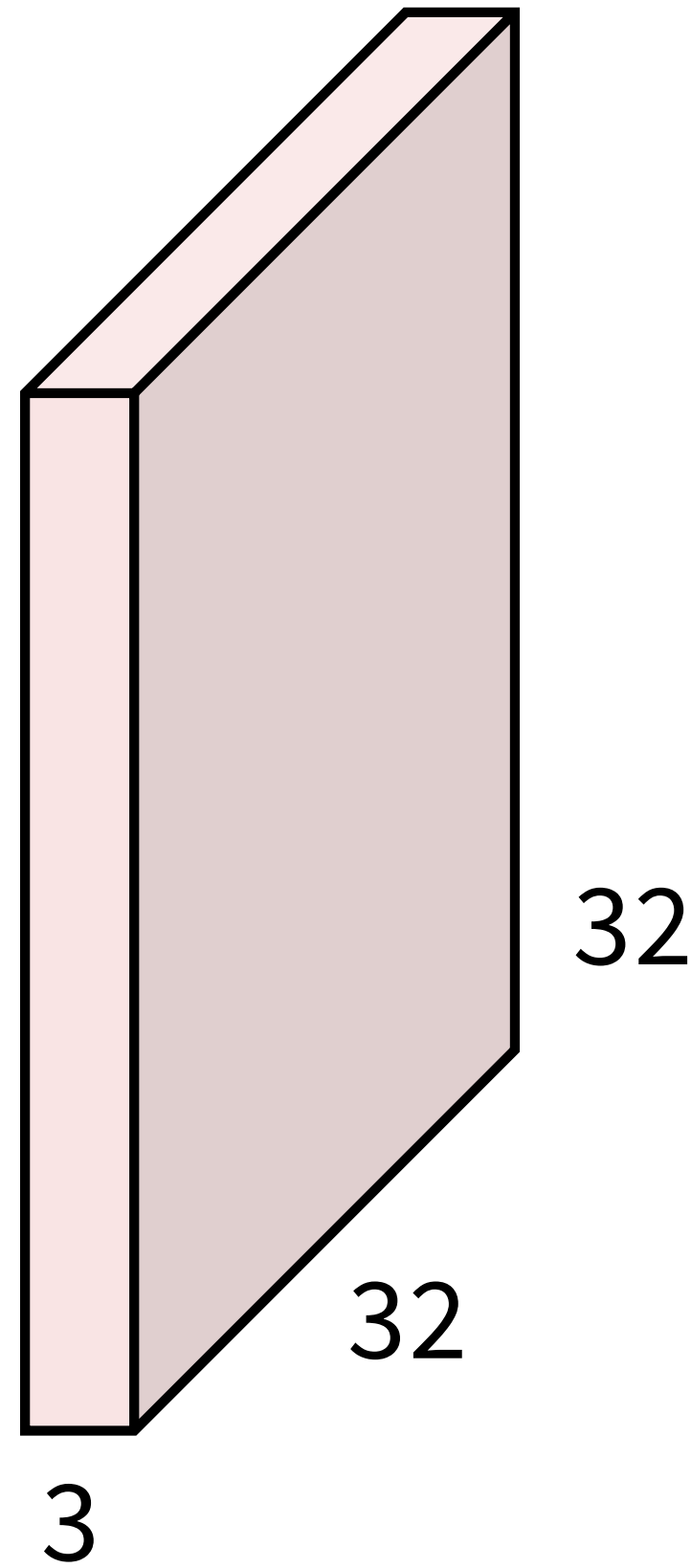
6 activation maps,  
each 1x28x28



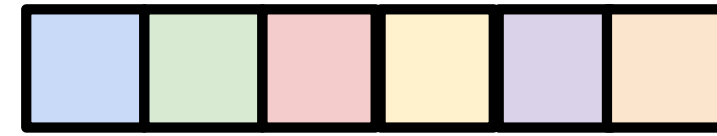
Stack activations to get a  
6x28x28 output image!

# Convolution Layer

3x32x32 image

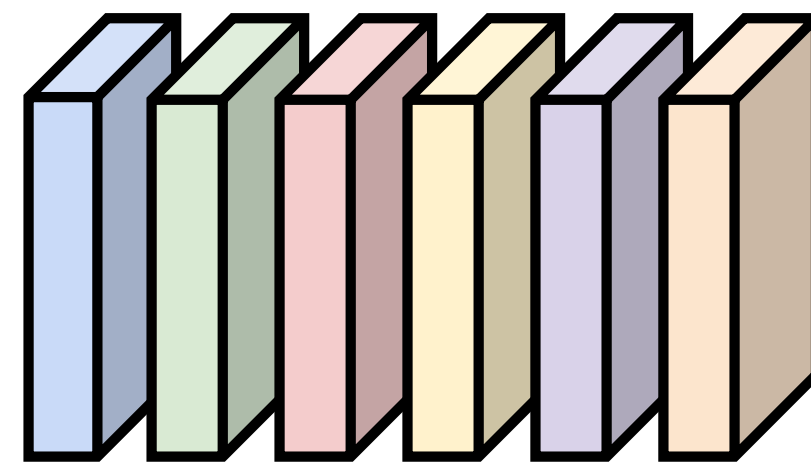


Also 6-dim bias vector:

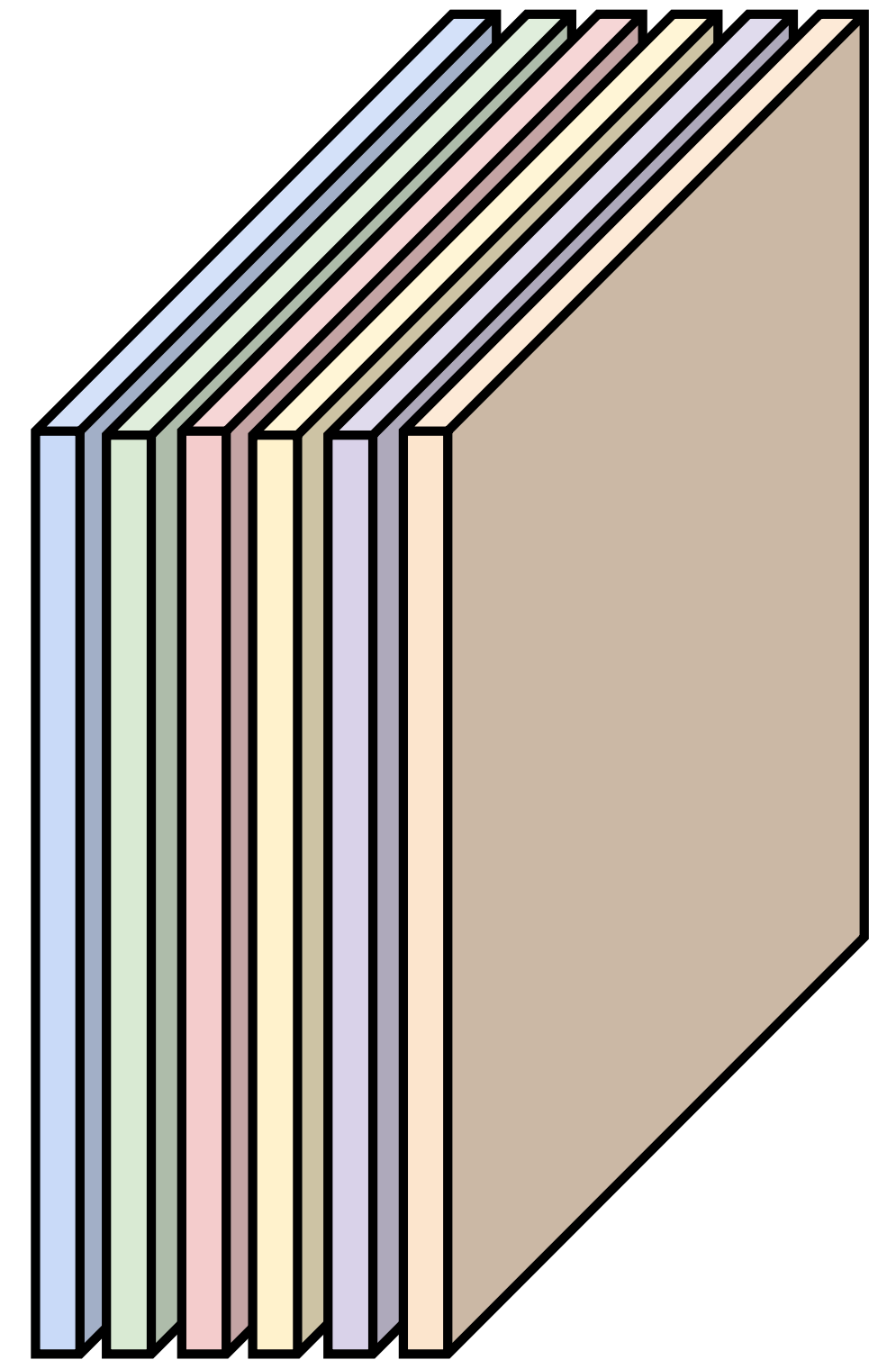


Convolution  
Layer

6x3x5x5  
filters



28x28 grid, at each  
point a 6-dim vector



Stack activations to get a  
6x28x28 output image!

# Convolution Layer

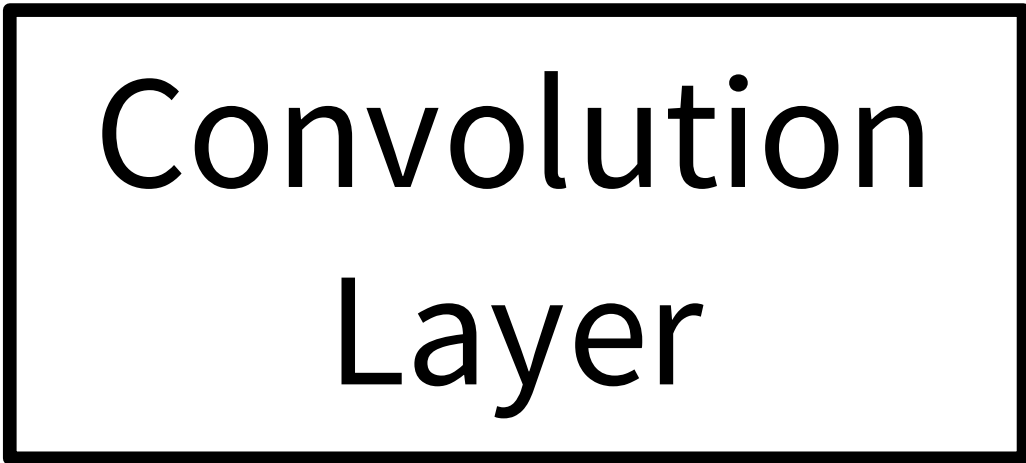
**Batch size = 2**

2x6x28x28

Batch of outputs

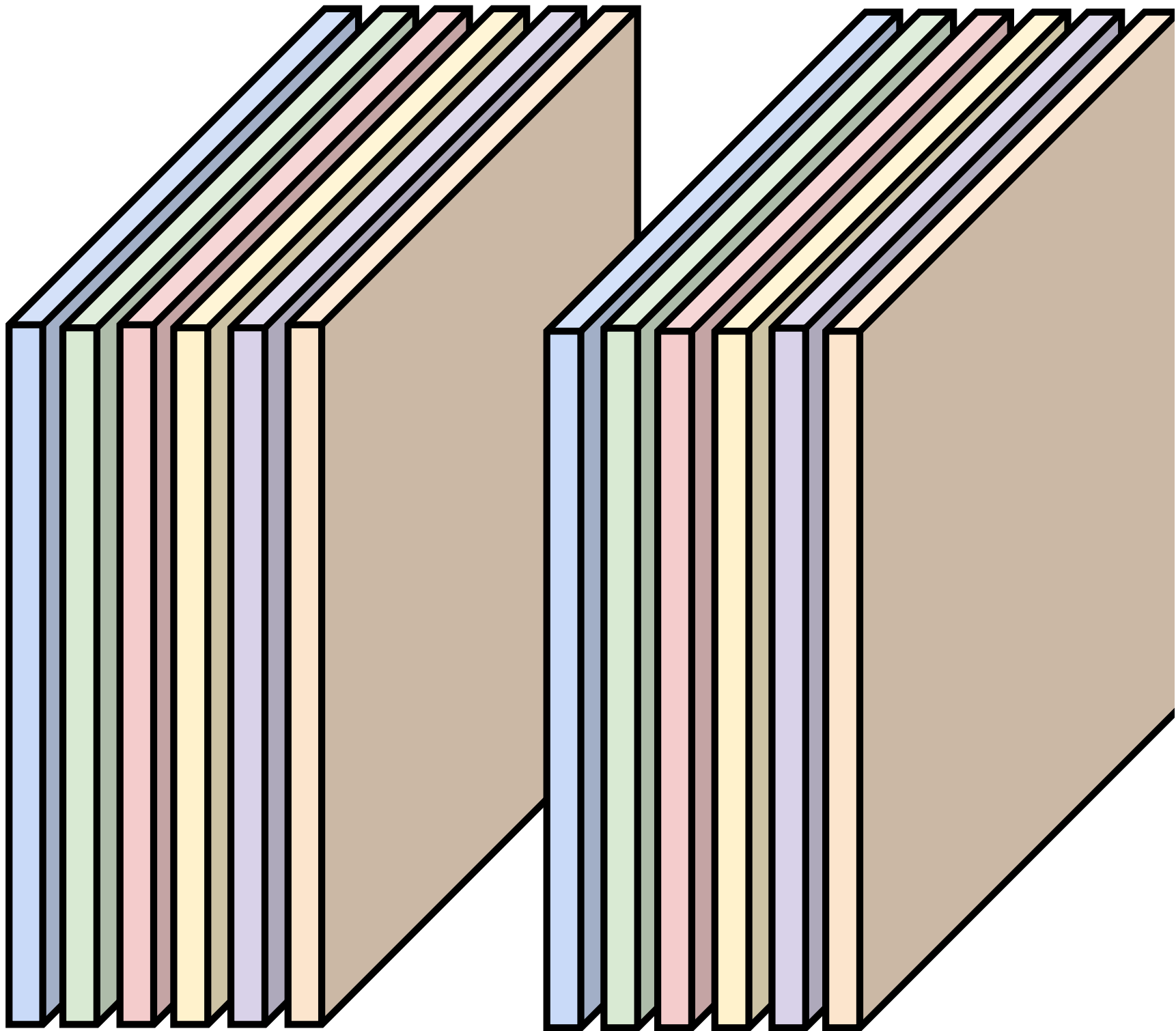
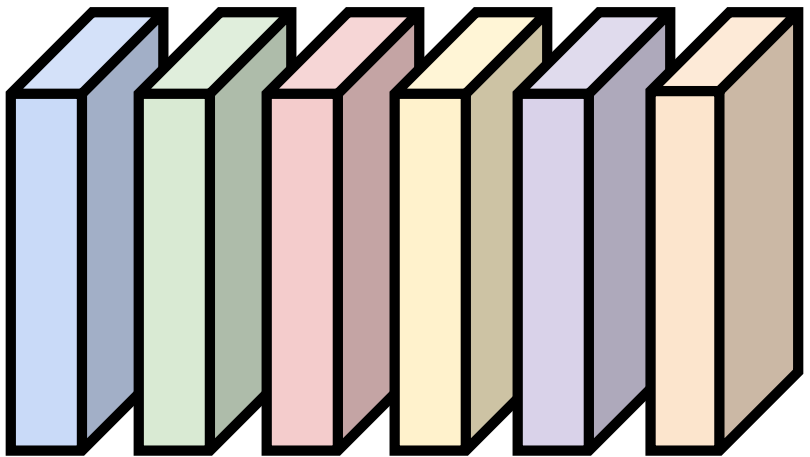
2x3x32x32  
Batch of images

Also 6-dim bias vector:



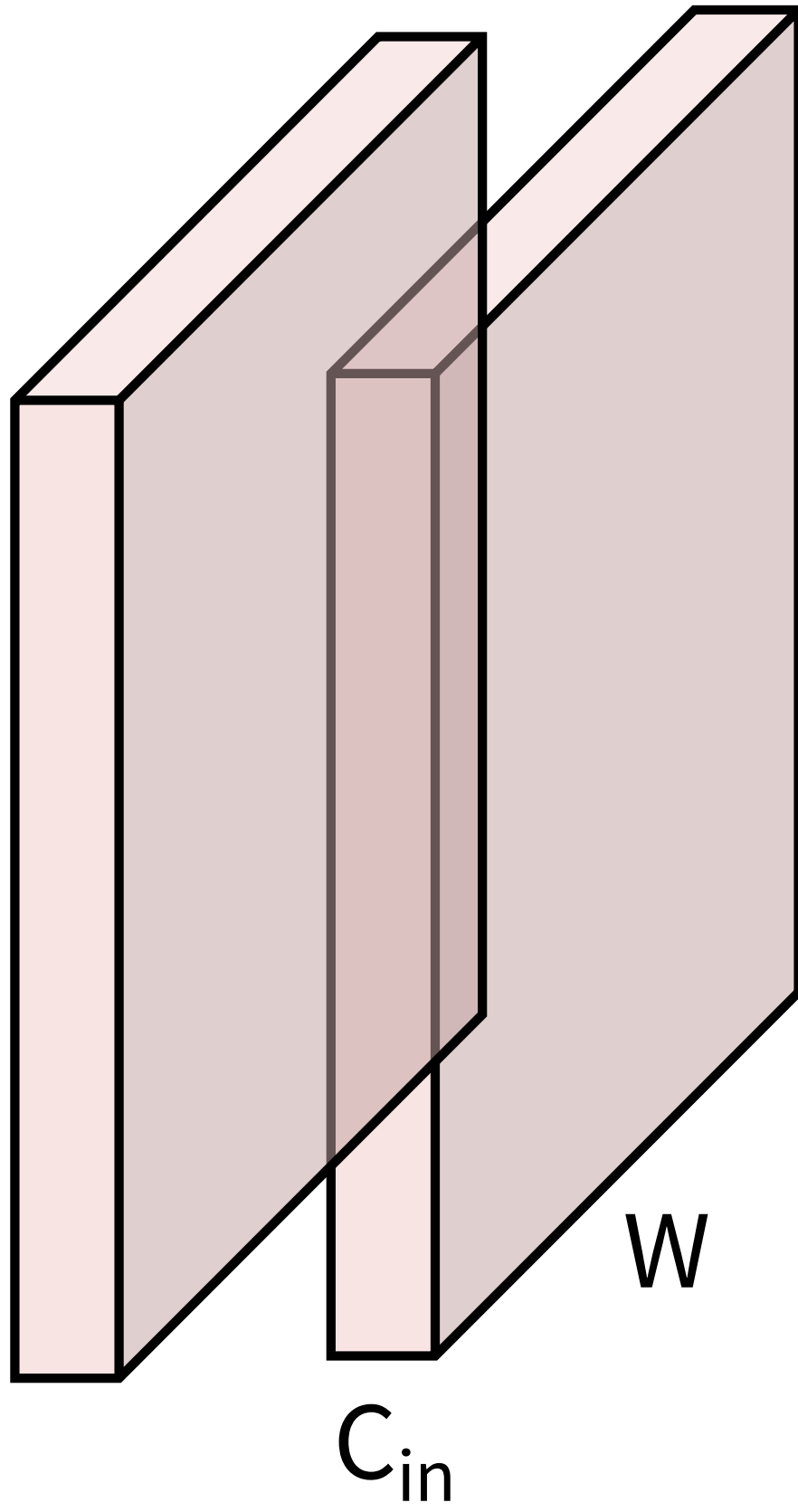
32

6x3x5x5  
filters

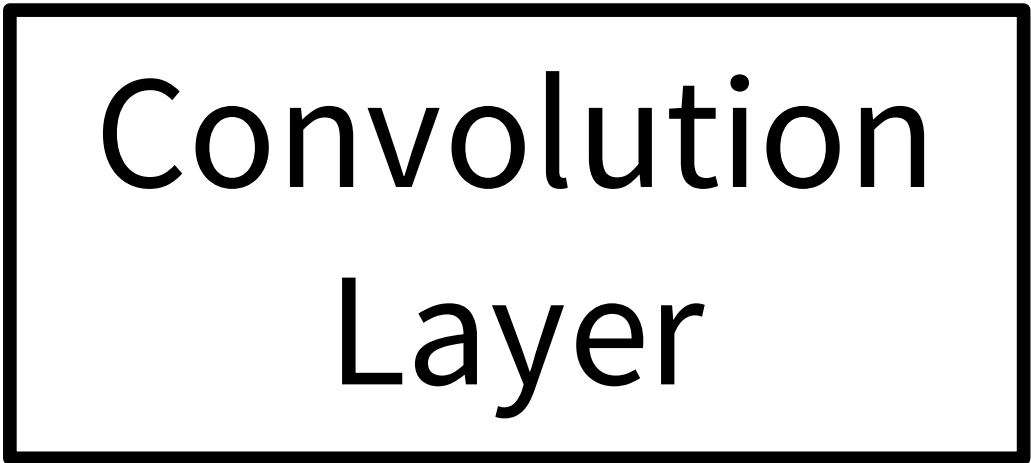
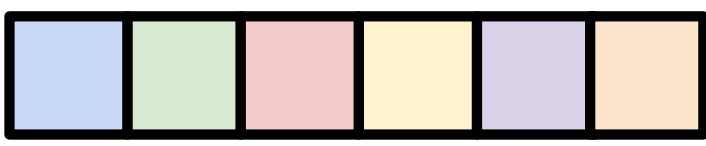


# Convolution Layer

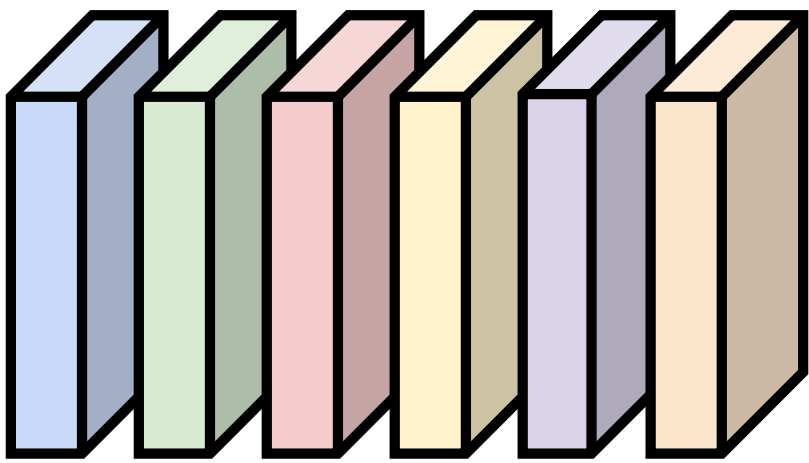
$N \times C_{in} \times H \times W$   
Batch of images



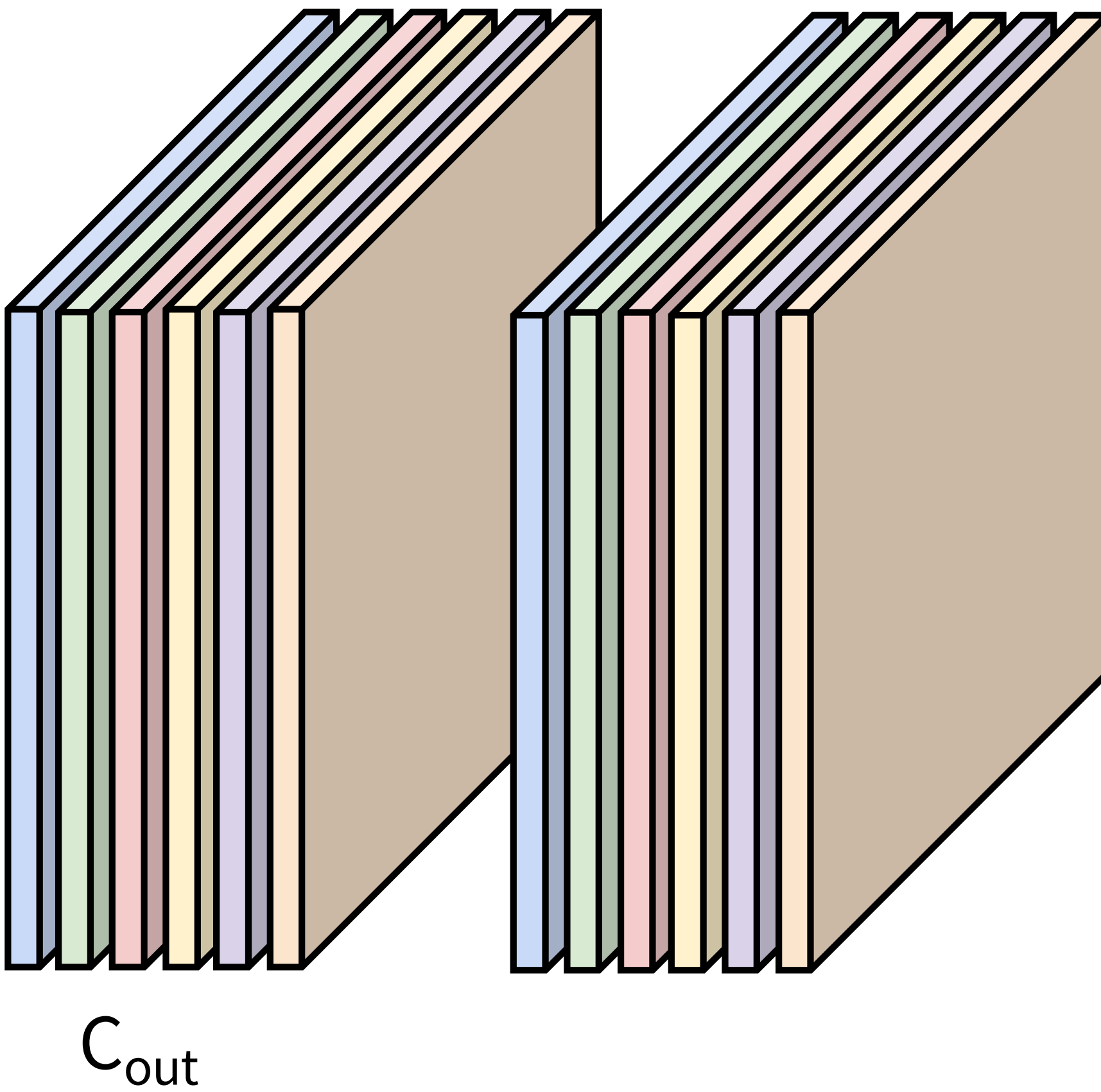
Also  $C_{out}$ -dim bias vector:



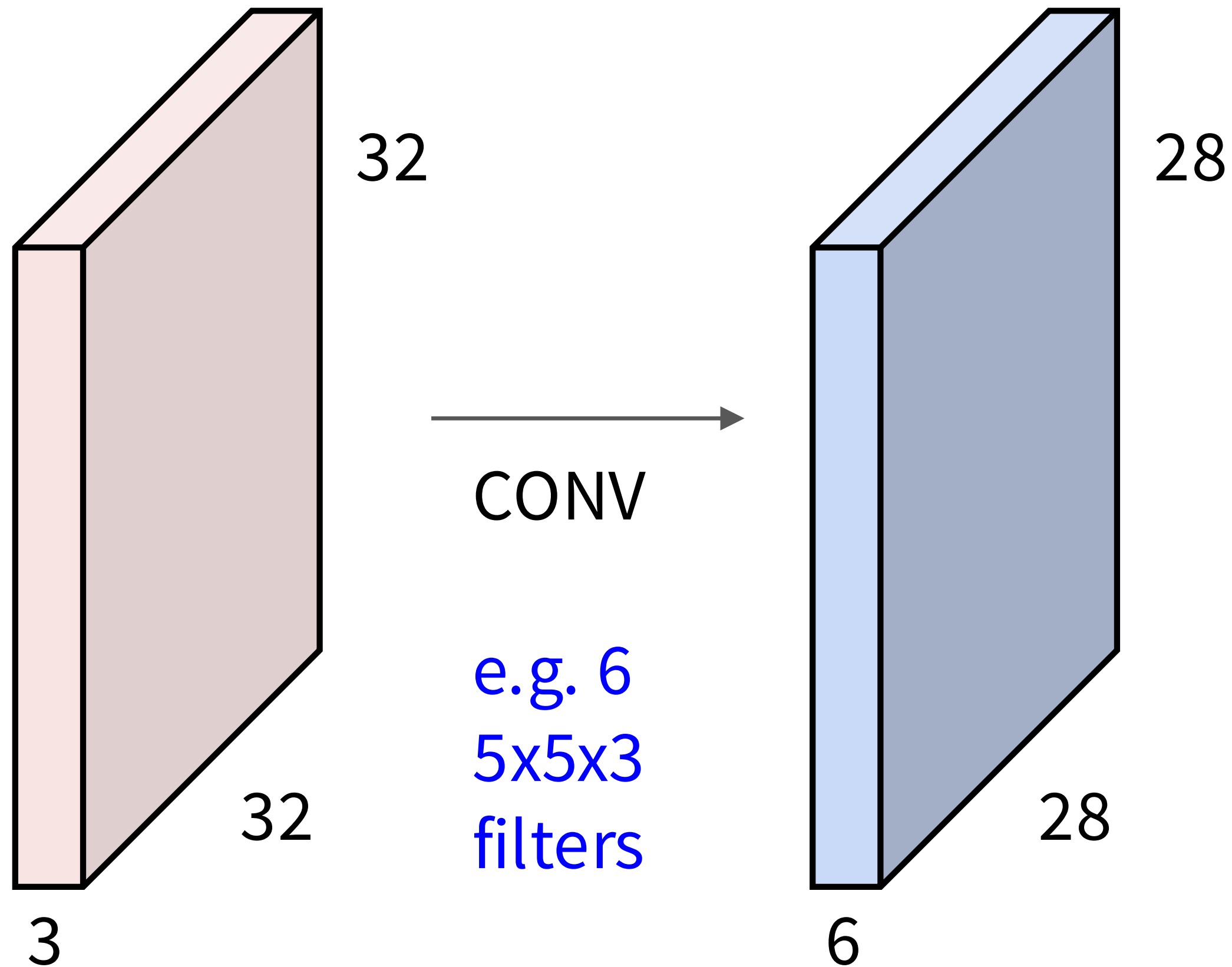
$C_{out} \times C_{in} \times K_w \times K_h$   
filters



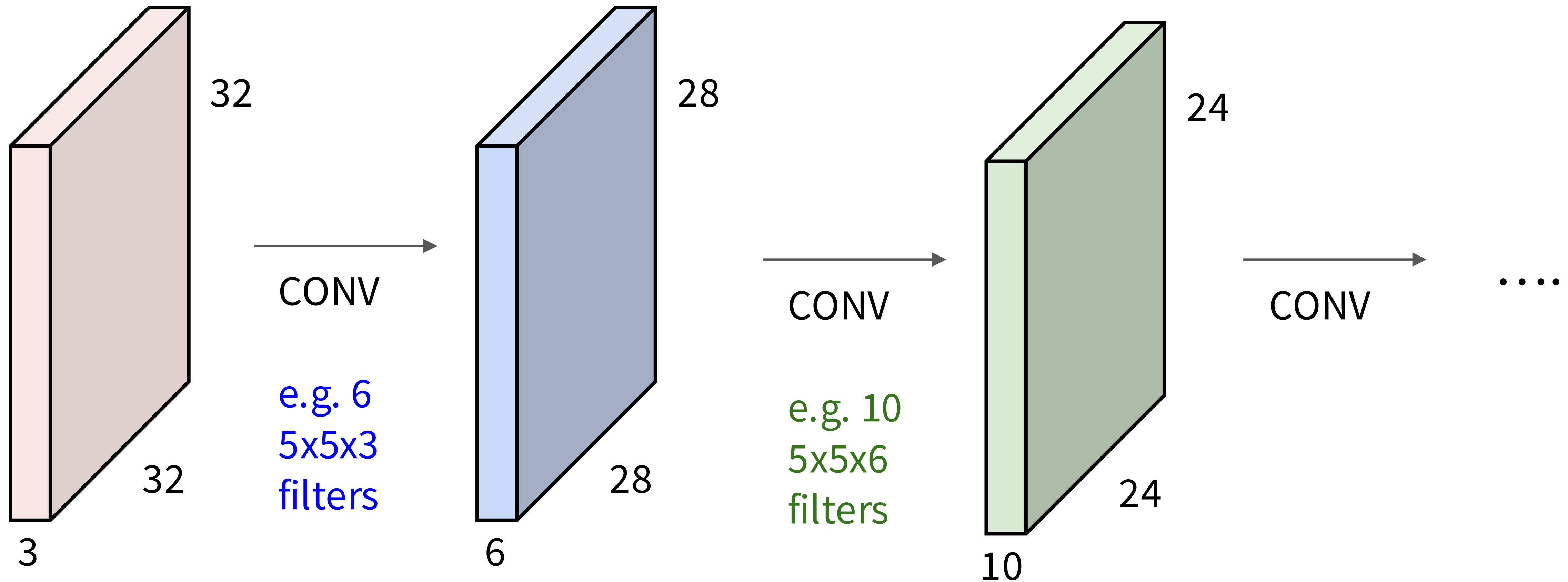
$N \times C_{out} \times H' \times W'$   
Batch of outputs



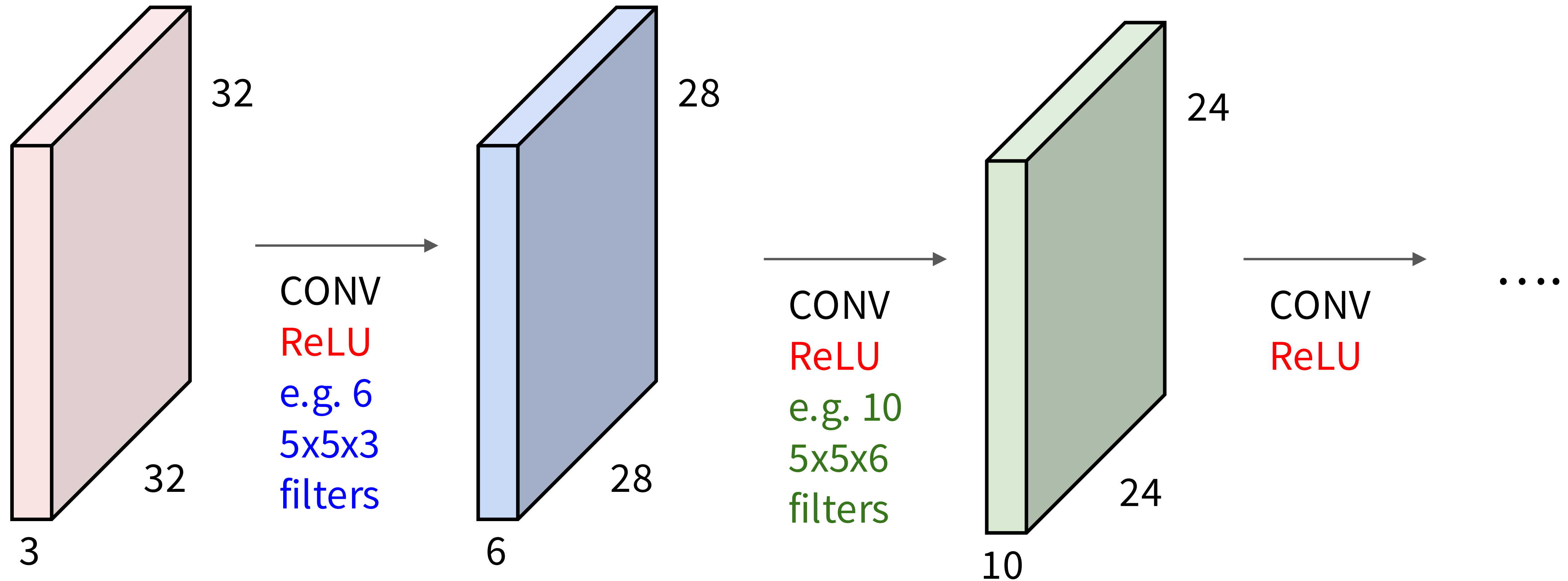
# A ConvNet is a neural network with Conv layers



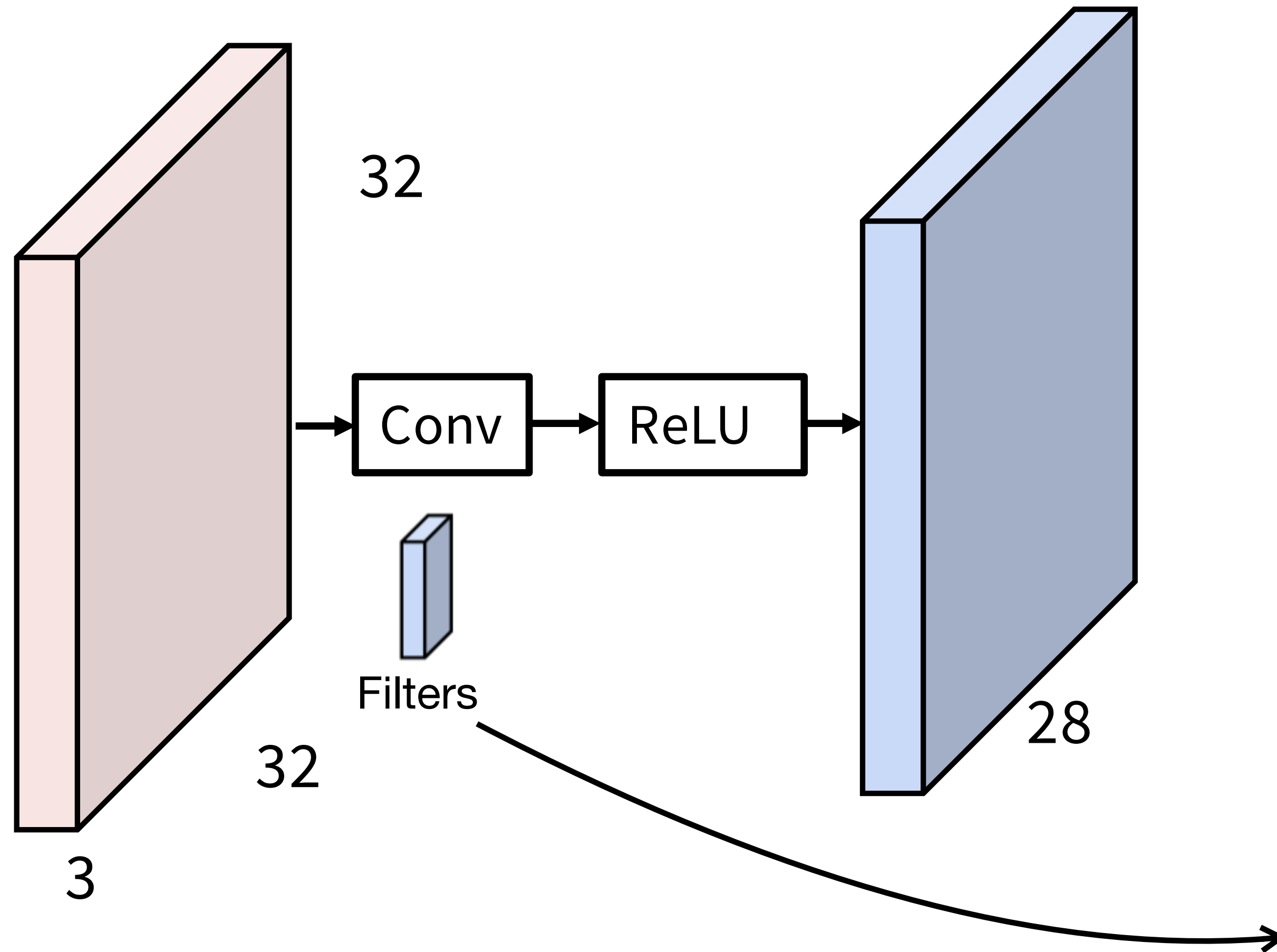
# A ConvNet is a neural network with Conv layers



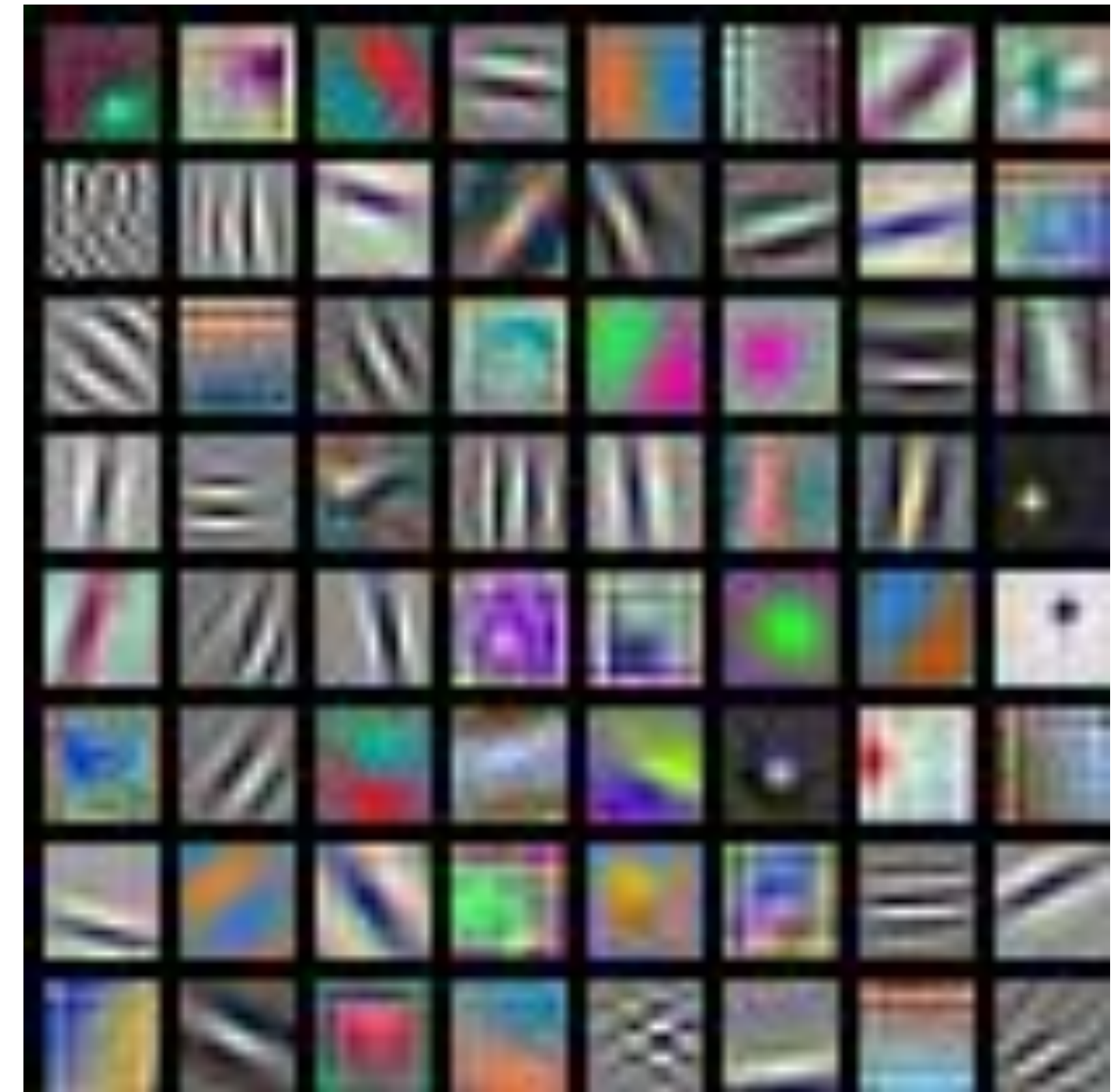
A ConvNet is a neural network with Conv layers  
with activation functions!



# What do Conv filters learn?



First-layer conv filters: local image templates  
(Often learns oriented edges, opposing colors)

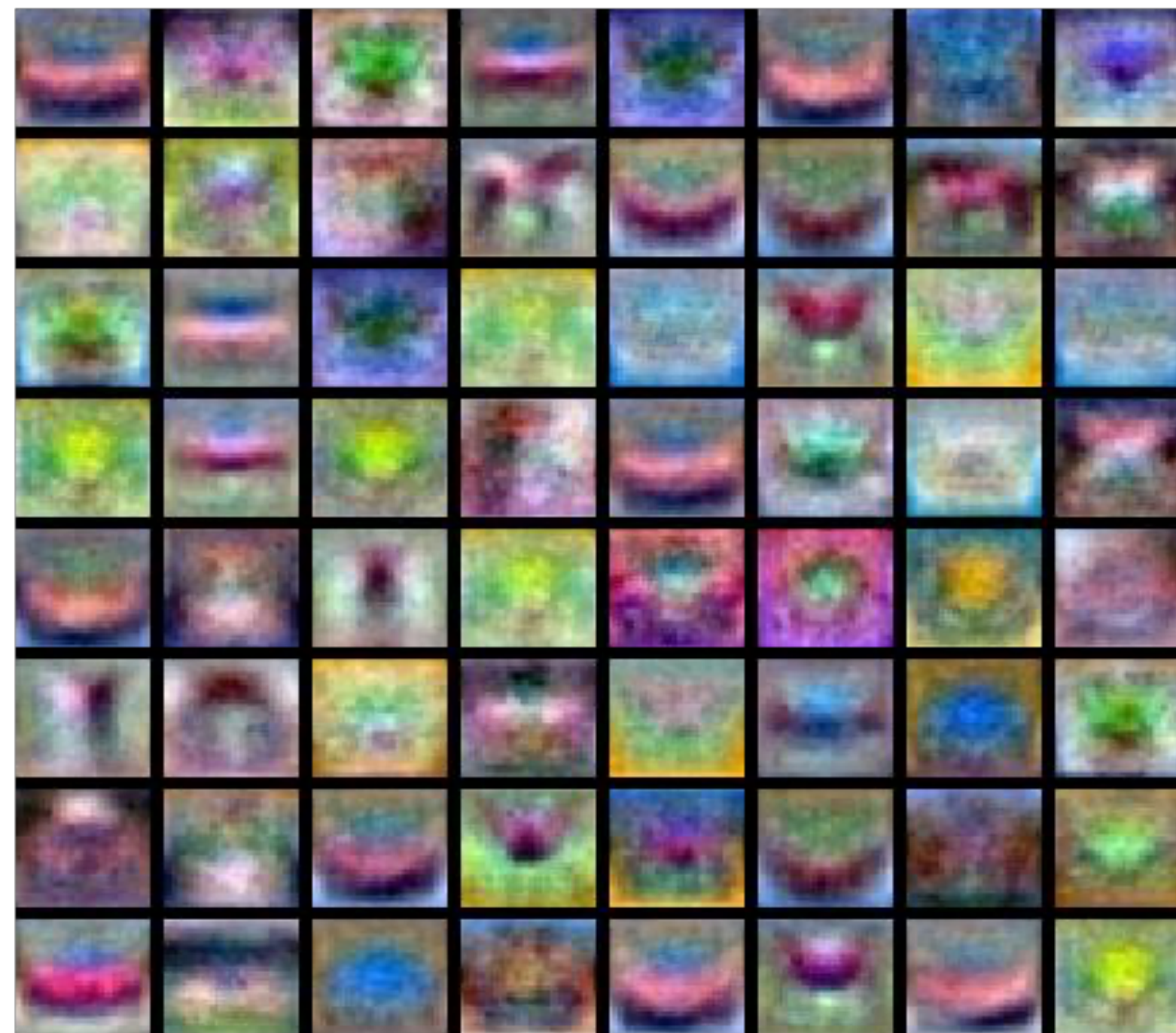


AlexNet: 64 filters, each 3x11x11

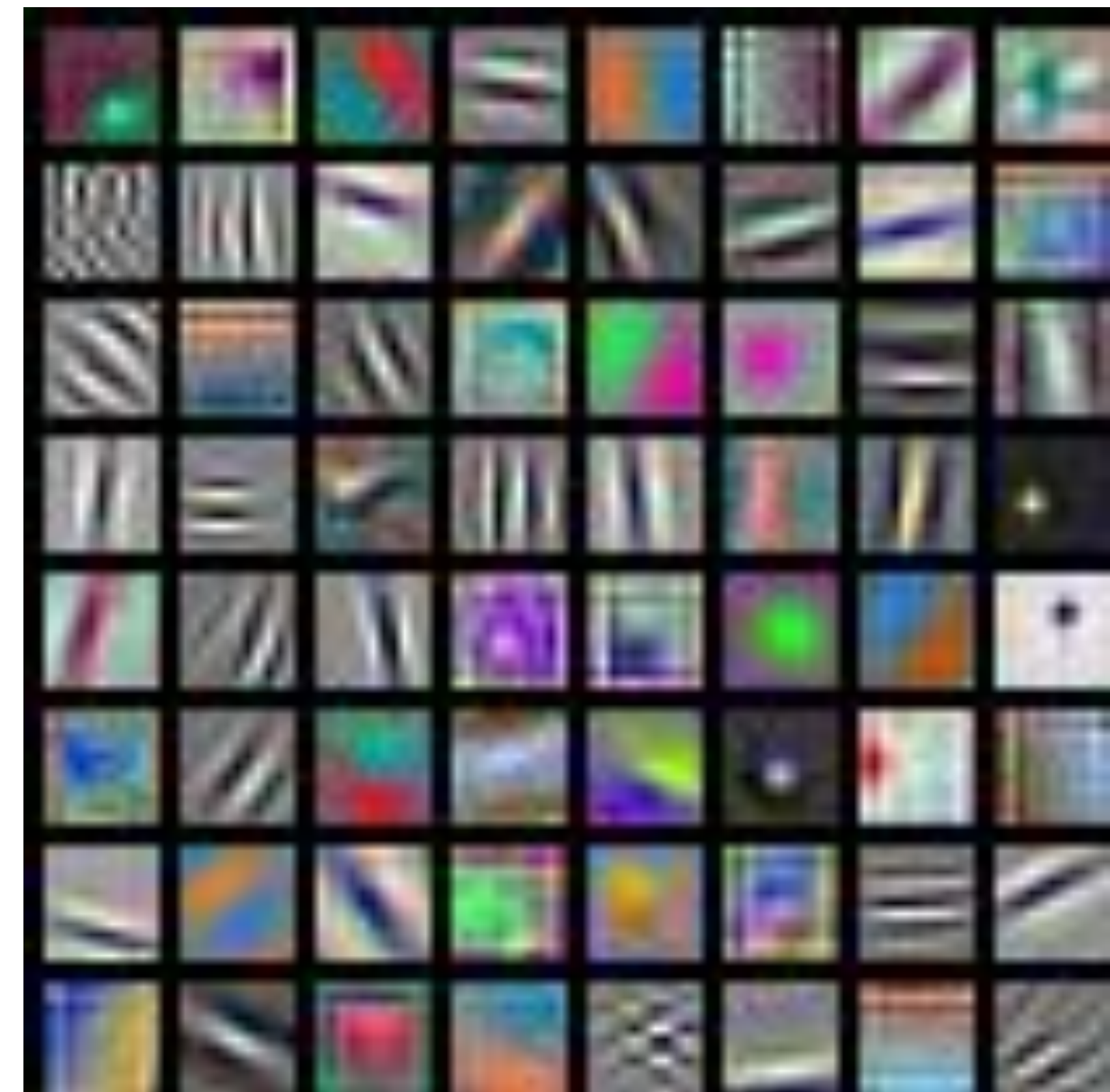
Linear classifier: One template per class



MLP: Bank of whole-image templates

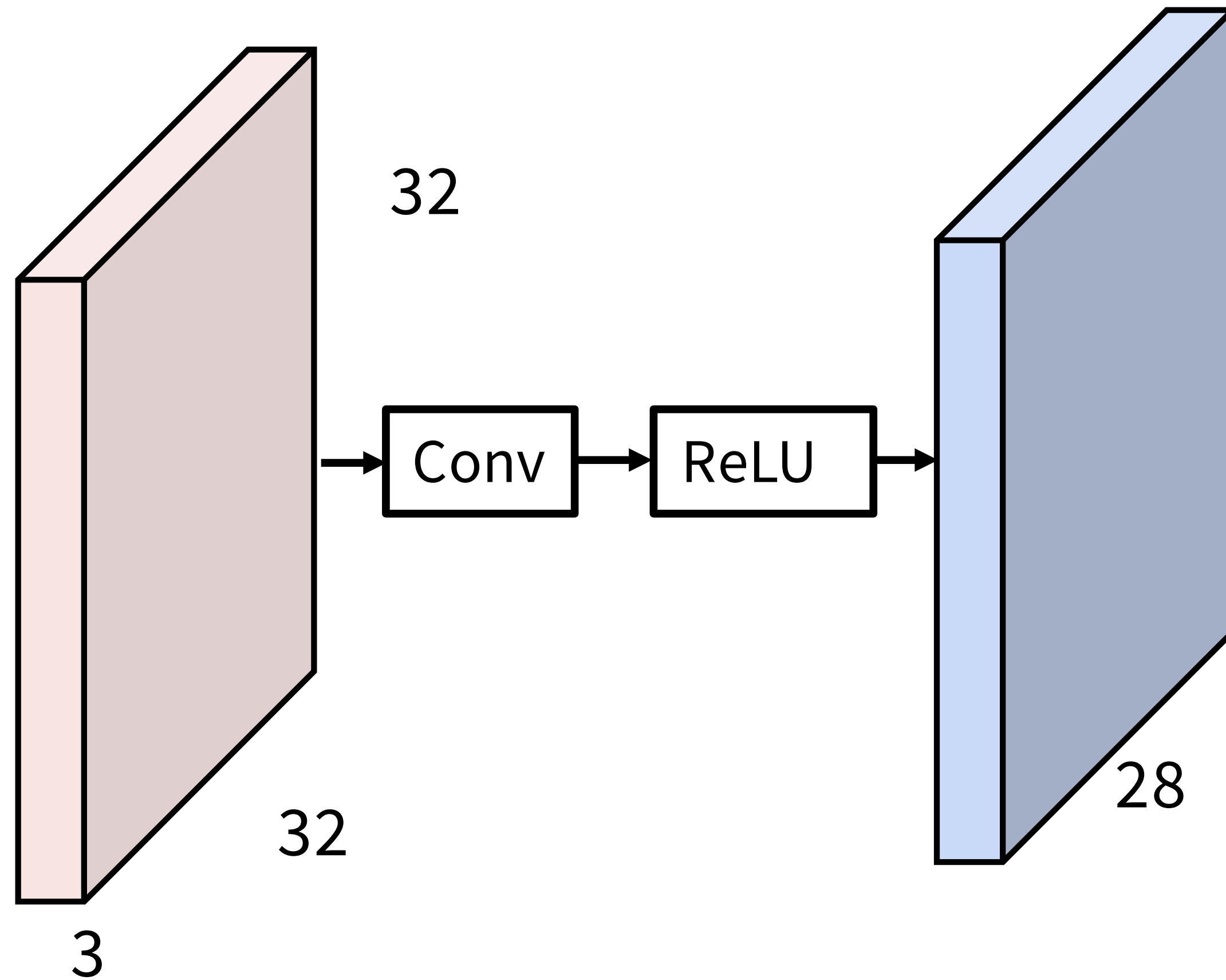


First-layer conv filters: local image templates  
(Often learns oriented edges, opposing colors)



AlexNet: 64 filters, each 3x11x11

# What do Conv filters learn?



Deeper conv layers: Harder to visualize  
Tend to learn larger structures e.g. eyes, letters



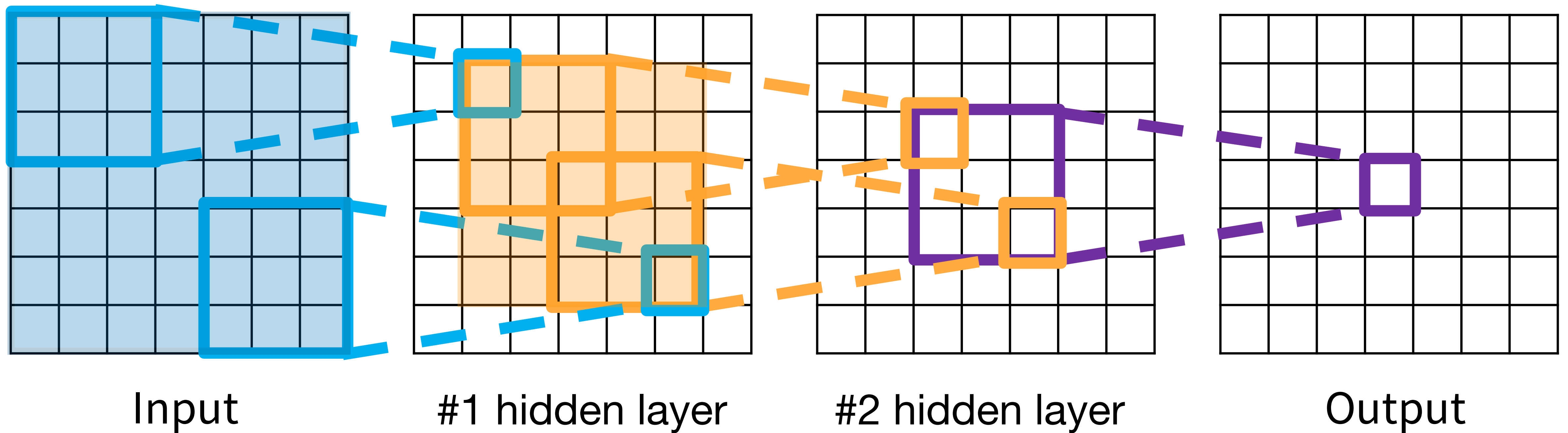
**Receptive field is larger!**

6<sup>th</sup> layer conv layer from an ImageNet model

Visualization from [Springenberg et al, ICLR 2015]

# Receptive Fields

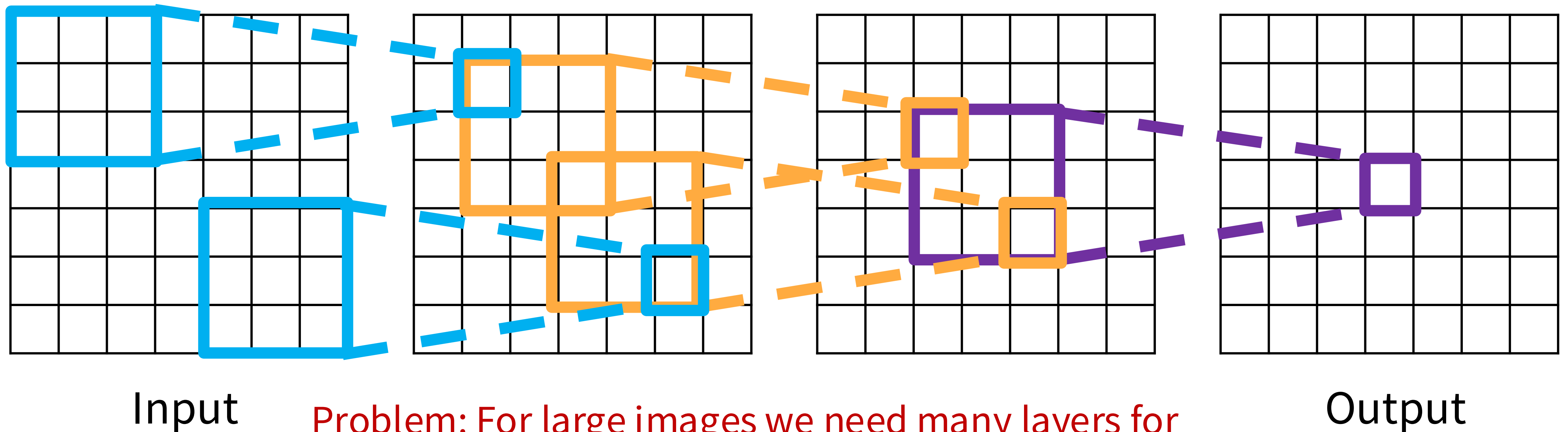
Each successive convolution adds  $K - 1$  to the receptive field size  
With  $L$  layers the receptive field size is  $1 + L * (K - 1)$



Be careful – “receptive field in the input” vs. “receptive field in the previous layer”

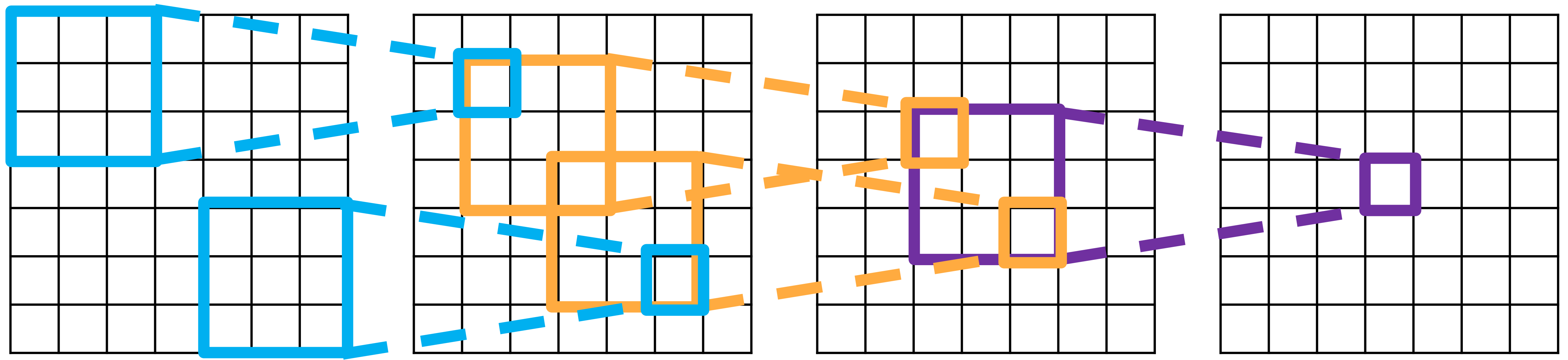
# Receptive Fields

Each successive convolution adds  $K - 1$  to the receptive field size  
With  $L$  layers the receptive field size is  $1 + L * (K - 1)$



# Receptive Fields

Each successive convolution adds  $K - 1$  to the receptive field size  
With  $L$  layers the receptive field size is  $1 + L * (K - 1)$



Input

Problem: For large images we need many layers for each output to “see” the whole image

Solution: Downsample inside the network

Output

# Strided Convolution

7

|   |   |   |  |  |  |  |
|---|---|---|--|--|--|--|
| 1 | 2 | 3 |  |  |  |  |
|   |   |   |  |  |  |  |
|   |   |   |  |  |  |  |
|   |   |   |  |  |  |  |
|   |   |   |  |  |  |  |
|   |   |   |  |  |  |  |
|   |   |   |  |  |  |  |

7

Input: 7x7

Filter: 3x3

Stride: 2

# Strided Convolution

7

|  |  |   |   |   |  |  |
|--|--|---|---|---|--|--|
|  |  | 3 | 4 | 5 |  |  |
|  |  |   |   |   |  |  |
|  |  |   |   |   |  |  |
|  |  |   |   |   |  |  |
|  |  |   |   |   |  |  |
|  |  |   |   |   |  |  |
|  |  |   |   |   |  |  |

7

Input: 7x7

Filter: 3x3

Stride: 2

# Strided Convolution

7

|  |  |  |  |   |   |   |
|--|--|--|--|---|---|---|
|  |  |  |  | 5 | 6 | 7 |
|  |  |  |  |   |   |   |
|  |  |  |  |   |   |   |
|  |  |  |  |   |   |   |
|  |  |  |  |   |   |   |
|  |  |  |  |   |   |   |
|  |  |  |  |   |   |   |

7

Input: 7x7

Filter: 3x3

Stride: 2

Output: 3x3

# Strided Convolution

7

|  |  |  |  |   |   |   |
|--|--|--|--|---|---|---|
|  |  |  |  | 5 | 6 | 7 |
|  |  |  |  |   |   |   |
|  |  |  |  |   |   |   |
|  |  |  |  |   |   |   |
|  |  |  |  |   |   |   |
|  |  |  |  |   |   |   |
|  |  |  |  |   |   |   |

7

Input: 7x7

Filter: 3x3

Stride: 2

Output: ?

In general:

Input: W

Filter: K

Padding: P

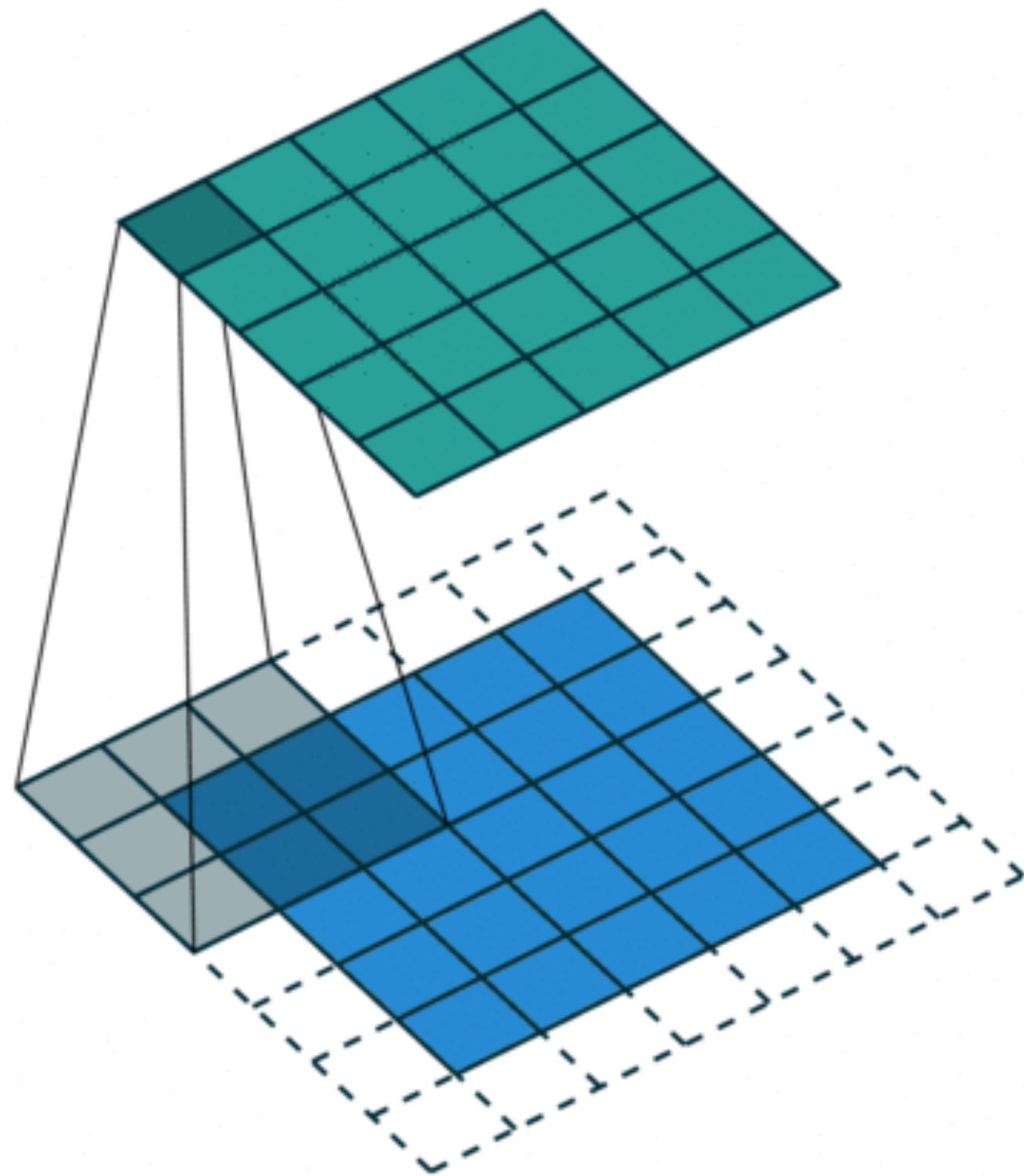
Stride: S



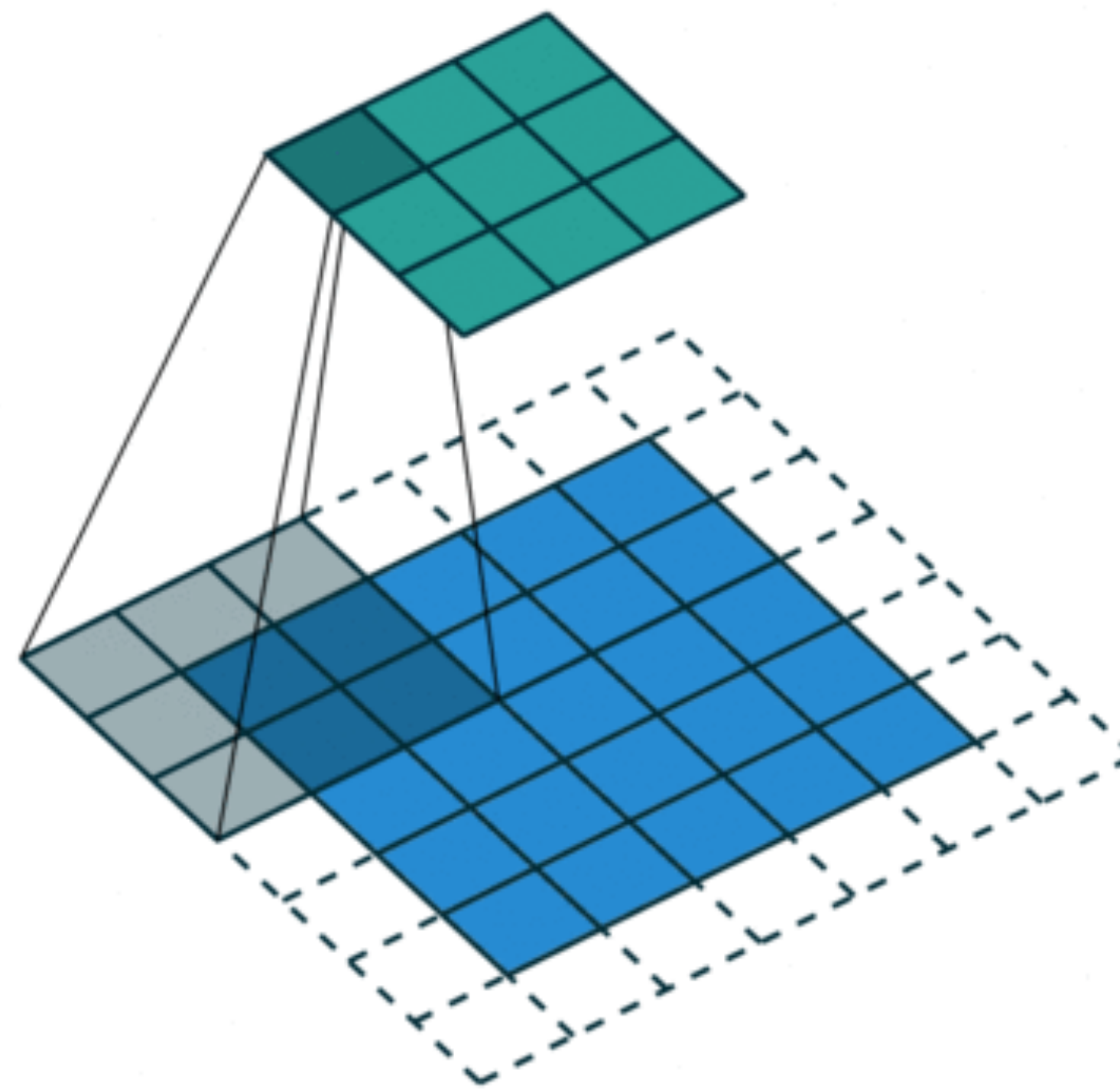
Output:

?

# Convolution Animations

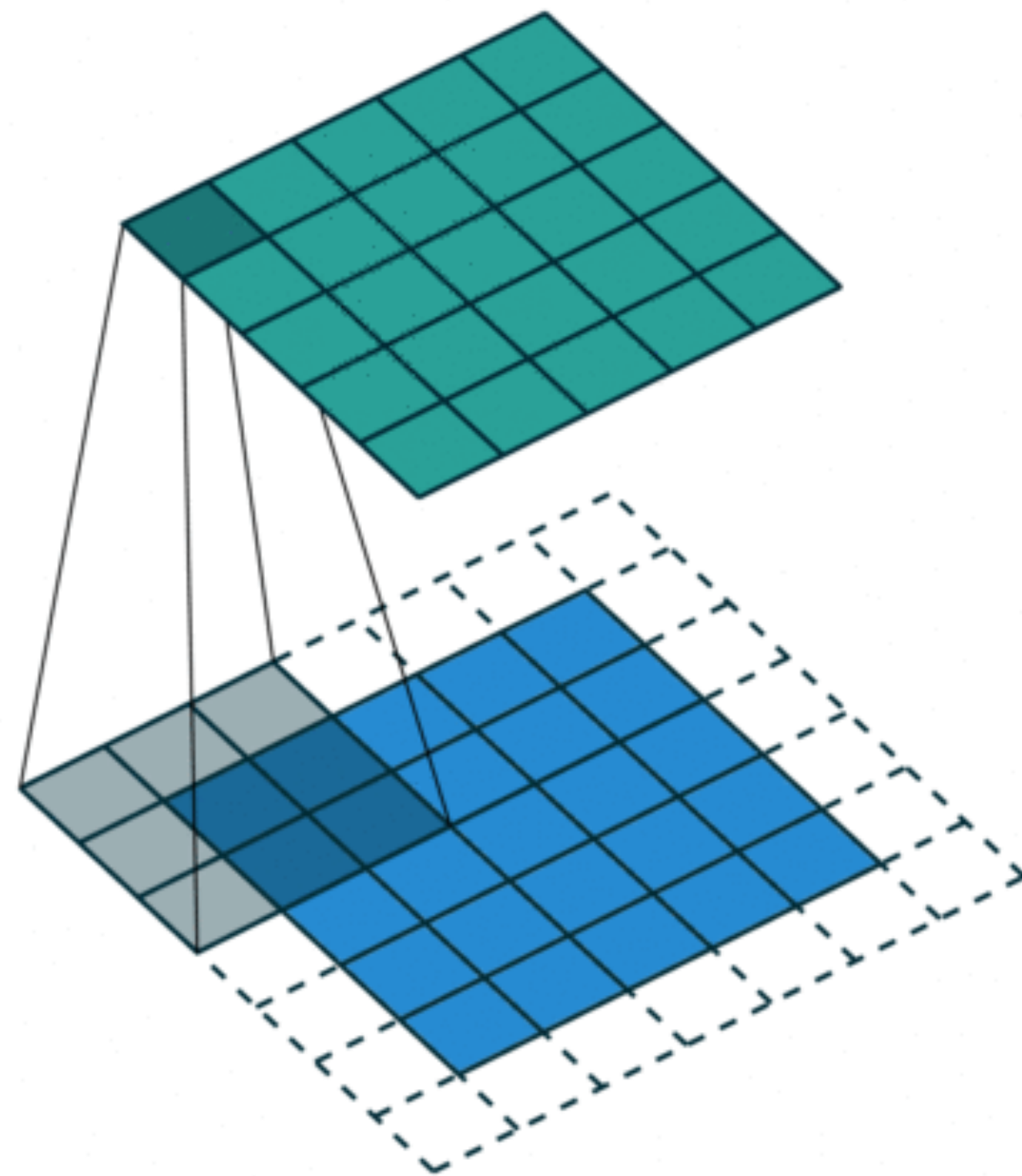


**3 x 3 kernel with Padding**

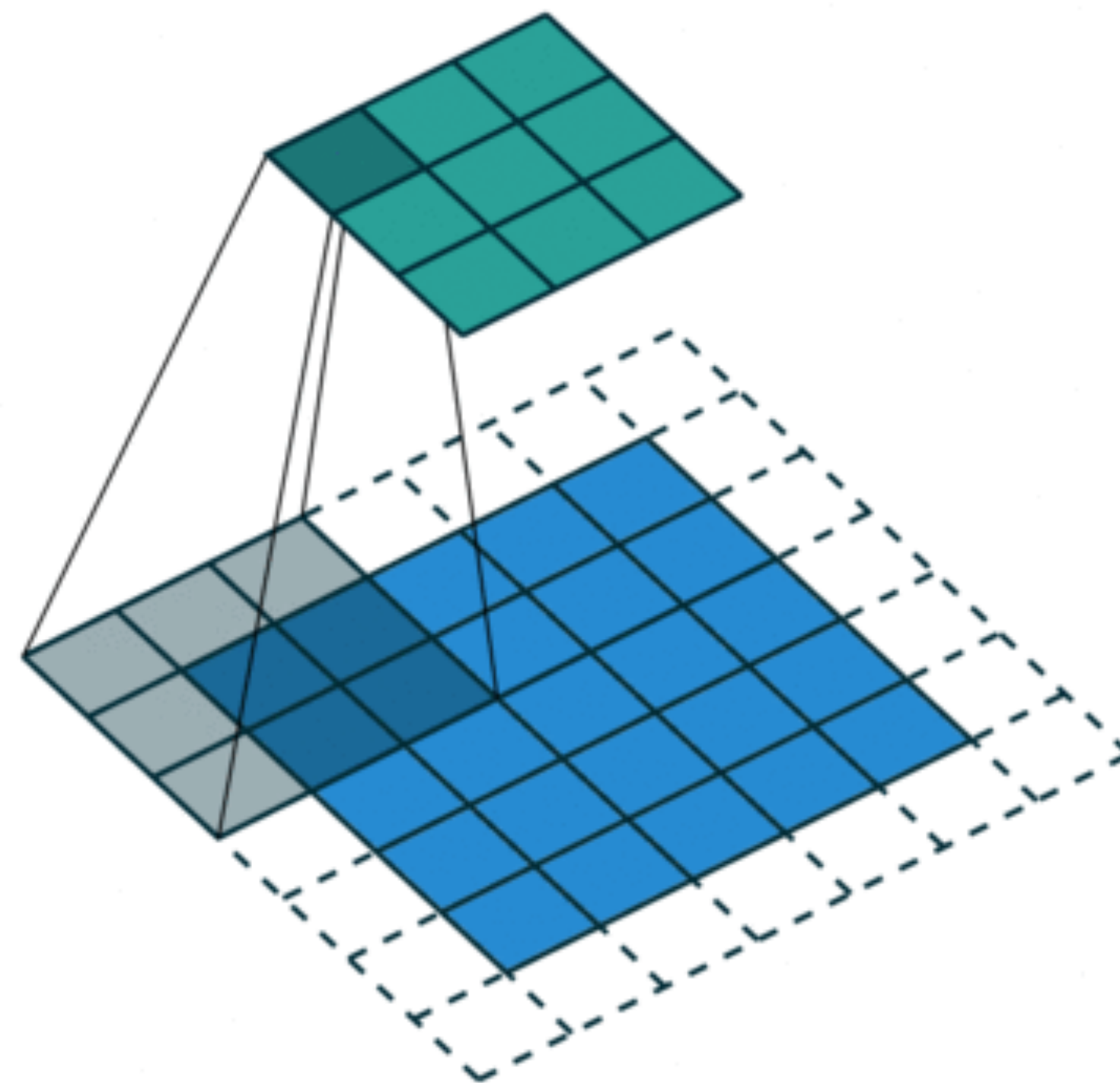


**3 x 3 kernel with Padding, stride=2**

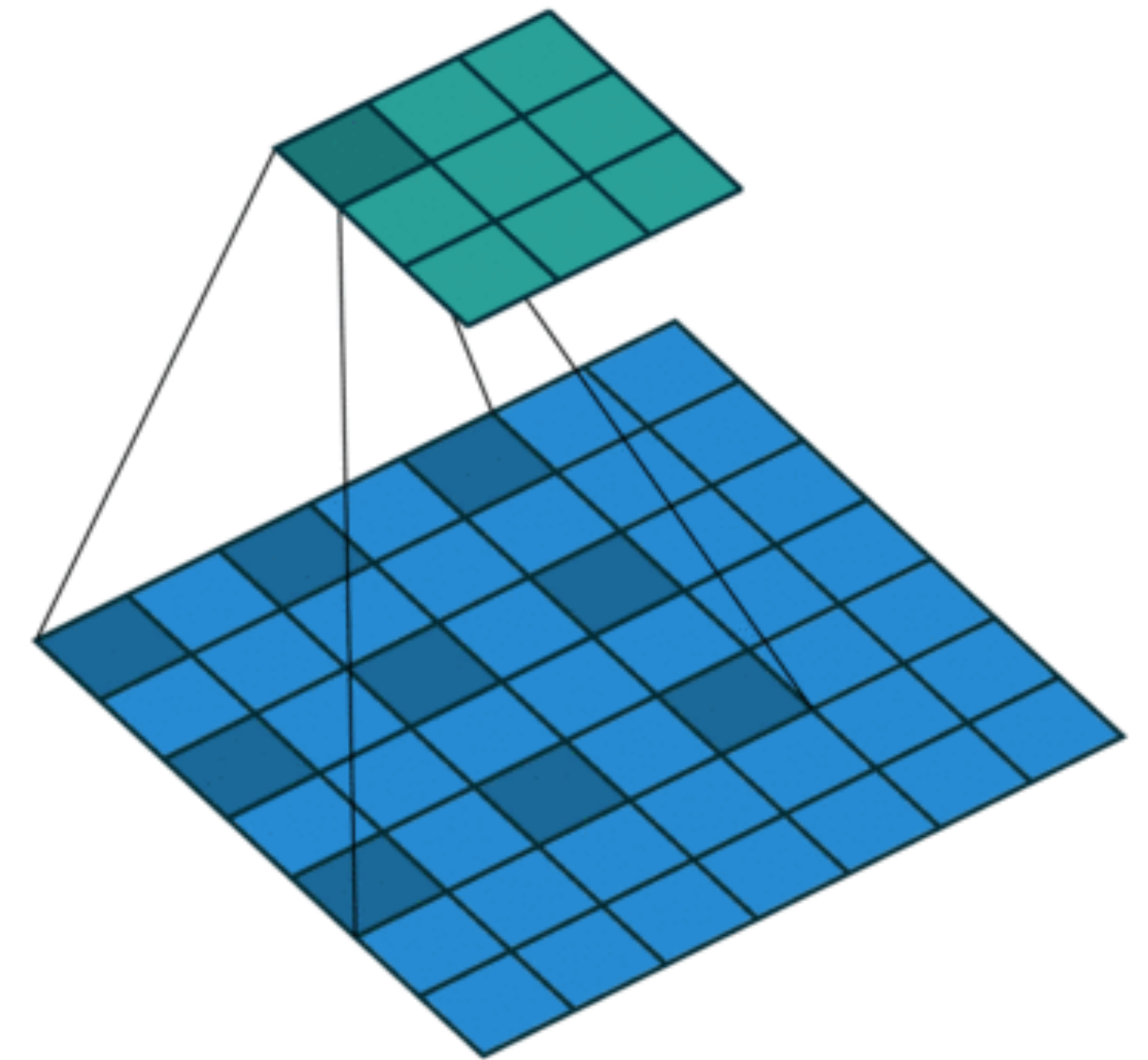
# Convolution Animations



**3 x 3 kernel with Padding**



**3 x 3 kernel with Padding, stride=2**



**3 x 3 kernel with Padding, stride=2,  
dilation=1**

# Convolution Summary

**Input:**  $C_{in} \times H \times W$

**Hyperparameters:**

- **Kernel size:**  $K_H \times K_W$
- **Number filters:**  $C_{out}$
- **Padding:**  $P$
- **Stride:**  $S$

**Weight matrix:**  $C_{out} \times C_{in} \times K_H \times K_W$   
giving  $C_{out}$  filters of size  $C_{in} \times K_H \times K_W$

**Bias vector:**  $C_{out}$

**Output size:**  $C_{out} \times H' \times W'$  where:

- $H' = (H - K + 2P) / S + 1$
- $W' = (W - K + 2P) / S + 1$

Common settings:

$K_H = K_W$  (Small square filters)       $K$  is odd.

$P = (K - 1) / 2$  ("Same" padding)

$C_{in}, C_{out} = 32, 64, 128, 256$  (powers of 2)

$K = 3, P = 1, S = 1$  (3x3 conv)

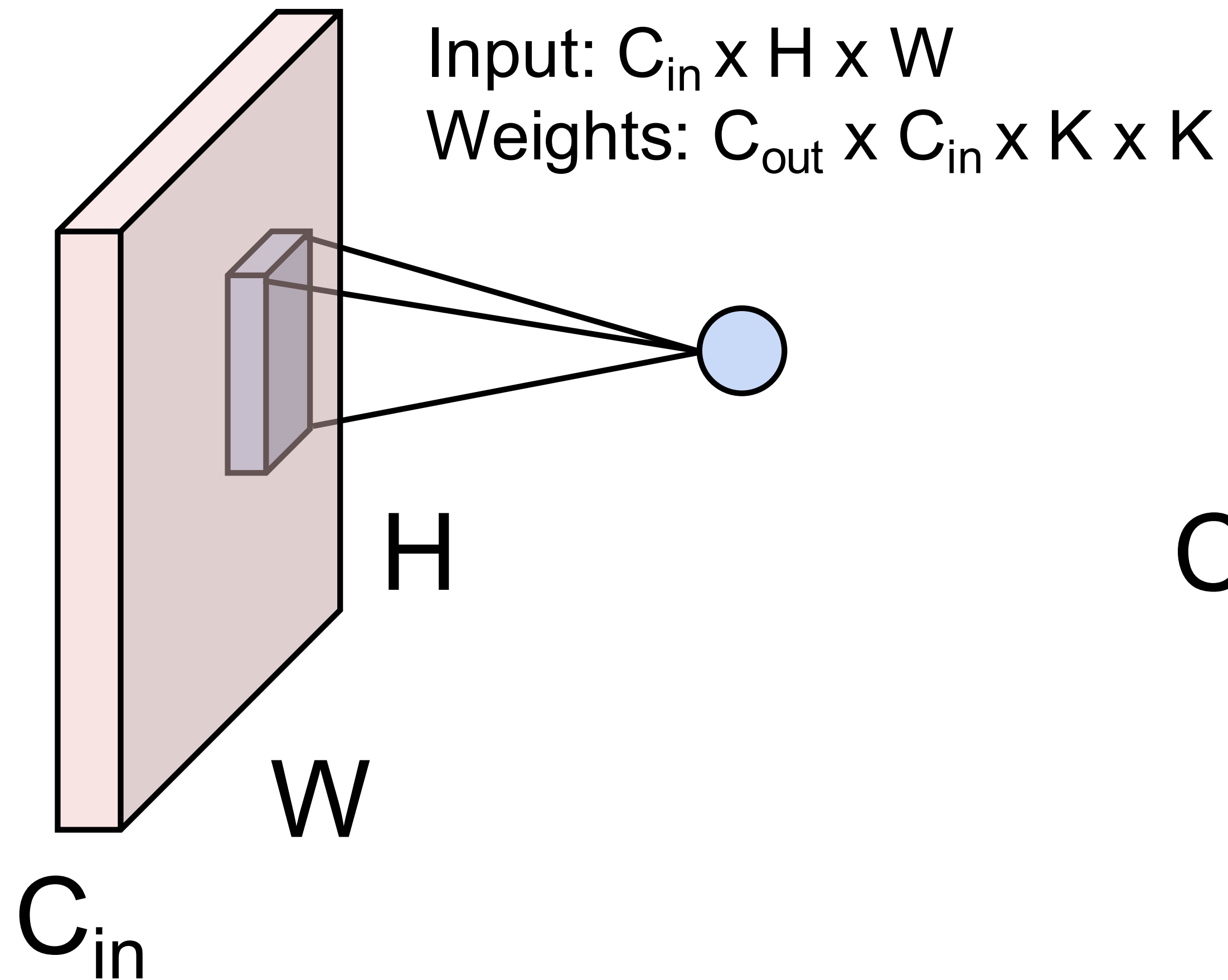
$K = 5, P = 2, S = 1$  (5x5 conv)

$K = 1, P = 0, S = 1$  (1x1 conv)

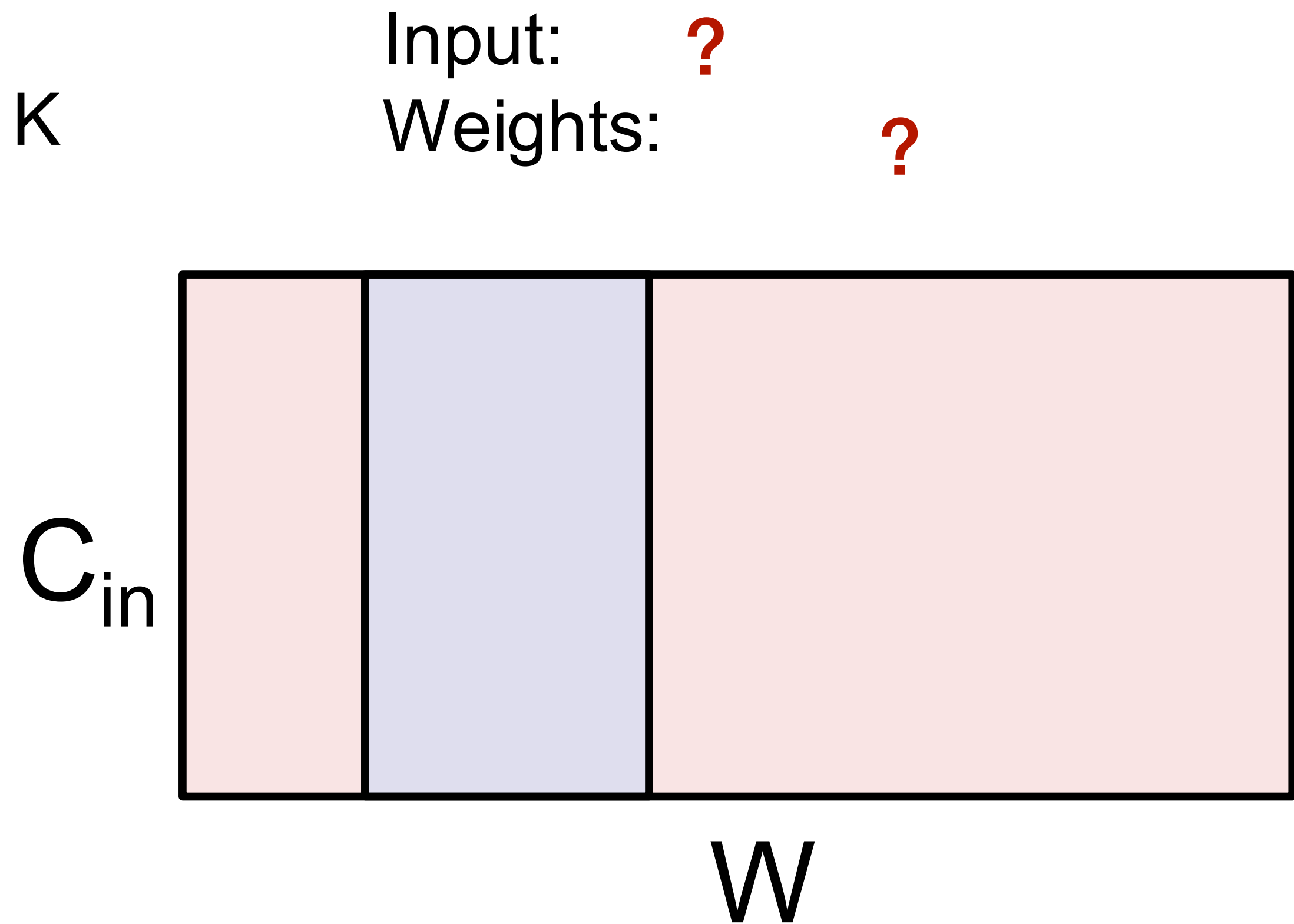
$K = 3, P = 1, S = 2$  (Downsample by 2)

# Other Types of Convolution

So far: 2D Convolution

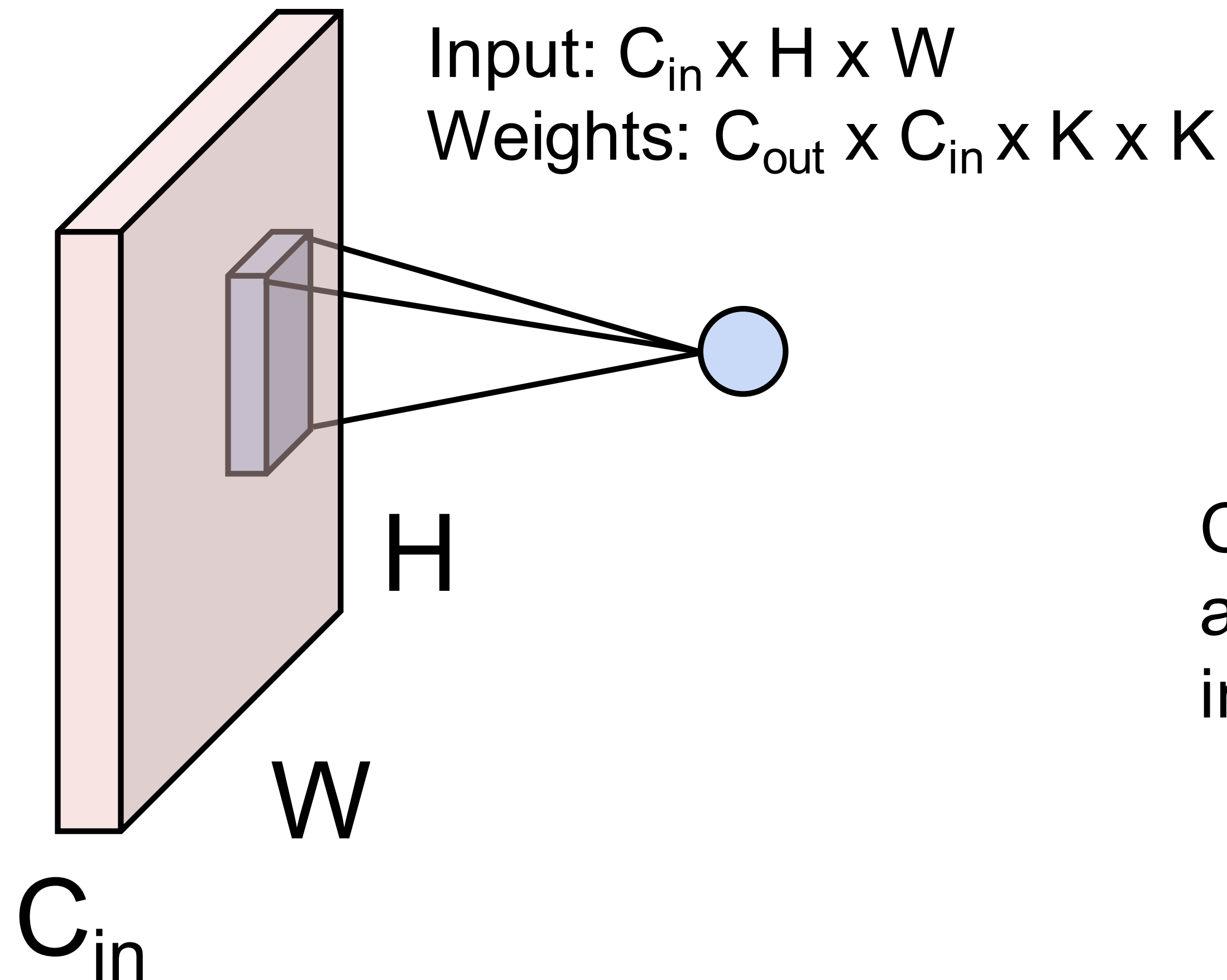


1D Convolution

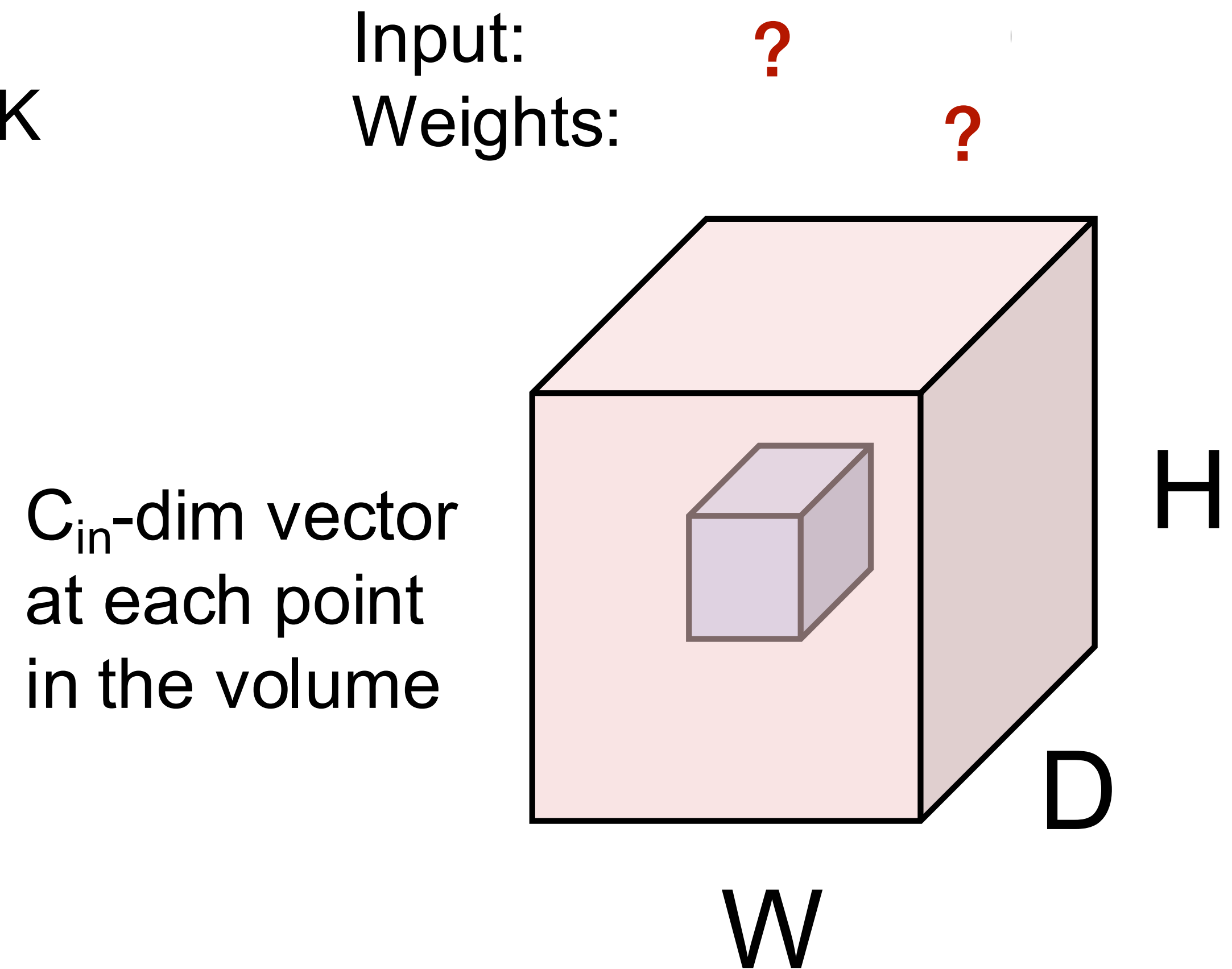


# Other Types of Convolution

So far: 2D Convolution



3D Convolution



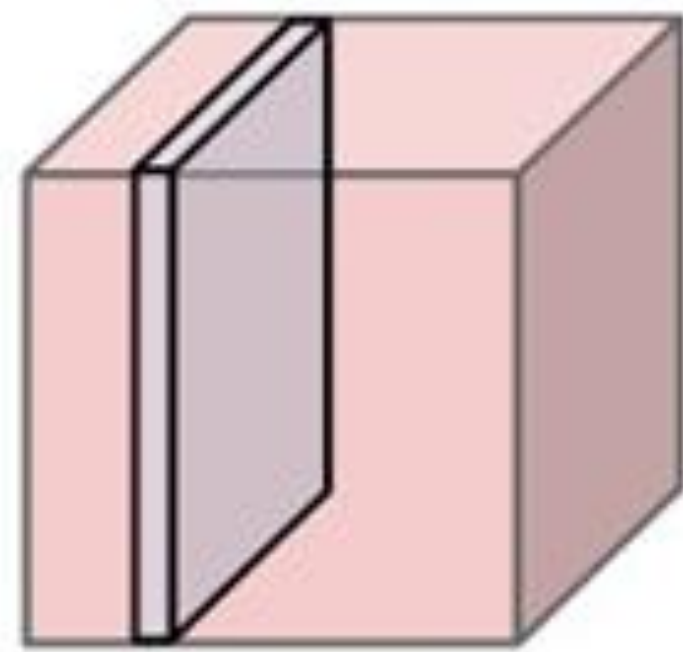
Pooling Layer

Normalization Layer

Dropout

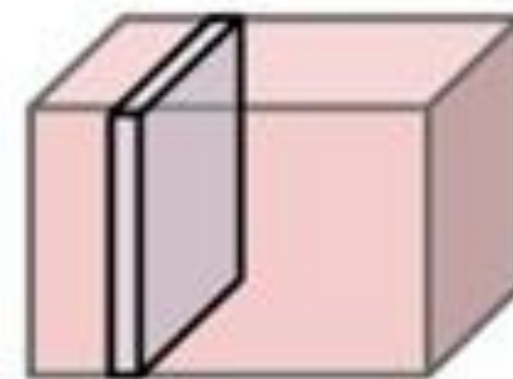
# Pooling Layers: Another way to downsample

64 x 224 x 224



pool

64 x 112 x 112



downsampling



Given an input  $C \times H \times W$ ,  
downsample each  $1 \times H \times W$  plane

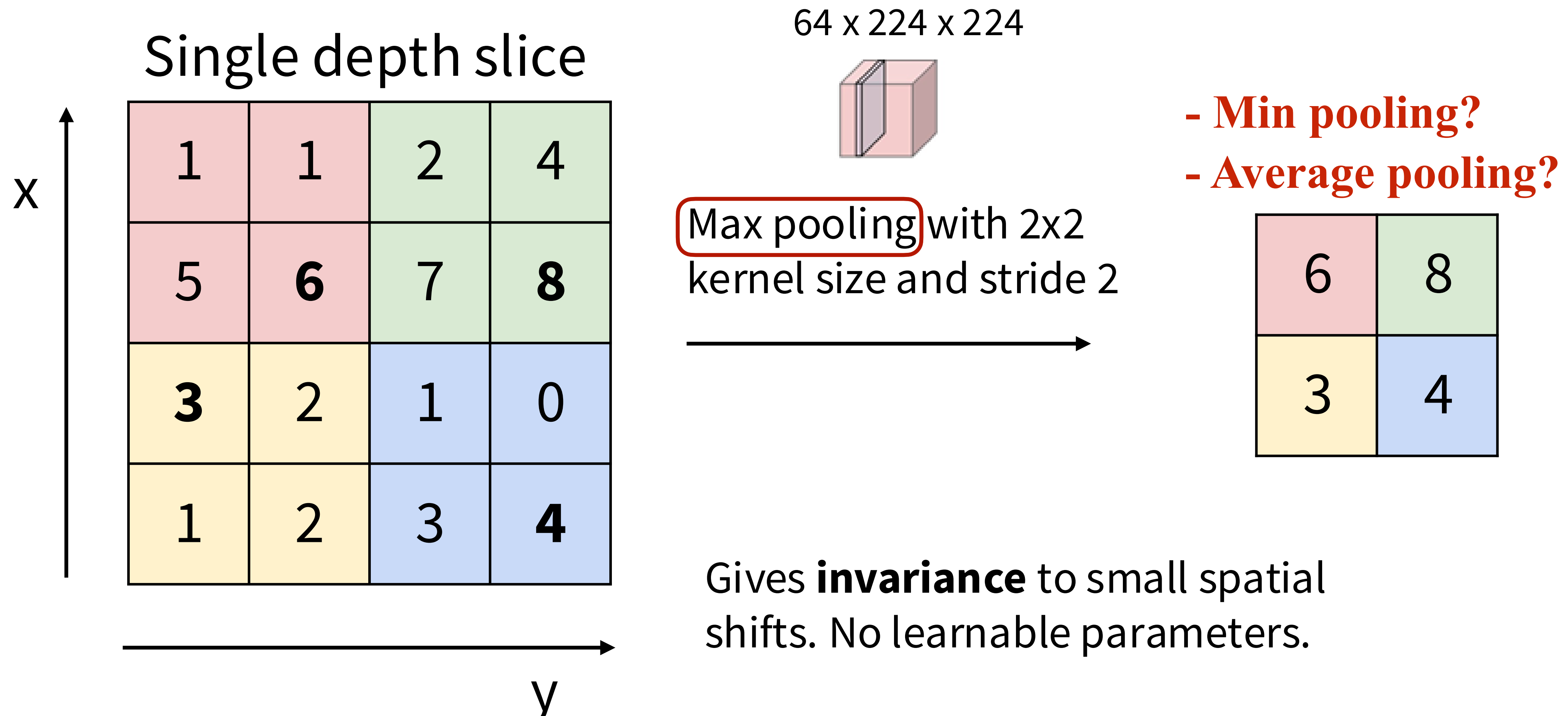
## Hyperparameters:

Kernel Size

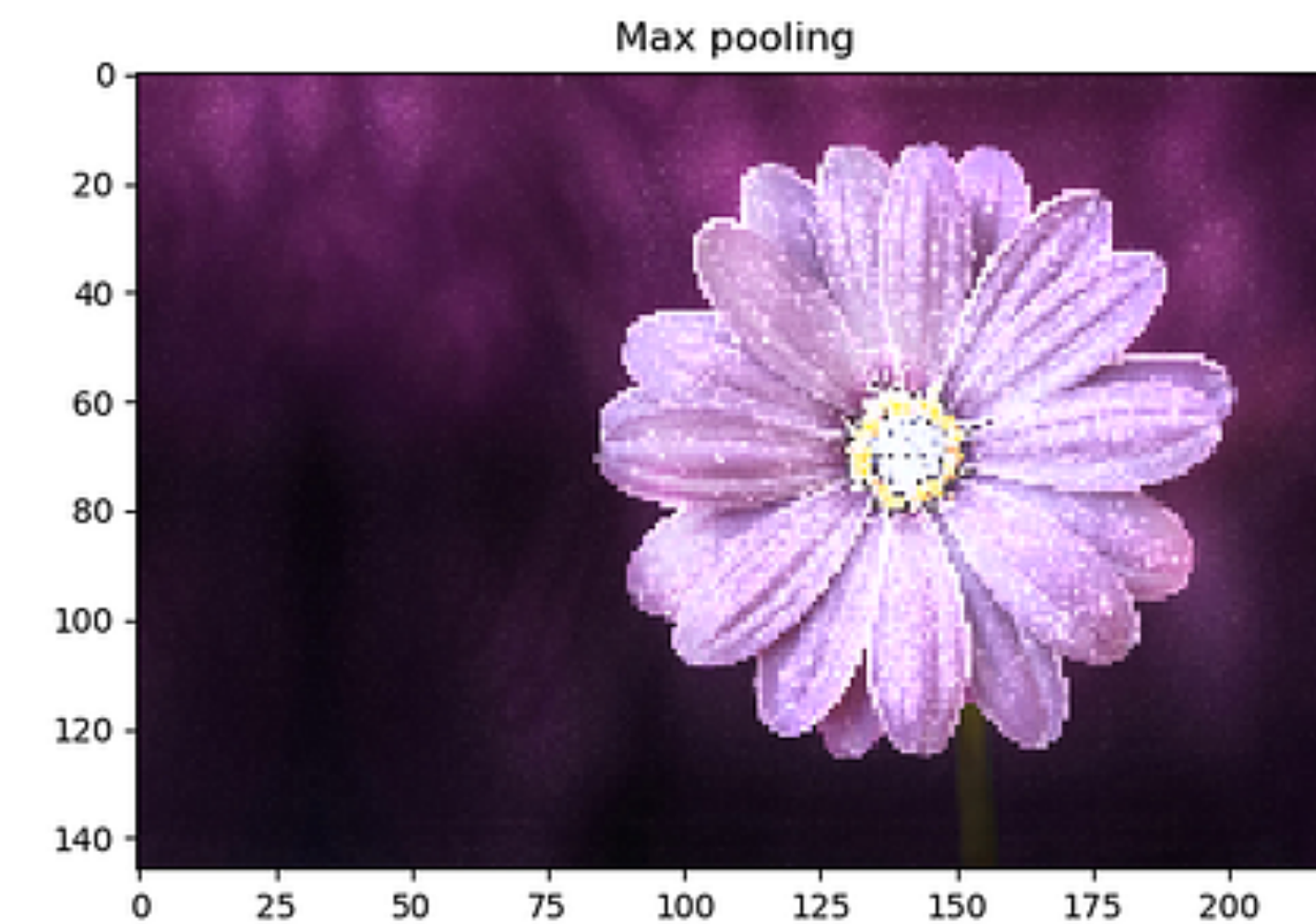
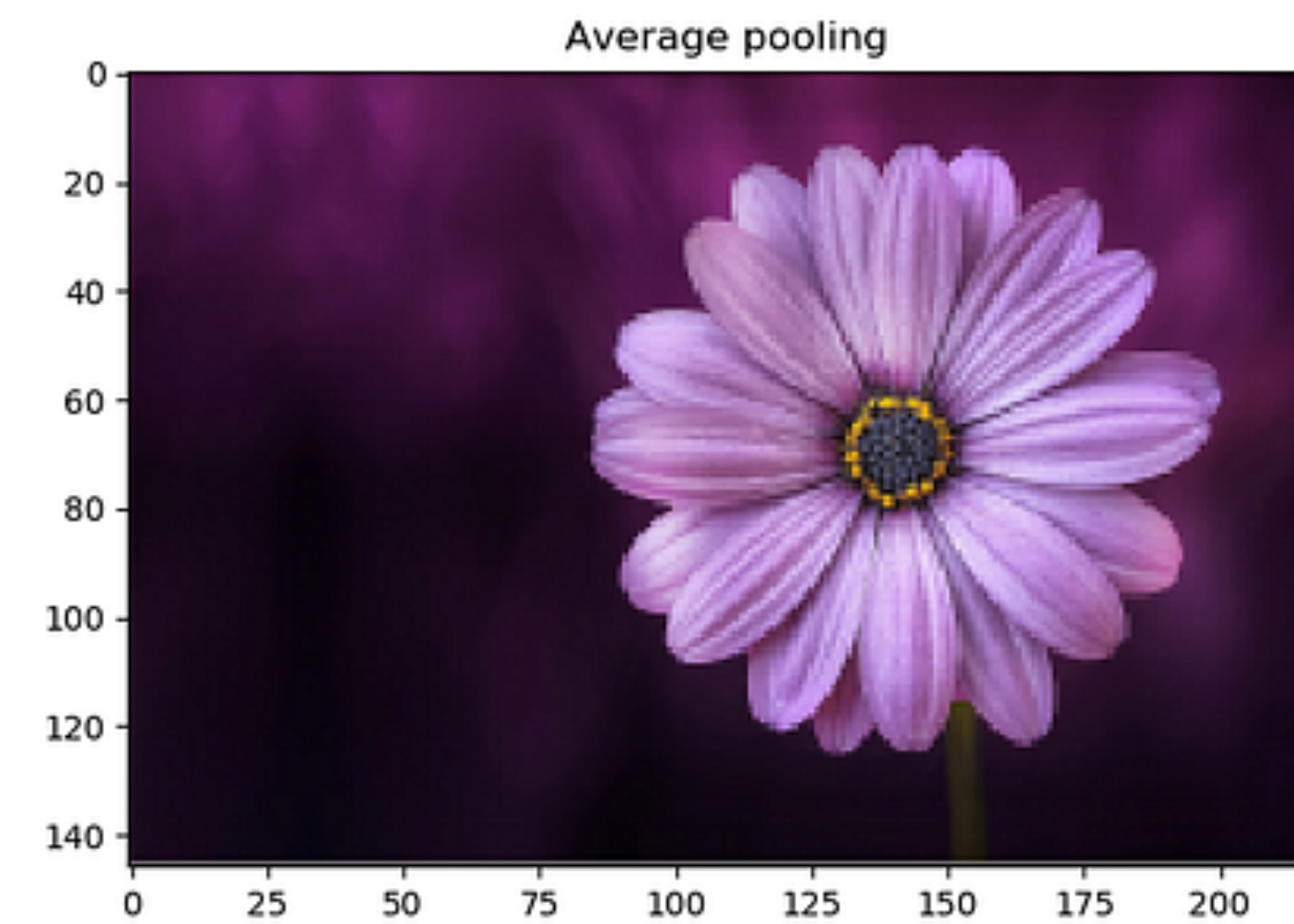
Stride

Pooling function

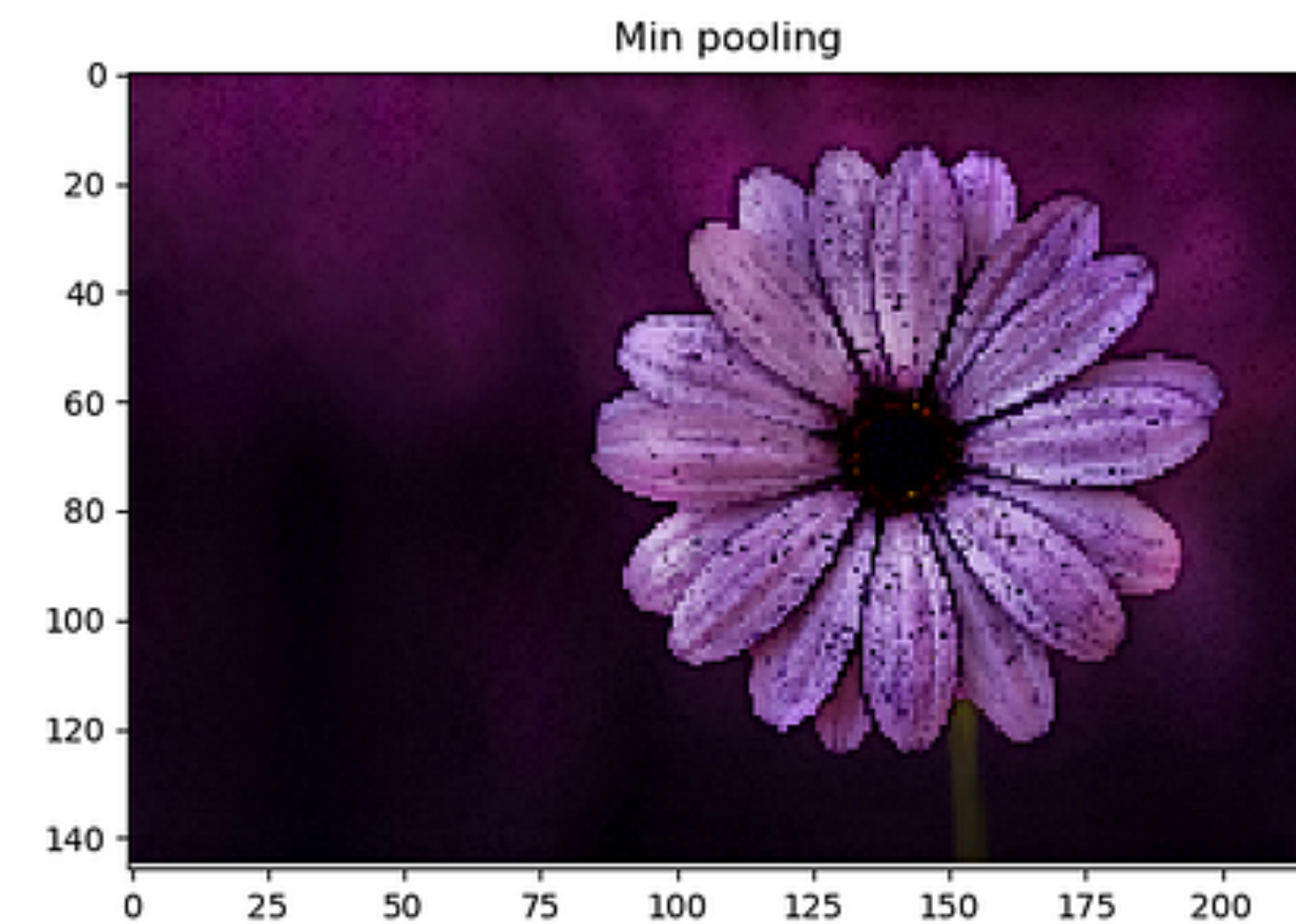
# Pooling Layers: Another way to downsample



Smooth but loose sharp details

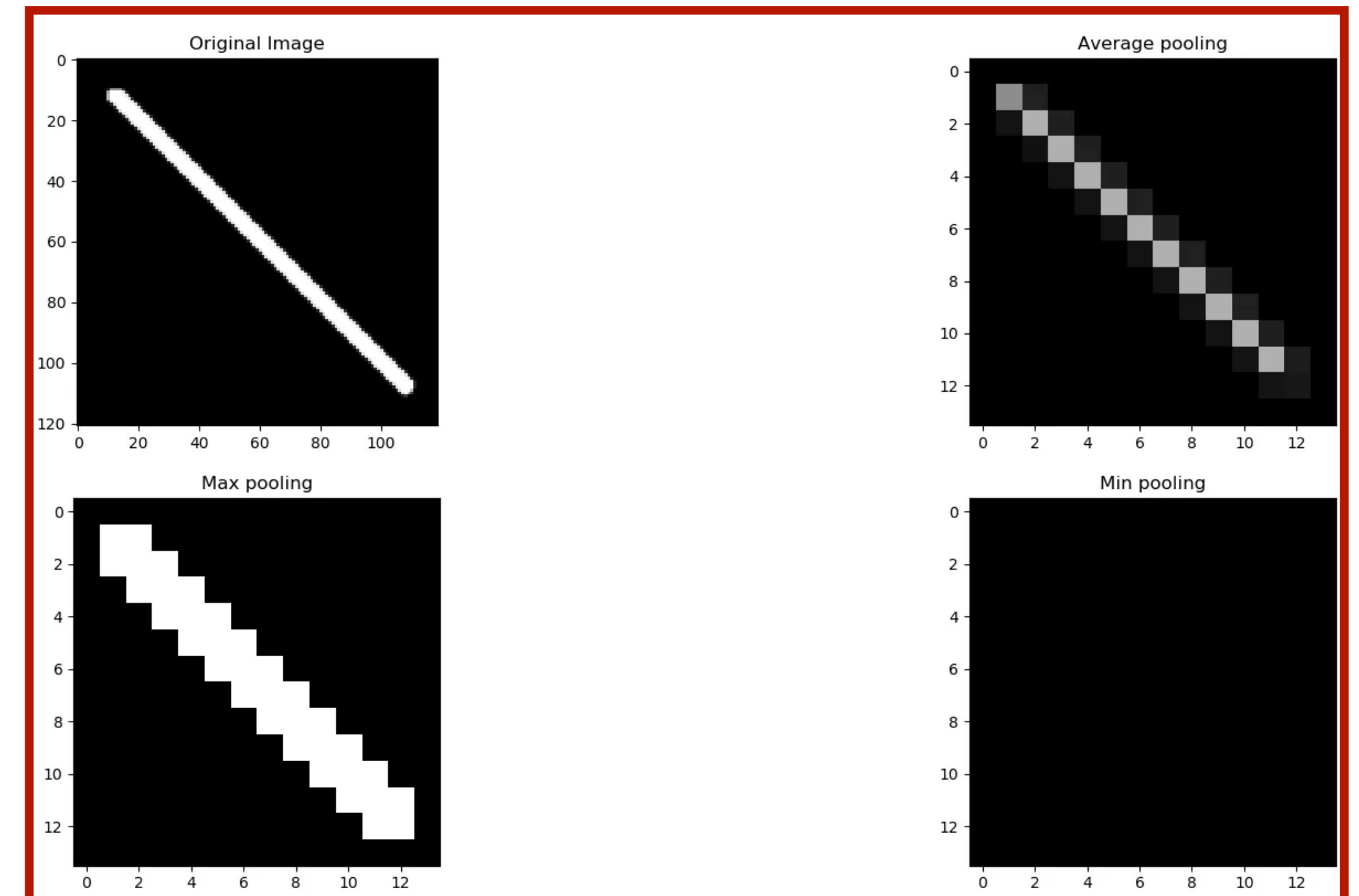
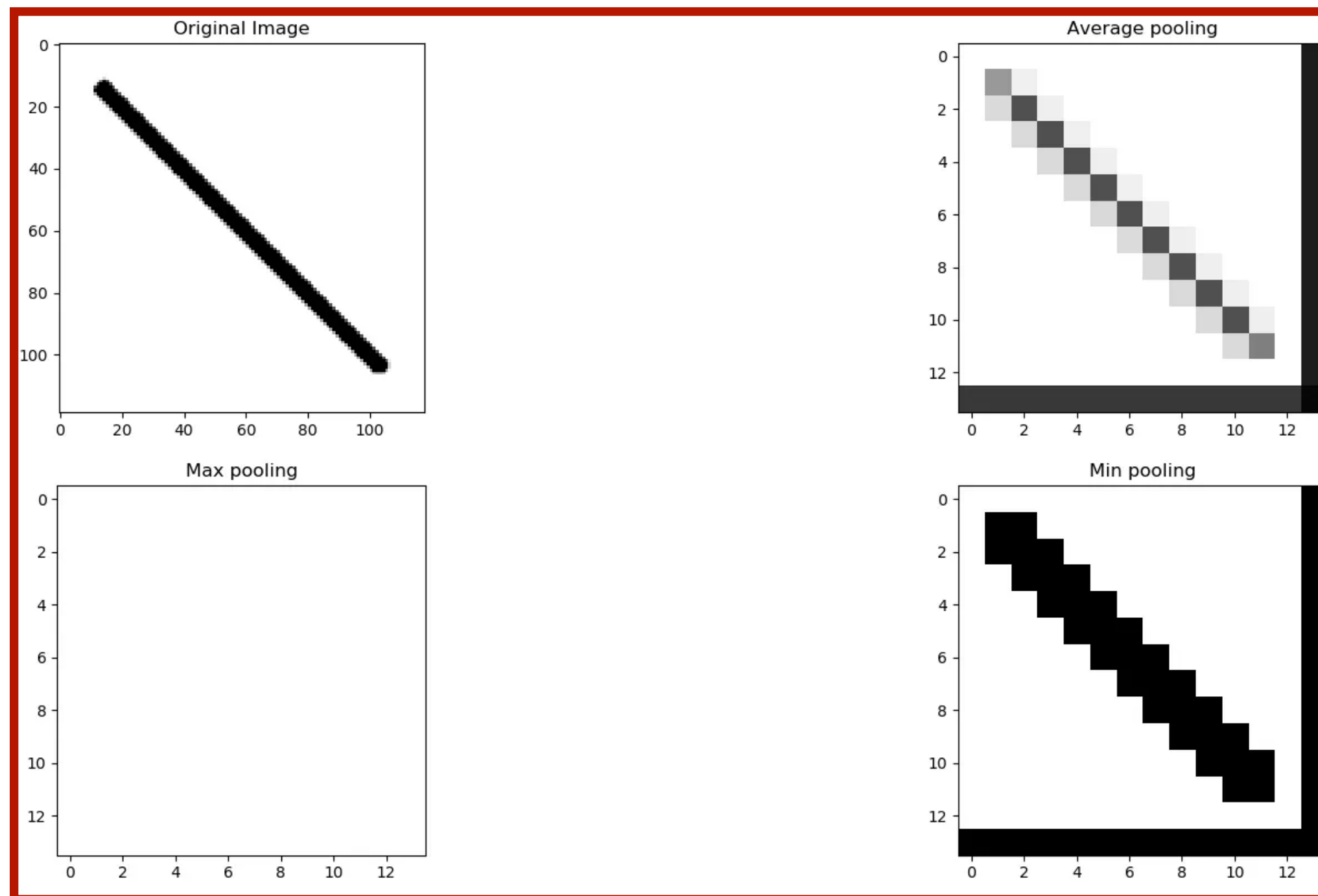


Preserves strong responses but is sensitive to high-value noise



Preserves weak responses but is sensitive to low-value noise

# Pooling Layers



Black: 0

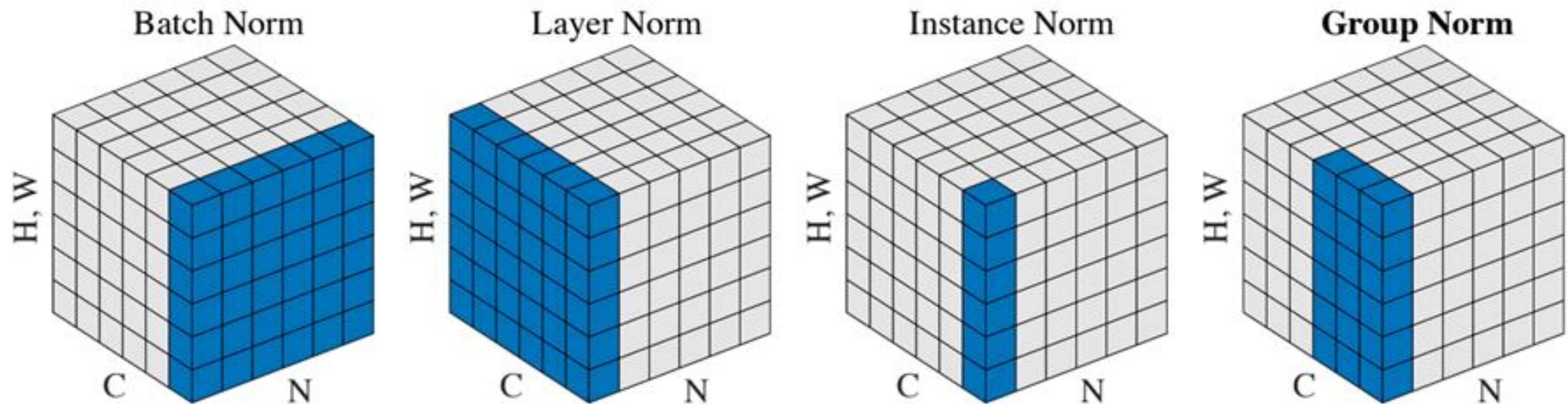
White: 255

# Example Normalization Layer: LayerNorm

**High-level Idea: Learn parameters that let us **scale** / **shift** the input data**

1. Normalize input data
2. Scale / shift using learned parameters

# Other Normalization Layers



**You will implement some of these in assignment 2!**

# Example Normalization Layer: LayerNorm

High-level Idea: Learn parameters that let us **scale / shift** the input data

1. Normalize input data
2. Scale / shift using learned parameters

$$\mathbf{x} : \text{CHW} \times \text{N}$$

Normalize



Statistics calculated per batch →

$$\boldsymbol{\mu}, \boldsymbol{\sigma} : \text{CHW} \times \mathbf{1}$$

Learned parameters applied to each sample →

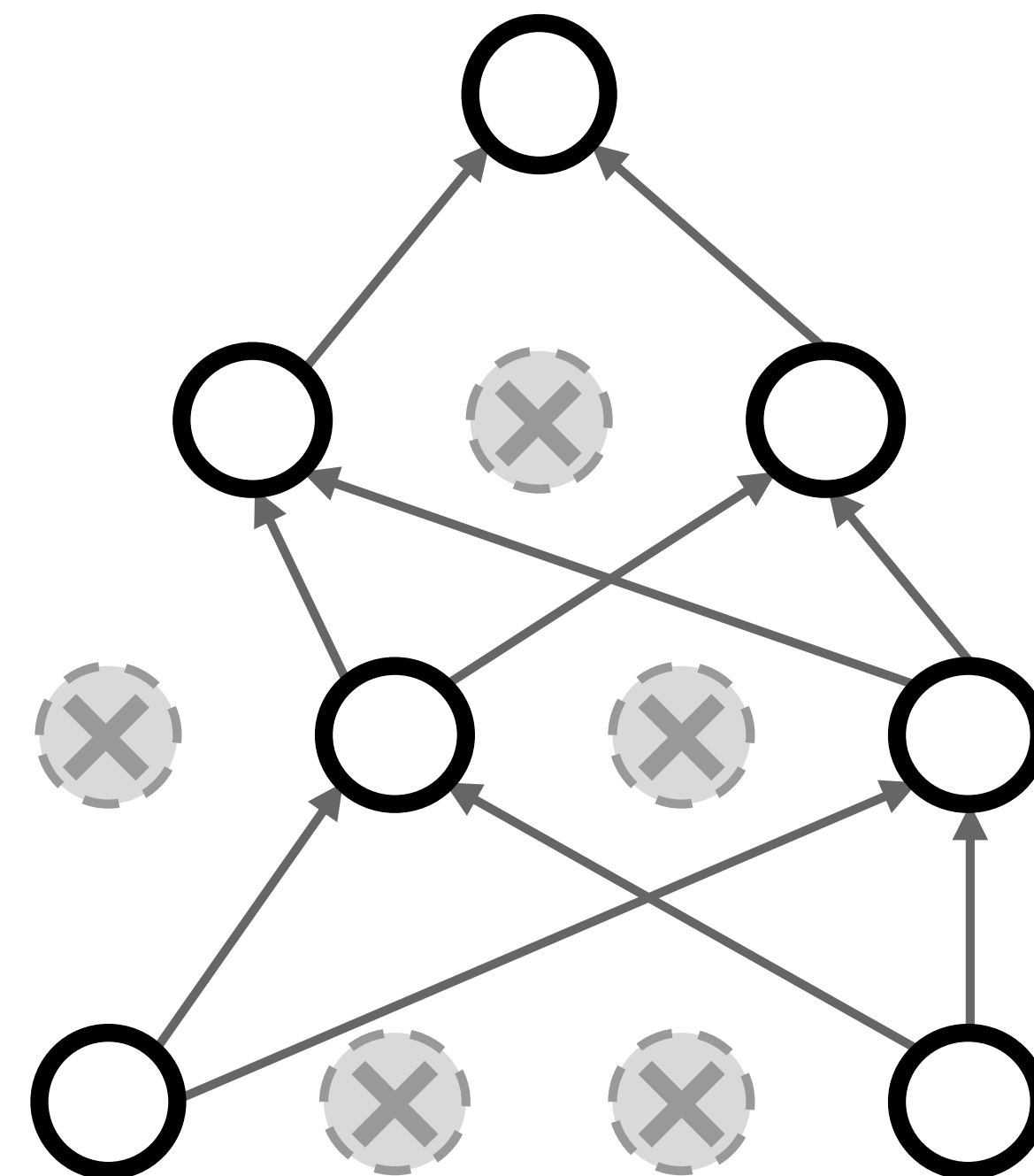
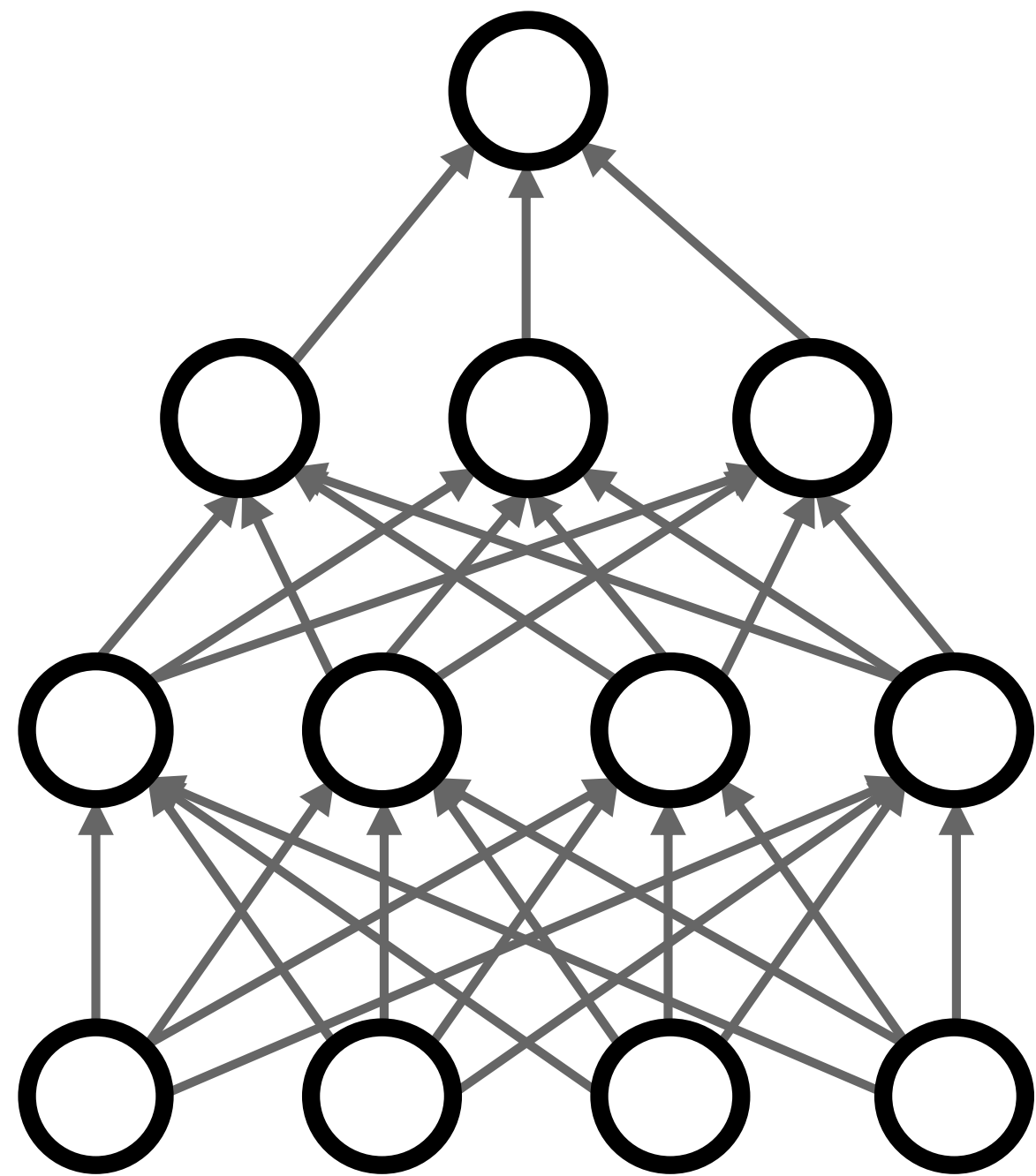
$$\boldsymbol{\gamma}, \boldsymbol{\beta} : \mathbf{1} \times \text{N}$$

$$\mathbf{y} = \boldsymbol{\gamma} (\mathbf{x} - \boldsymbol{\mu}) / \boldsymbol{\sigma} + \boldsymbol{\beta}$$

# Regularization: Dropout

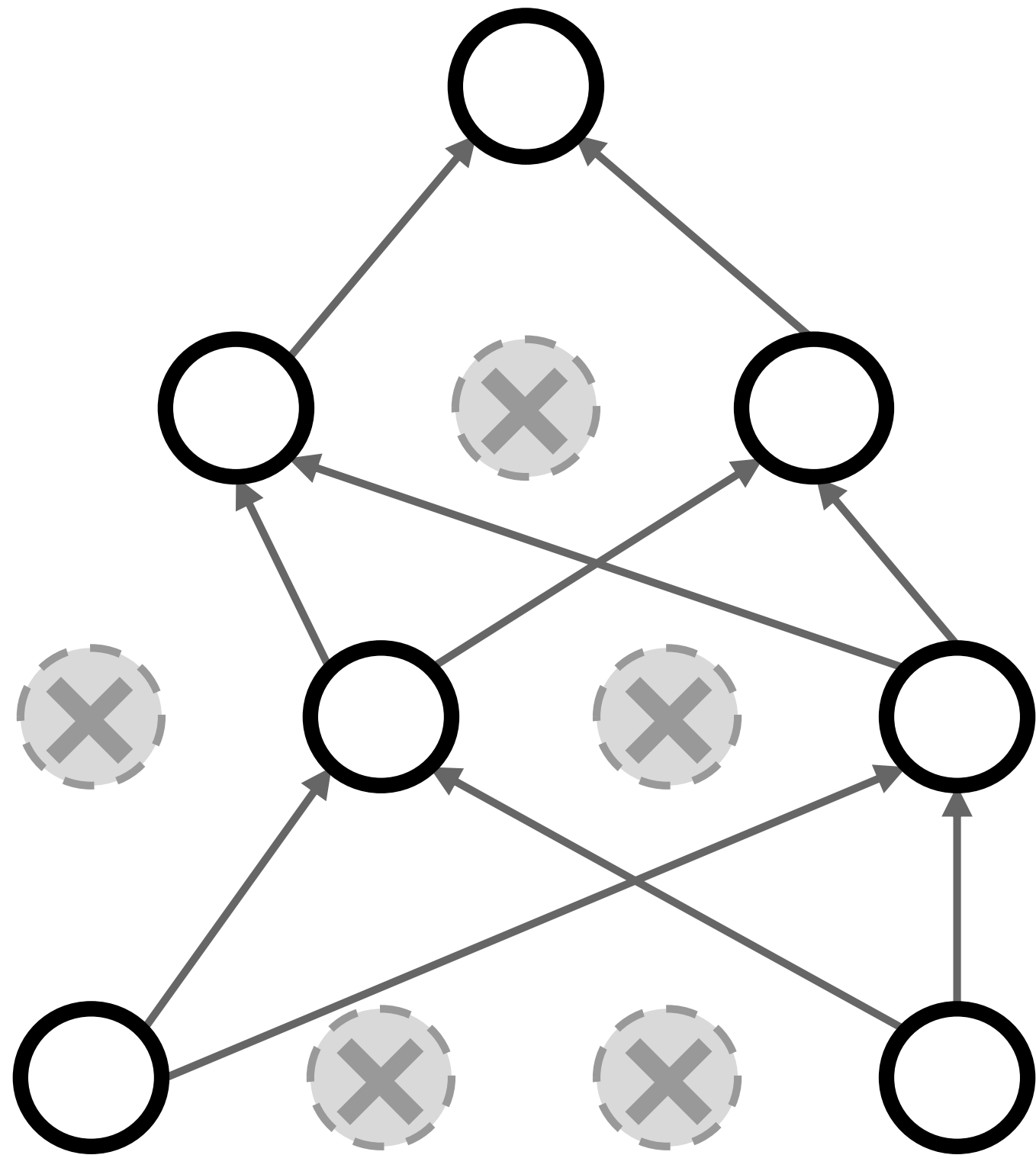
In each forward pass, randomly set some neurons to zero

Probability of dropping is a hyperparameter; 0.5 is common



# Regularization: Dropout

How can this possibly be a good idea?

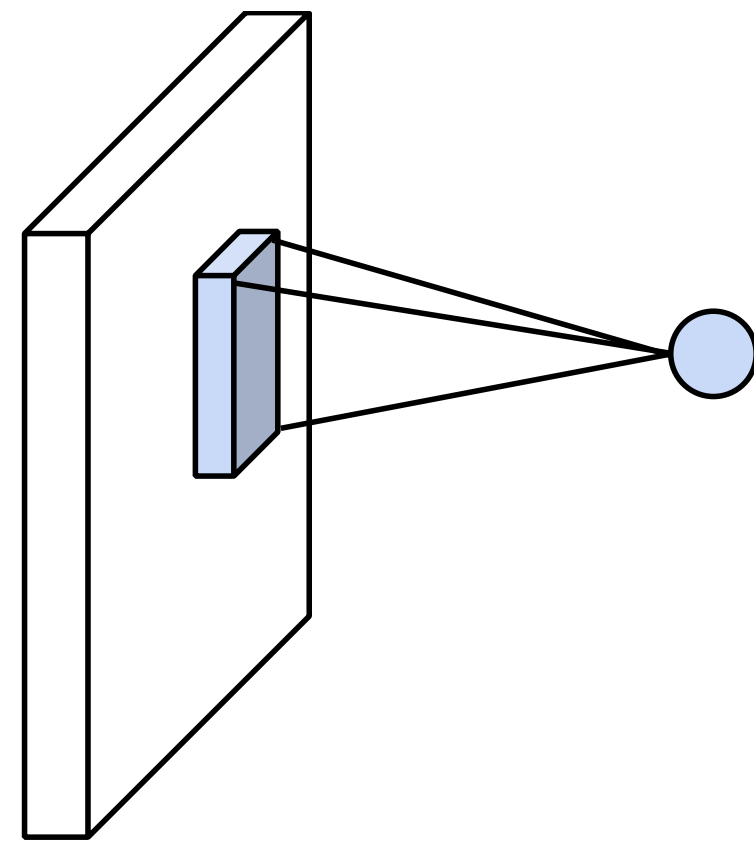


Forces the network to have a redundant representation;  
Prevents co-adaptation of features

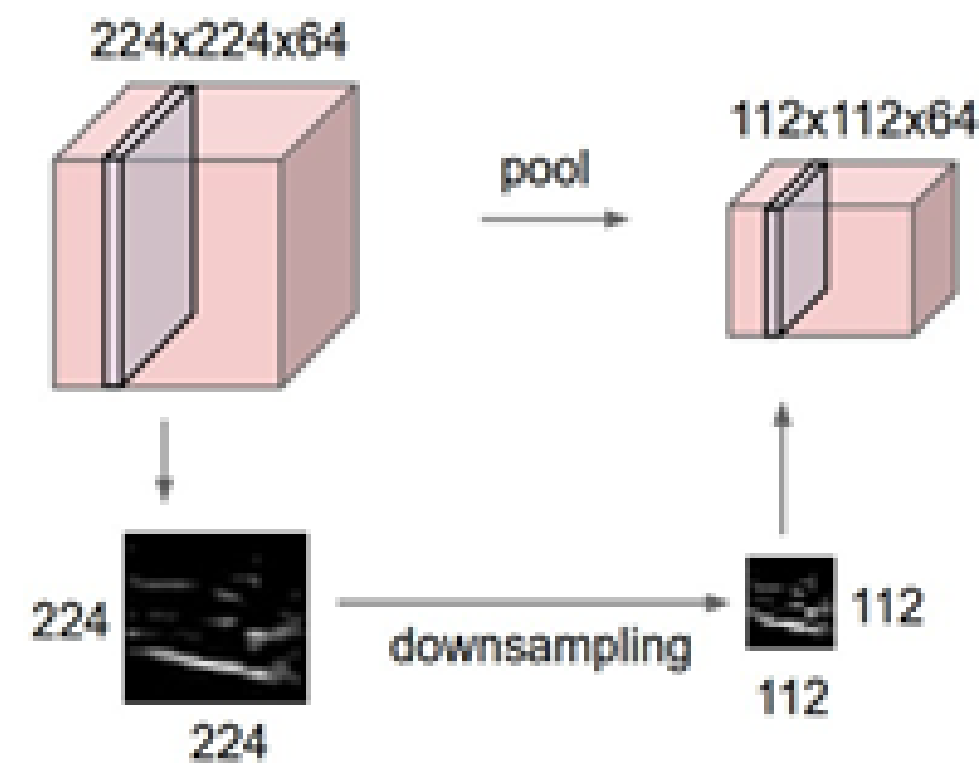


# Components of CNNs

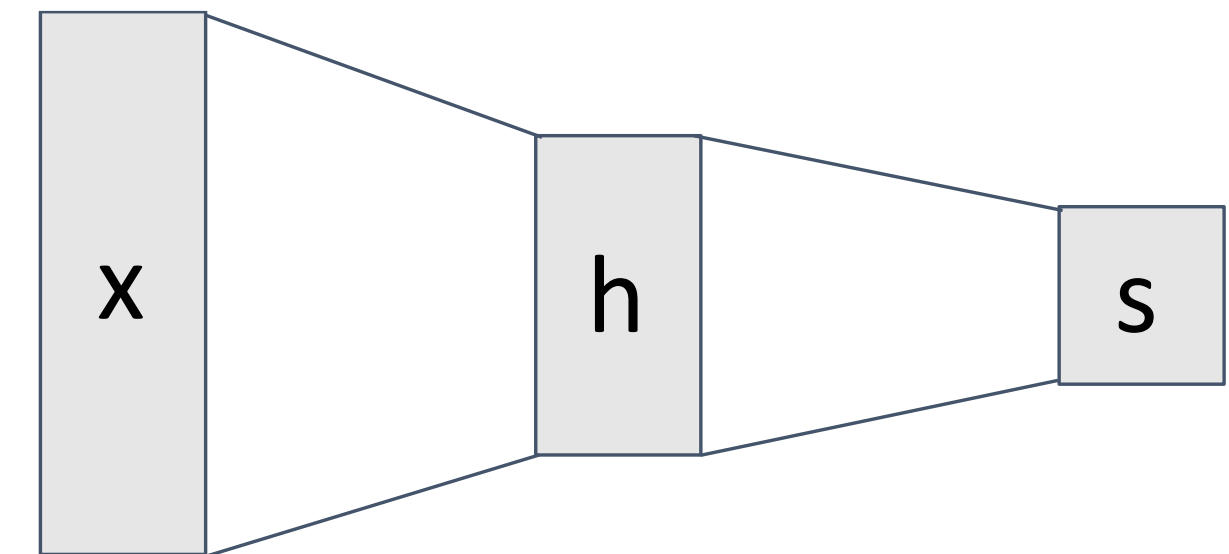
## Convolution Layers



## Pooling Layers



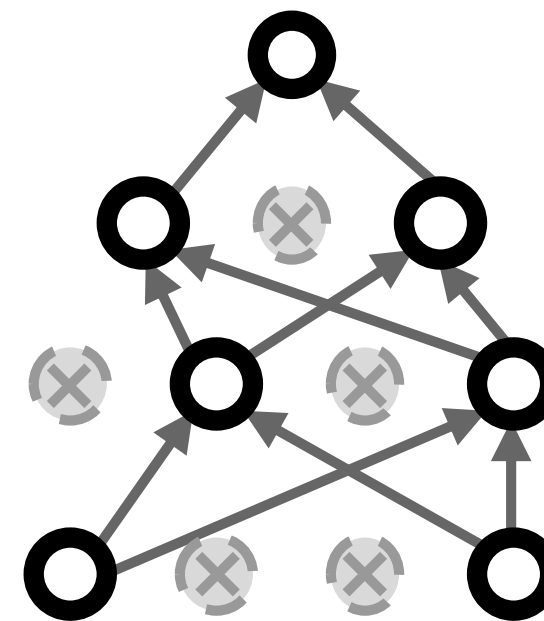
## Fully-Connected Layers



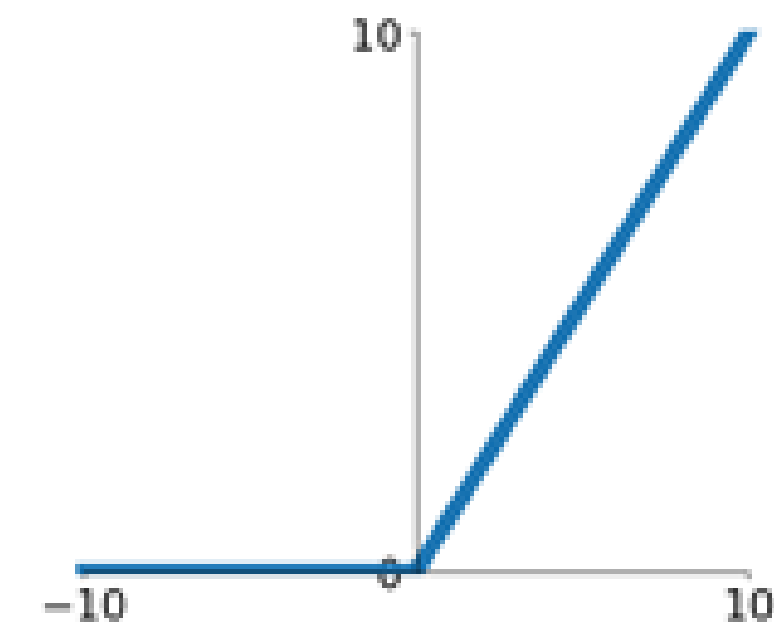
## Normalization Layers

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

## Dropout (sometimes)



## Activation Functions



# Convolutional Neural Networks

How to build CNNs?

Layers in CNNs

Activation Functions

**CNN Architecture**

How to train CNNs?

Weight Initialization

Data Preprocessing & Augmentation

Hyperparameter Selection

# Convolutional Neural Networks

**Convolution** and **pooling** operators extract features while respecting 2D image structure

**Fully-Connected** layers form an MLP at the end to predict scores

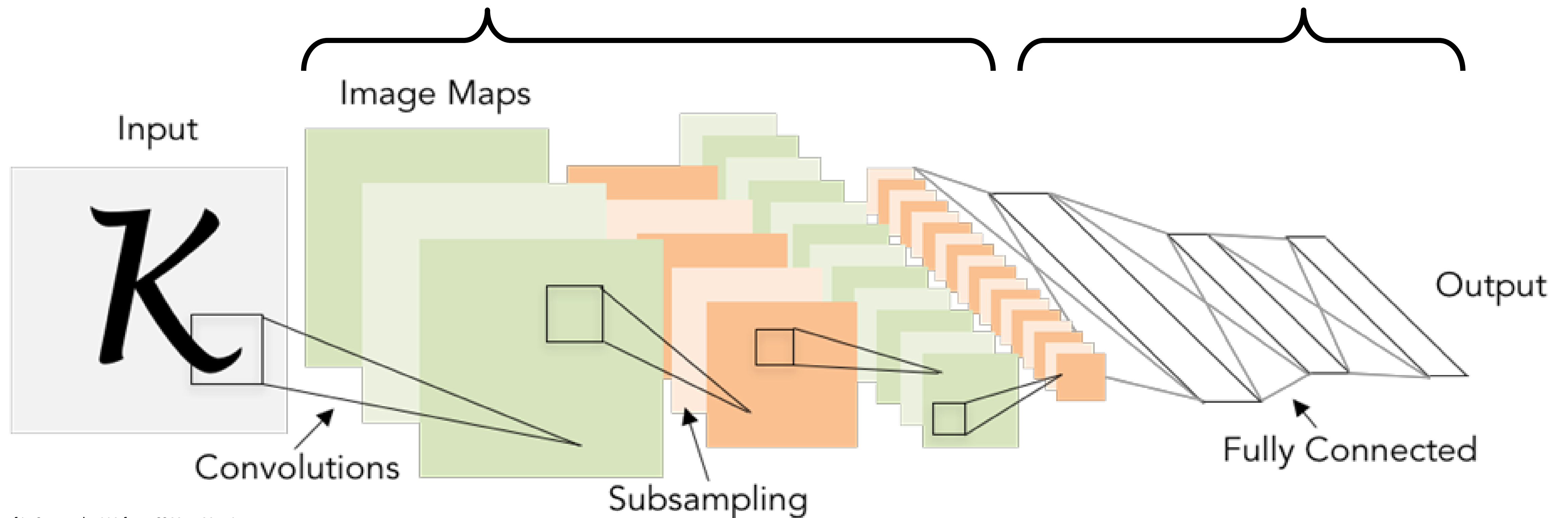
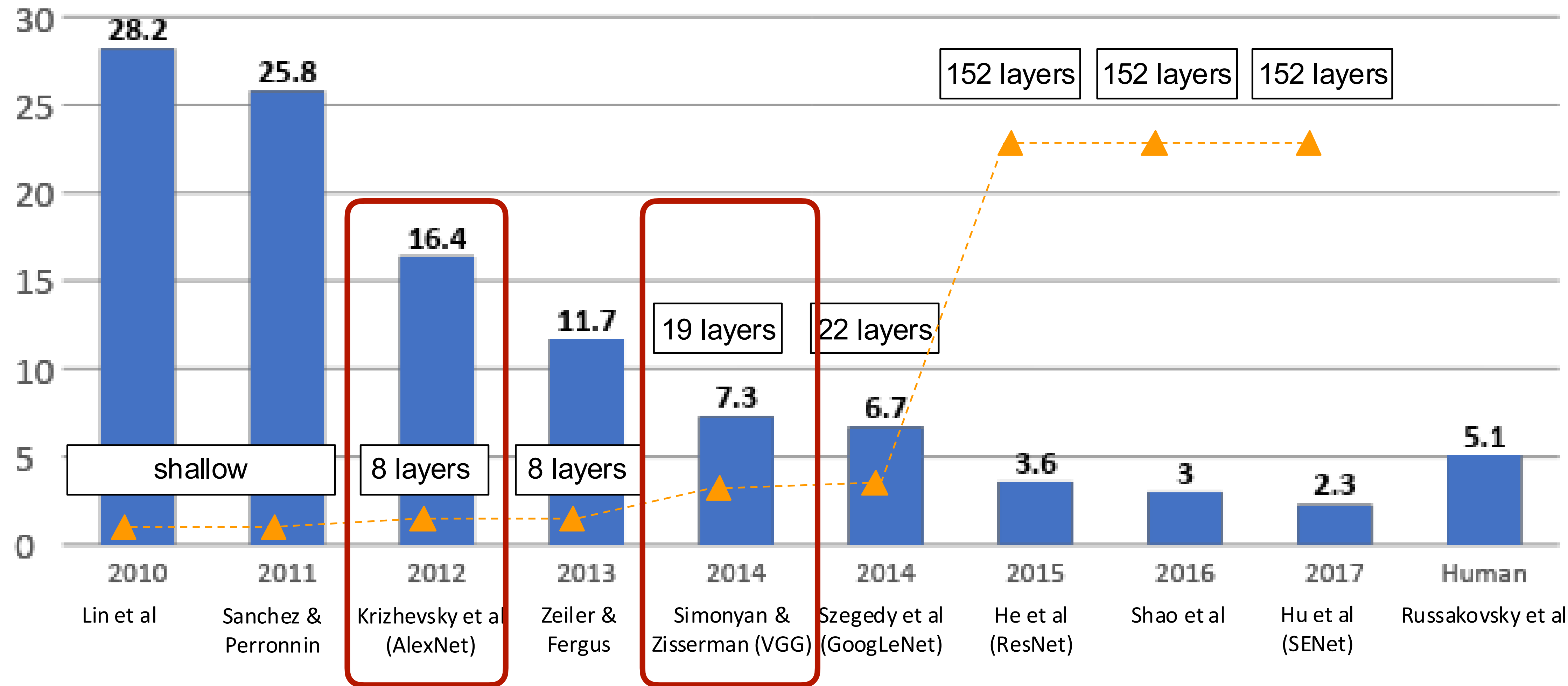


Illustration of LeCun et al. 1998 from CS231n 2017 Lecture 1

**LeNet5 in 1998 (Code demo)**

# ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners



# Case Study: VGGNet

[Simonyan and Zisserman, 2014]

Small filters, Deeper networks

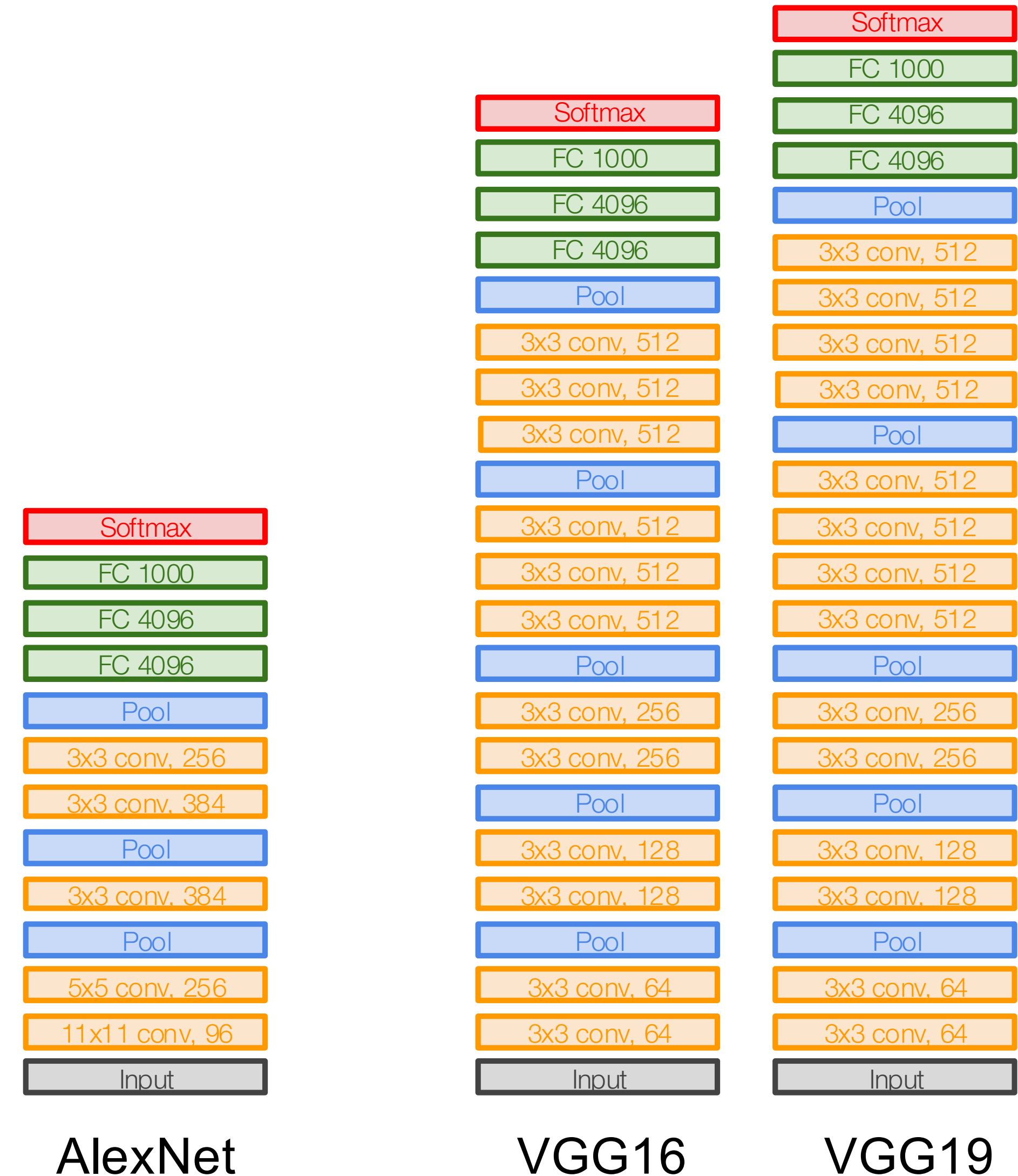
8 layers (AlexNet)

-> 16 - 19 layers (VGG16Net)

Only 3x3 CONV stride 1, pad 1  
and 2x2 MAX POOL stride 2

11.7% top 5 error in ILSVRC'13  
(ZFNet)

-> 7.3% top 5 error in ILSVRC'14



# How to train CNNs?



Github link: [our course GitHub repo](#)

```
class AlexNetCIFAR(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=3, padding=1), # 32x32
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2), # 16x16
            nn.Conv2d(64, 192, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2), # 8x8
            nn.Conv2d(192, 384, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(384, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(256, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2), # 4x4
        )
        self.avgpool = nn.AdaptiveAvgPool2d((4, 4))
        self.classifier = nn.Sequential(
            nn.Dropout(0.5),
            nn.Linear(256 * 4 * 4, 1024),
            nn.ReLU(inplace=True),
            nn.Dropout(0.5),
            nn.Linear(1024, 512),
            nn.ReLU(inplace=True),
            nn.Linear(512, num_classes),
        )

    def forward(self, x):
        x = self.features(x)
        x = self.avgpool(x)
        x = torch.flatten(x, 1)
        return self.classifier(x)
```

```
model = AlexNetCIFAR(num_classes=len(class_names)).to(device)
model
```

```
Input: (1, 3, 32, 32)
features[ 0] Conv2d      -> (1, 64, 32, 32)
features[ 1] ReLU       -> (1, 64, 32, 32)
features[ 2] MaxPool2d  -> (1, 64, 16, 16)
features[ 3] Conv2d     -> (1, 192, 16, 16)
features[ 4] ReLU       -> (1, 192, 16, 16)
features[ 5] MaxPool2d  -> (1, 192, 8, 8)
features[ 6] Conv2d     -> (1, 384, 8, 8)
features[ 7] ReLU       -> (1, 384, 8, 8)
features[ 8] Conv2d     -> (1, 256, 8, 8)
features[ 9] ReLU       -> (1, 256, 8, 8)
features[10] Conv2d     -> (1, 256, 8, 8)
features[11] ReLU       -> (1, 256, 8, 8)
features[12] MaxPool2d  -> (1, 256, 4, 4)
avgpool: (1, 256, 4, 4)
Output logits: (1, 10)

Trainable parameters: 6,976,842
```

# Convolutional Neural Networks

How to build CNNs?

Layers in CNNs

Activation Functions

CNN Architecture

How to train CNNs?

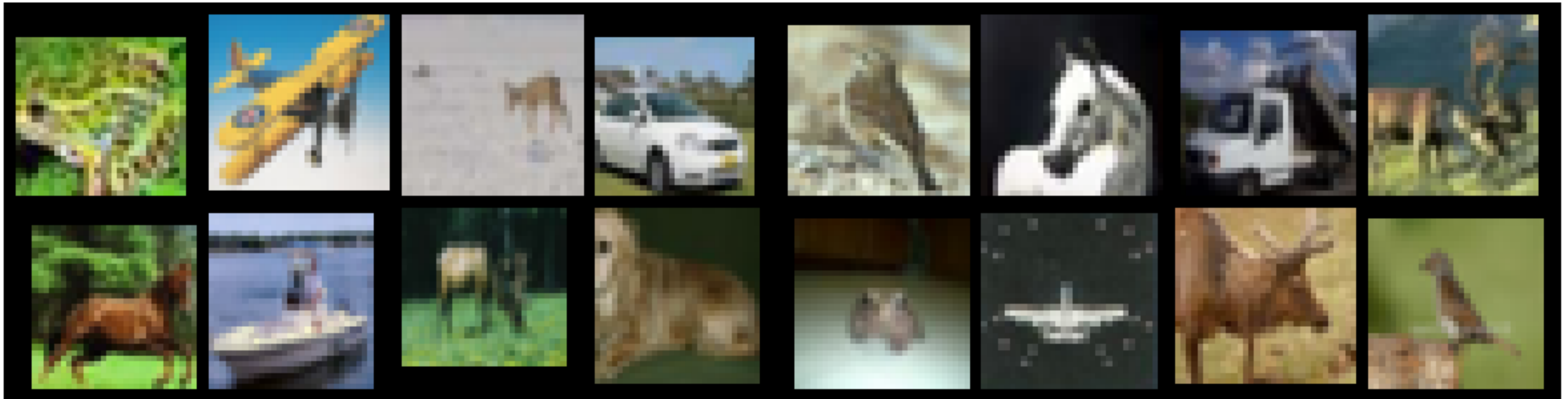
Weight Initialization

Data Preprocessing & Augmentation

Hyperparameter Selection

# Data: CIFAR10

CIFAR-10 Training Examples



Categories:

```
['frog', 'airplane', 'deer', 'automobile', 'bird', 'horse', 'truck', 'deer', 'horse',  
'ship', 'deer', 'dog', 'frog', 'airplane', 'deer', 'bird']
```

# Data preprocessing

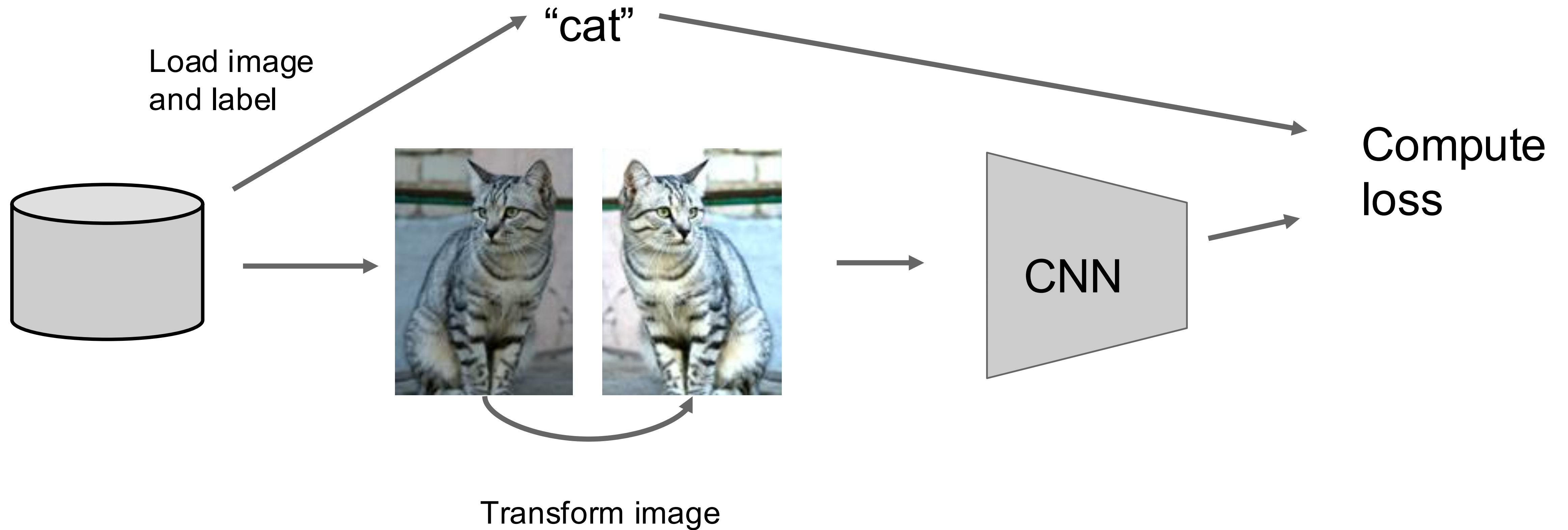
## Image Normalization:

center and scale for each channel

- Subtract per-channel mean and  
Divide by per-channel std (almost all modern models)  
(stats along each channel = 3 numbers)
- Requires pre-computing means and std for each  
pixel channel (given your dataset)

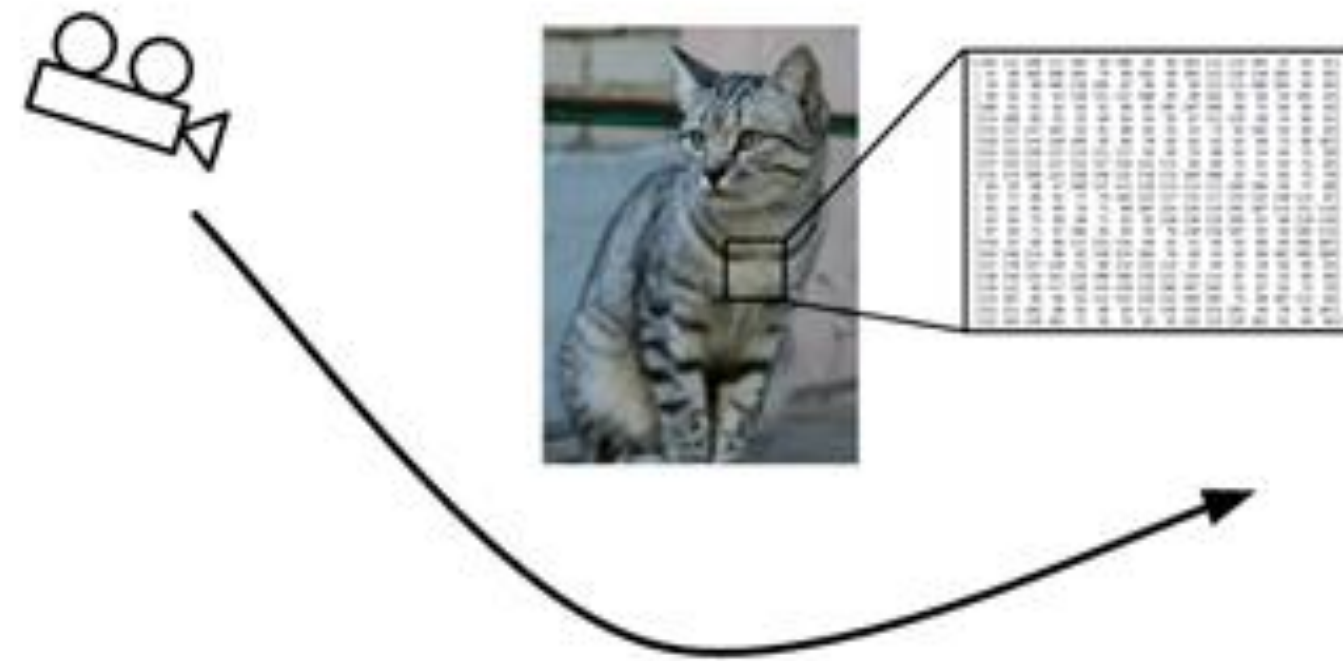
```
norm_pixel[i,j,c] = (pixel[i,j,c] - np.mean(pixel[:, :, c])) / np.std(pixel[:, :, c])
```

# Regularization: Data Augmentation



# Recall from last time: Challenges of recognition

Viewpoint



Illumination



[This image](#) is [CC0 1.0](#) public domain

Deformation



[This image](#) by [Umberto Salvagnin](#) is licensed under [CC-BY 2.0](#)

Occlusion



[This image](#) by [jonsson](#) is licensed under [CC-BY 2.0](#)

Clutter



[This image](#) is [CC0 1.0](#) public domain

Intraclass Variation



[This image](#) is [CC0 1.0](#) public domain

# Data Augmentation

## Horizontal Flips



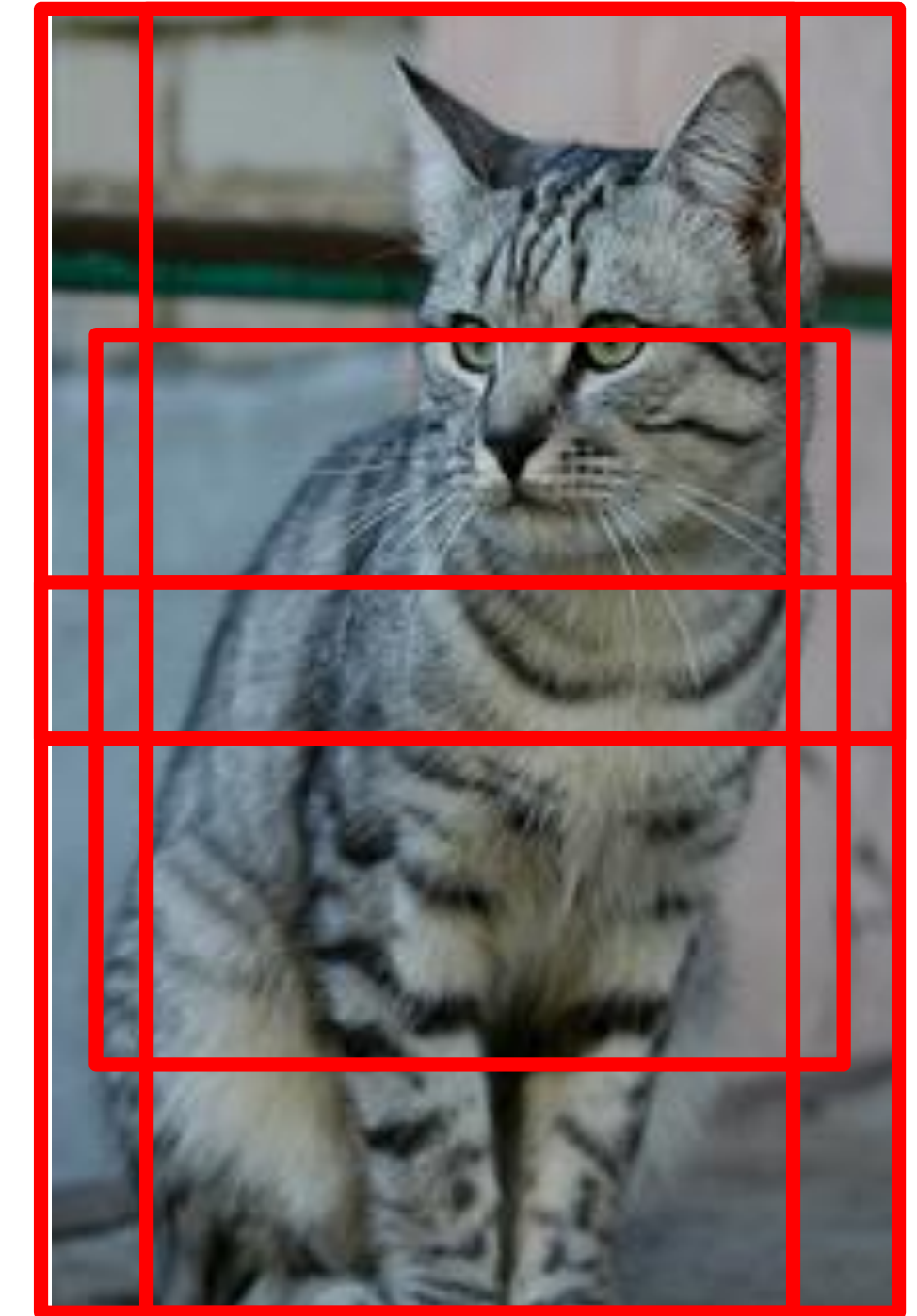
# Data Augmentation

## Random crops and scales

**Training:** sample random crops / scales

ResNet:

1. Pick random  $L$  in range  $[256, 480]$
2. Resize training image, short side =  $L$
3. Sample random  $224 \times 224$  patch



**Test Time Augmentation:** average a fixed set of crops

ResNet:

1. Resize image at 5 scales:  $\{224, 256, 384, 480, 640\}$
2. For each size, use 10  $224 \times 224$  crops: 4 corners + center, + flips

# Data augmentation

```
mean = (0.485, 0.456, 0.406)
std   = (0.229, 0.224, 0.225)
```

```
train_transform = transforms.Compose([
    transforms.RandomCrop(32, padding=4),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(mean, std),
])
test_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean, std),
])
```

```
train_dataset = datasets.CIFAR10(DATA_DIR, train=True, download=True, transform=train_transform)
test_dataset = datasets.CIFAR10(DATA_DIR, train=False, download=True, transform=test_transform)
class_names = train_dataset.classes
```

Let's train a  
model now!  
(See you tomorrow!)

```
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=5e-4)
EPOCHS = 3 if QUICK_RUN else 15

def run_epoch(model, loader, training):
    model.train(training)
    total_loss = 0.0
    total_correct = 0
    total_examples = 0

    for inputs, targets in loader:
        inputs, targets = inputs.to(device), targets.to(device)
        if training:
            optimizer.zero_grad(set_to_none=True)

            with torch.set_grad_enabled(training):
                logits = model(inputs)
                loss = criterion(logits, targets)
                if training:
                    loss.backward()
                    optimizer.step()

            total_loss += loss.item() * inputs.size(0)
            total_correct += (logits.argmax(dim=1) == targets).sum().item()
            total_examples += inputs.size(0)

    return total_loss / total_examples, total_correct / total_examples

history = {'train_loss': [], 'train_acc': [], 'test_loss': [], 'test_acc': []}
for epoch in range(1, EPOCHS + 1):
    train_loss, train_acc = run_epoch(model, train_loader, training=True)
    test_loss, test_acc = run_epoch(model, test_loader, training=False)
    history['train_loss'].append(train_loss)
    history['train_acc'].append(train_acc)
    history['test_loss'].append(test_loss)
    history['test_acc'].append(test_acc)
    print(
        f'Epoch {epoch:02d}/{EPOCHS} | '
        f'train loss {train_loss:.4f}, acc {train_acc:.1%} | '
        f'test loss {test_loss:.4f}, acc {test_acc:.1%}'
    )
```

# Next Lecture: Sequence data & RNNs

